

Cluster-Based LSTM Models for Forecasting Crime Rate Dynamics in London

Preparation

- [Github link](#)
- Number of words: 1487
- Runtime: 0.12 hours hours (*Memory 16 GB, CPU Apple M3*)
- Coding environment: SDS Docker
- License: this notebook is made available under the [Creative Commons Attribution license](#)
- Additional library:
 - **tensorflow**: An open-source deep learning framework developed by Google for building and training neural network models.
 - tensorflow 2.19.0
 - tensorflow-io-gcs-filesystem 0.37.1

Table of contents

1. [Introduction](#)
2. [Research questions](#)
3. [Data](#)
4. [Methodology](#)
5. [Results and discussion](#)
6. [Conclusion](#)
7. [References](#)

Introduction

[\[go back to the top \]](#)

Accurate forecasting of crime trends is essential for supporting data-informed public safety strategies. Recent research has examined a variety of modelling techniques for this purpose. Traditional statistical methods such as ARIMA have demonstrated some success in capturing historical crime trends (Vijayarani et al., 2021; Faujdar et al., 2021), while deep learning approaches—particularly Long Short-Term Memory (LSTM) networks—have been increasingly applied to time

series data due to their ability to model temporal dependencies (Safat et al., 2021). However, performance across different studies remains mixed, suggesting room for improvement in this model design and data structuring.

Meanwhile, spatial clustering techniques such as K-means have been widely used to identify regions that share similar crime patterns over time (Agarwal et al., 2013; Nath, 2006). These methods provide a foundation for tailoring prediction models to the unique temporal dynamics of different areas.

Building on these approaches, this study first applies K-means clustering to group London boroughs based on long-term crime rate trajectories. Then, for each identified cluster, an LSTM model is trained to capture and forecast area-specific crime trends. This research aims to test and analyze the model based on real-time data for crime prediction.

Research questions

[\[go back to the top \]](#)

To what extent can cluster-based LSTM models effectively capture and predict long-term temporal trends in crime rates across London boroughs?

Data

[\[go back to the top \]](#)

Comprehensive Data Science & ML Package

Core Packages for data analysis, visualization, machine learning, and geospatial modeling in this research.

```
In [1]: import time

start_time = time.time()
```

```
In [2]: # packages for file operations and basic functionality
import datetime
import os
import pickle
import subprocess
import sys

import contextily as ctx

# Geospatial analysis and mapping package
import geopandas as gpd
import matplotlib.colors as mcolors
import matplotlib.dates as mdates

# Data visualization packages
import matplotlib.pyplot as plt

# data manipulation packages
```

```

import numpy as np
import pandas as pd
import seaborn as sns
from matplotlib.colors import LinearSegmentedColormap
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA

# Machine learning packages for preprocessing, clustering and dimensionality reduction
from sklearn.preprocessing import MinMaxScaler, StandardScaler

# Time series analysis packages
from statsmodels.tsa.seasonal import seasonal_decompose

```

```

In [3]: # Deep learning framework and components
try:
    import tensorflow as tf
    from tensorflow.keras.callbacks import (
        EarlyStopping,
        ModelCheckpoint,
        ReduceLROnPlateau,
    )
    from tensorflow.keras.layers import (
        LSTM,
        Add,
        BatchNormalization,
        Bidirectional,
        Concatenate,
        Dense,
        Dropout,
        Input,
        LayerNormalization,
        MultiHeadAttention,
        Multiply,
    )
    from tensorflow.keras.models import Model, Sequential
except ImportError:
    print("TensorFlow is not installed. Installing now...")
    subprocess.check_call([sys.executable, "-m", "pip", "install", "tensorflow"])
    import tensorflow as tf
    from tensorflow.keras.callbacks import (
        EarlyStopping,
        ModelCheckpoint,
        ReduceLROnPlateau,
    )
    from tensorflow.keras.layers import (
        LSTM,
        Add,
        BatchNormalization,
        Bidirectional,
        Concatenate,
        Dense,
        Dropout,
        Input,
        LayerNormalization,
        MultiHeadAttention,
        Multiply,
    )
    from tensorflow.keras.models import Model, Sequential
tf.keras.backend.clear_session()

```

Overview of Datasets

- Input Dataset Metadata

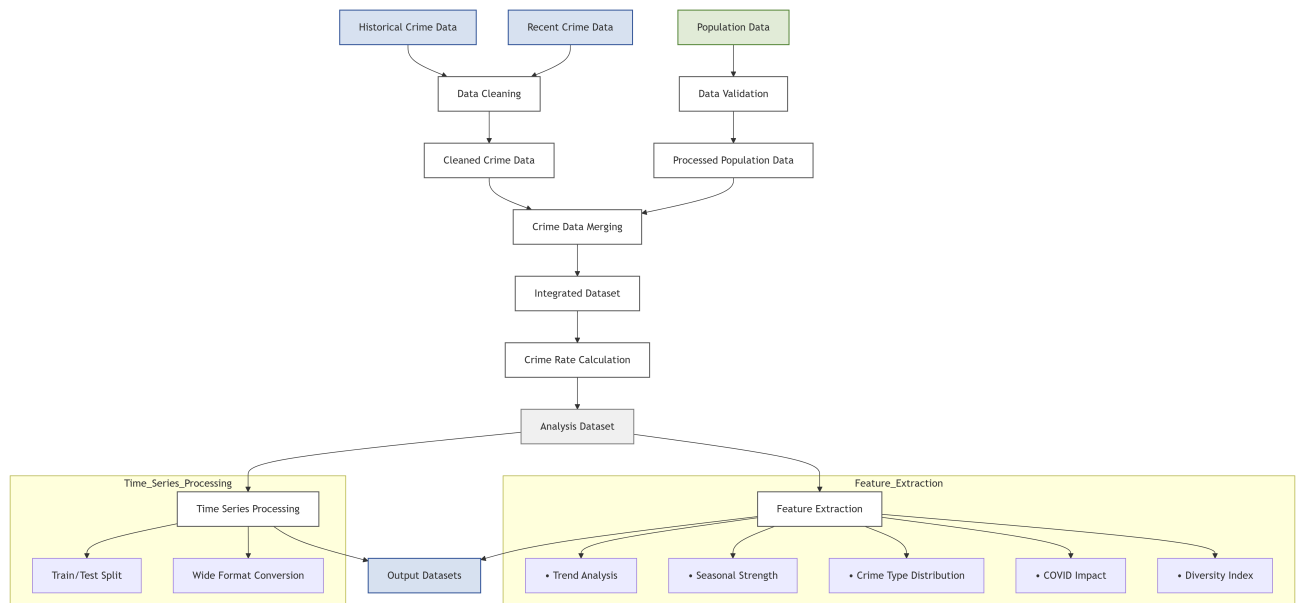
Dataset	Source	Contains	Counting method	Quantity
MPS Borough Level Crime (Historical)	Metropolitan Police Service	Date Crime Type Borough	Monthly	2010.4 - 2023.03
MPS Borough Level Crime (24 months)	Metropolitan Police Service	Date Crime Type Borough	Monthly	Last 24 months
Population data	GLA (Greater London Authority)	Date Population Count	Yearly	2010 - 2025

- Generated Datasets for Analysis

Dataset	Purpose	Contains	Format	Primary use
<code>london_yearly_crime_rates.csv</code>	Annual crime rate aggregation	Year Borough Crime Rate	Yearly	Trend feature extraction for long-term patterns
<code>london_monthly_crime_rates.csv</code>	Monthly detailed crime rates	Year Month Borough Crime Type Crime Rate	Monthly	Seasonal feature extraction for periodic patterns
<code>london_yearly_crime_type_rates.csv</code>	Crime type-specific yearly rates	Year Borough Crime Type Crime Rate	Yearly by type	Crime distribution feature extraction
<code>london_monthly_borough.csv</code>	Borough-level monthly aggregation	Date Borough Aggregate Crime Rate	Monthly	Time series feature extraction for modeling
<code>london_crime_train.csv</code>	Training dataset	Date Borough Crime Metrics	Monthly	Feature set for model training (pre-2023-03)
<code>london_crime_test.csv</code>	Test dataset	Date Borough Crime Metrics	Monthly	Feature set for model evaluation (post-2023-03)
<code>london_crime_ts_data.csv</code>	Time series analysis	Date Borough	Wide format	Feature extraction for time series

Dataset	Purpose	Contains	Format	Primary use
	data	Crime Rates by Type		components
<code>borough_features_for_clustering.csv</code>	Features for K-means clustering	Borough Trend Features Seasonal Features Crime Diversity	Feature matrix	Comprehensive feature set for borough clustering
<code>london_borough_clusters.csv</code>	Borough clustering results	Borough Cluster	Lookup table	Grouping reference for generating <code>lstm_by_cluster</code> directories

- *Overview of data pre-processing*



Data collection

In this research we use crime data from the London Datastore provided by the Metropolitan Police Service(London Metropolitan City Police, 2025). Based on the geographic level, the crime data on the website is presented in two formats: current data from the previous 24 months, and historical data from 2010. For This analysis, We selected the borough-level datasets, incorporating both the historical records and the most recent data until March 2025. We also use London population by borough provided by GLA (Greater London Authority, 2018) to caculate the crime rate for each borough.

```

In [4]: # Set path to data folder
data_dir = "data"

# Load data files
historical_file = os.path.join(data_dir, "MPS Borough Level Crime (Historical).csv")
recent_file = os.path.join(
    data_dir, "MPS Borough Level Crime (most recent 24 months).csv"

```

```
)
historical = pd.read_csv(historical_file)
recent = pd.read_csv(recent_file)
population_df = pd.read_csv("data/population.csv")
```

Data Pre-processing

Data Cleaning

Firstly, both the historical and recent crime datasets were cleaned by removing invalid borough entries like "Unknown" locations, standardizing text fields, and converting date columns to proper numeric format. Additionally, standardization was used to solve inconsistencies between the datasets, particularly with burglary classifications that used different terminology.

```
In [5]: # Define invalid borough names
invalid_boroughs = ["Unknown", "London Heathrow And London City Airports"]
invalid_boroughs_lower = [b.lower() for b in invalid_boroughs]

# Clean historical data
hist_clean = historical.copy()
hist_clean.columns = hist_clean.columns.str.strip()

# Clean text columns - trim whitespace and standardize case
text_cols = ["MajorText", "MinorText", "BoroughName"]
for col in text_cols:
    hist_clean[col] = hist_clean[col].astype(str).str.strip().str.title()

# Remove rows with invalid borough names
hist_clean = hist_clean[
    ~hist_clean["BoroughName"].str.lower().isin(invalid_boroughs_lower)
]

# Convert all date columns to numeric and handle missing values
date_cols = [col for col in hist_clean.columns if col not in text_cols]
for col in date_cols:
    hist_clean[col] = (
        pd.to_numeric(hist_clean[col], errors="coerce").fillna(0).astype(int)
    )

# Clean recent data
recent_clean = recent.copy()
recent_clean.columns = recent_clean.columns.str.strip()

# First convert all text to lowercase for consistent matching
for col in text_cols:
    recent_clean[col] = recent_clean[col].astype(str).str.strip().str.lower()

# Replace MinorText categories using lowercase matching (NOT MajorText)
minor_replace_dict = {
    "burglary - residential": "burglary in a dwelling",
    "burglary non-dwelling": "burglary business and community",
}
recent_clean["MinorText"] = recent_clean["MinorText"].replace(minor_replace_dict)

# Then convert back to title case for consistency
for col in text_cols:
    recent_clean[col] = recent_clean[col].str.title()
```

```

# Remove rows with invalid borough names
recent_clean = recent_clean[
    ~recent_clean["BoroughName"].str.lower().isin(invalid_boroughs_lower)
]

# Convert all date columns to numeric and handle missing values
recent_date_cols = [col for col in recent_clean.columns if col not in text_cols]
for col in recent_date_cols:
    recent_clean[col] = (
        pd.to_numeric(recent_clean[col], errors="coerce").fillna(0).astype(int)
    )

# Print unique MinorText values to verify replacement
print("\nUnique MinorText values in recent data after replacement:")
for val in sorted(recent_clean["MinorText"].unique()):
    if "Burglary" in val:
        print(f" - {val}")

# Save cleaned files
hist_clean.to_csv(os.path.join(data_dir, "historical_cleaned.csv"), index=False)
recent_clean.to_csv(os.path.join(data_dir, "recent_cleaned.csv"), index=False)

print(f"Historical data: {len(hist_clean)} rows after cleaning")
print(f"Recent data: {len(recent_clean)} rows after cleaning")

```

Unique MinorText values in recent data after replacement:

- Burglary Business And Community
- Burglary In A Dwelling

Historical data: 896 rows after cleaning

Recent data: 959 rows after cleaning

Merging Historical and Recent Crime Data

After data cleaning process, we integrated the historical crime records with the recent data to create a unified dataset for subsequent analysis.

In [6]:

```

# Merge datasets
# Use MajorText, MinorText and BoroughName as merge keys
merge_keys = ["MajorText", "MinorText", "BoroughName"]

# Create a combined dataframe with all unique combinations from both datasets
hist_keys = hist_clean[merge_keys].drop_duplicates()
recent_keys = recent_clean[merge_keys].drop_duplicates()
all_keys = pd.concat([hist_keys, recent_keys]).drop_duplicates()

print(f"Historical data unique key combinations: {len(hist_keys)}")
print(f"Recent data unique key combinations: {len(recent_keys)}")
print(f"Total unique key combinations: {len(all_keys)}")

# Create the base merged dataframe with all unique keys
merged_df = all_keys.copy()

# Merge with historical data
merged_df = pd.merge(merged_df, hist_clean, on=merge_keys, how="left")

# Merge with recent data
merged_df = pd.merge(
    merged_df, recent_clean, on=merge_keys, how="left", suffixes=("", "_recent")
)

```

```

# Fill NaN values with 0 for numeric columns
for col in date_cols + recent_date_cols:
    if col in merged_df.columns and col not in merge_keys:
        merged_df[col] = merged_df[col].fillna(0).astype(int)

# Save merged data
merged_file = os.path.join(data_dir, "london_crime_merged.csv")
merged_df.to_csv(merged_file, index=False)

print(f"Merge completed. Merged dataset contains {len(merged_df)} rows.")
print(f"Merged file saved as: {merged_file}")

# Display brief statistics of the results
print("\nMerged dataset statistics:")
print(f"Unique boroughs: {merged_df['BoroughName'].nunique()}")
print(f"Unique categories: {merged_df['MajorText'].nunique()}")
print(f"Unique minor categories: {merged_df['MinorText'].nunique()}")

```

Historical data unique key combinations: 896
 Recent data unique key combinations: 900
 Total unique key combinations: 907
 Merge completed. Merged dataset contains 966 rows.
 Merged file saved as: data/london_crime_merged.csv

Merged dataset statistics:
 Unique boroughs: 32
 Unique categories: 13
 Unique minor categories: 29

Calculating Crime Rate

Instead of using Crime count which can be misleading when comparing areas with different population sizes, we use crime rate for further analysis. After merging, we computed the crime rate monthly and yearly of 32 London boroughs (excluding the City of London) categorized by the major and minor crime type.

```

In [7]: def calculate_crime_rates():
        """
        Calculate crime rates for London boroughs
        Excludes City of London and handles all files in the data folder
        """

        # Read the data from data folder
        crime_df = pd.read_csv("data/london_crime_merged.csv")
        population_df = pd.read_csv("data/population.csv")

        print(f"Crime data dimensions: {crime_df.shape}")
        print(f"Population data dimensions: {population_df.shape}")

        # Remove City of London from population data
        population_df = population_df[
            ~population_df["Area_name"].str.lower().str.contains("city of london")
        ]
        print(
            f"Population data dimensions after removing City of London: {population_df.sh
        )

        # Process population data - convert strings to numeric values
        year_columns = [str(year) for year in range(2010, 2026)]
        for col in year_columns:

```



```

    if col in population_df.columns:
        # Remove possible commas and convert to numeric
        population_df[col] = pd.to_numeric(
            population_df[col].astype(str).str.replace(",", ""), errors="coerce"
        )

# Transform crime data from wide to long format
id_vars = ["MajorText", "MinorText", "BoroughName"]
value_vars = [col for col in crime_df.columns if col not in id_vars]

crime_long = pd.melt(
    crime_df,
    id_vars=id_vars,
    value_vars=value_vars,
    var_name="YearMonth",
    value_name="CrimeCount",
)

# Extract year and month from YearMonth column
crime_long["Year"] = crime_long["YearMonth"].str[:4].astype(int)
crime_long["Month"] = crime_long["YearMonth"].str[4:6].astype(int)
crime_long["Date"] = pd.to_datetime(crime_long["YearMonth"], format="%Y%m")

# Standardize borough names
crime_long["BoroughName_std"] = crime_long["BoroughName"].str.strip().str.lower()
population_df["Area_name_std"] = population_df["Area_name"].str.strip().str.lower

# Check matching between datasets
crime_boroughs = set(crime_long["BoroughName_std"])
pop_boroughs = set(population_df["Area_name_std"])

print(f"Number of boroughs in crime data: {len(crime_boroughs)}")
print(f"Number of boroughs in population data: {len(pop_boroughs)}")

# Find non-matching boroughs
crime_only = crime_boroughs - pop_boroughs
pop_only = pop_boroughs - crime_boroughs

if crime_only:
    print(f"Boroughs only in crime data: {crime_only}")
if pop_only:
    print(f"Boroughs only in population data: {pop_only}")

# Convert population data to long format for merging
population_long = pd.melt(
    population_df,
    id_vars=["Code", "Area_name", "Area_name_std"],
    value_vars=year_columns,
    var_name="Year",
    value_name="Population",
)
population_long["Year"] = population_long["Year"].astype(int)

# Merge crime data and population data
yearly_crime_summary = (
    crime_long.groupby(["BoroughName", "BoroughName_std", "Year"])["CrimeCount"]
    .sum()
    .reset_index()
)

# Merge summarized crime data with population data

```

```

yearly_merged = pd.merge(
    yearly_crime_summary,
    population_long,
    left_on=["BoroughName_std", "Year"],
    right_on=["Area_name_std", "Year"],
    how="left",
)

# Check for missing population data
missing_pop = yearly_merged["Population"].isna().sum()
print(f"Number of records with missing population data after merge: {missing_pop}")

if missing_pop > 0:
    print("Examples of missing population data:")
    print(
        yearly_merged[yearly_merged["Population"].isna()][
            ["BoroughName", "Year"]
        ].head()
    )

    # Filter out rows with missing population
    yearly_merged = yearly_merged.dropna(subset=["Population"])
    print(f"Removed {missing_pop} rows with missing population data")

# Calculate crime rates
# Per 1,000 population (commonly used for local analysis)
yearly_merged["CrimeRate_per_1000"] = (
    yearly_merged["CrimeCount"] / yearly_merged["Population"]
) * 1000

# Per 100,000 population (international standard)
yearly_merged["CrimeRate_per_100k"] = (
    yearly_merged["CrimeCount"] / yearly_merged["Population"]
) * 100000

# 11. Create monthly crime rates including crime types
crime_monthly = crime_long.copy()

monthly_merged = pd.merge(
    crime_monthly,
    population_long,
    left_on=["BoroughName_std", "Year"],
    right_on=["Area_name_std", "Year"],
    how="left",
)

# Filter out rows with missing population
monthly_merged = monthly_merged.dropna(subset=["Population"])

# Calculate monthly crime rates
monthly_merged["CrimeRate_per_1000"] = (
    monthly_merged["CrimeCount"] / monthly_merged["Population"]
) * 1000
monthly_merged["CrimeRate_per_100k"] = (
    monthly_merged["CrimeCount"] / monthly_merged["Population"]
) * 100000

# Create crime type specific yearly rates
crime_type_yearly = (
    crime_long.groupby(["BoroughName", "BoroughName_std", "Year", "MajorText"])[
        "CrimeCount"
    ]

```

```

    ]
    .sum()
    .reset_index()
)

crime_type_yearly = pd.merge(
    crime_type_yearly,
    population_long,
    left_on=["BoroughName_std", "Year"],
    right_on=["Area_name_std", "Year"],
    how="left",
)

# Filter out rows with missing population
crime_type_yearly = crime_type_yearly.dropna(subset=["Population"])

crime_type_yearly["CrimeRate_per_1000"] = (
    crime_type_yearly["CrimeCount"] / crime_type_yearly["Population"]
) * 1000
crime_type_yearly["CrimeRate_per_100k"] = (
    crime_type_yearly["CrimeCount"] / crime_type_yearly["Population"]
) * 100000

# Create datasets for time series analysis
monthly_borough = (
    monthly_merged.groupby(["BoroughName", "Date", "Year", "Month"])
    .agg(
        {
            "CrimeCount": "sum",
            "Population": "mean",
            "CrimeRate_per_1000": "sum",
            "CrimeRate_per_100k": "sum",
        }
    )
    .reset_index()
)

# Create training and test datasets (pre/post 2023-03)
split_date = pd.to_datetime("2023-03-01")

train_data = monthly_borough[monthly_borough["Date"] <= split_date]
test_data = monthly_borough[monthly_borough["Date"] > split_date]

yearly_merged.to_csv("data/london_yearly_crime_rates.csv", index=False)
monthly_merged.to_csv("data/london_monthly_crime_rates.csv", index=False)
crime_type_yearly.to_csv("data/london_yearly_crime_type_rates.csv", index=False)
monthly_borough.to_csv("data/london_monthly_borough.csv", index=False)

train_data.to_csv("data/london_crime_train.csv", index=False)
test_data.to_csv("data/london_crime_test.csv", index=False)

# Create a wide format dataset for time series analysis
ts_pivot = monthly_merged.pivot_table(
    index=["BoroughName", "Date"],
    columns=["MajorText"],
    values="CrimeRate_per_1000",
    aggfunc="sum",

```

```

).reset_index()

# Create a total crime rate column
ts_pivot["TotalCrimeRate"] = ts_pivot.iloc[:, 2:].sum(axis=1)

# Save the time series dataset
ts_pivot.to_csv("data/london_crime_ts_data.csv", index=False)

print("Saved files in data folder:")
print("- london_yearly_crime_rates.csv (yearly overall crime rates)")
print("- london_monthly_crime_rates.csv (monthly detailed crime rates)")
print("- london_yearly_crime_type_rates.csv (crime type specific yearly rates)")
print(
    "- london_monthly_borough.csv (monthly aggregated data for time series analysis)"
)
print("- london_crime_train.csv (training dataset, pre-2023-03)")
print("- london_crime_test.csv (test dataset, post-2023-03)")
print("- london_crime_ts_data.csv (wide format data for time series analysis)")

return {
    "yearly_rates": yearly_merged,
    "monthly_rates": monthly_merged,
    "crime_type_rates": crime_type_yearly,
    "monthly_borough": monthly_borough,
    "train_data": train_data,
    "test_data": test_data,
    "ts_data": ts_pivot,
}

if __name__ == "__main__":
    results = calculate_crime_rates()

    # Basic statistical analysis
    yearly_data = results["yearly_rates"]

    print("\n==== Crime Rate Statistics =====")
    print(f>Data year range: {yearly_data['Year'].min()} - {yearly_data['Year'].max()}

    # Statistics for the most recent year
    recent_year = yearly_data["Year"].max()
    recent_data = yearly_data[yearly_data["Year"] == recent_year]

    print(f"\nCrime rate statistics for {recent_year}:")
    print(
        f"Average crime rate (per 1000 people): {recent_data['CrimeRate_per_1000'].mean()}"
    )
    print(
        f"Highest crime rate borough: {recent_data.loc[recent_data['CrimeRate_per_1000'] == recent_data['CrimeRate_per_1000'].max()]['Borough']}"
    )
    print(
        f"Lowest crime rate borough: {recent_data.loc[recent_data['CrimeRate_per_1000'] == recent_data['CrimeRate_per_1000'].min()]['Borough']}"
    )

```

Crime data dimensions: (966, 183)
Population data dimensions: (33, 18)
Population data dimensions after removing City of London: (32, 18)
Number of boroughs in crime data: 32
Number of boroughs in population data: 32
Number of records with missing population data after merge: 0
Saved files in data folder:

- london_yearly_crime_rates.csv (yearly overall crime rates)
- london_monthly_crime_rates.csv (monthly detailed crime rates)
- london_yearly_crime_type_rates.csv (crime type specific yearly rates)
- london_monthly_borough.csv (monthly aggregated data for time series analysis)
- london_crime_train.csv (training dataset, pre-2023-03)
- london_crime_test.csv (test dataset, post-2023-03)
- london_crime_ts_data.csv (wide format data for time series analysis)

==== Crime Rate Statistics ====

Data year range: 2010 – 2025

Crime rate statistics for 2025:

Average crime rate (per 1000 people): 22.49

Highest crime rate borough: Westminster (78.27)

Lowest crime rate borough: Richmond Upon Thames (12.90)

London Crime Rate Annual overview

As an initial overview, the top 8 boroughs with the highest average crime rates from 2010 to 2024 were showed below. Data from 2025 is excluded for now, as only the first three months are currently available.

```
In [8]: # Set up matplotlib
plt.style.use("seaborn-v0_8-whitegrid") # Use a clean, modern style

# Set file paths
yearly_file = "data/london_yearly_crime_rates.csv"

# Load data
yearly_df = pd.read_csv(yearly_file)
yearly_df = yearly_df[yearly_df["Year"] < 2025]
plt.figure(figsize=(15, 10))
boroughs = yearly_df["BoroughName"].unique()

top_boroughs = (
    yearly_df.groupby("BoroughName")["CrimeRate_per_1000"]
    .mean()
    .nlargest(8)
    .index.tolist()
)

print(
    f"Showing top 8 boroughs with highest average crime rates: {'', '.join(top_boroughs)'"
)

for borough in top_boroughs:
    borough_data = yearly_df[yearly_df["BoroughName"] == borough]
    plt.plot(
        borough_data["Year"],
        borough_data["CrimeRate_per_1000"],
        marker="o",
        linewidth=2,
        label=borough,
    )
```

```
plt.title("Annual Crime Rate Trends by London Borough (per 1,000 people)", fontsize=12)
plt.xlabel("Year", fontsize=12)
plt.ylabel("Crime Rate (per 1,000 people)", fontsize=12)
plt.grid(True, linestyle="--", alpha=0.7)
plt.legend(loc="best")
plt.xticks(yearly_df["Year"].unique())
plt.tight_layout()

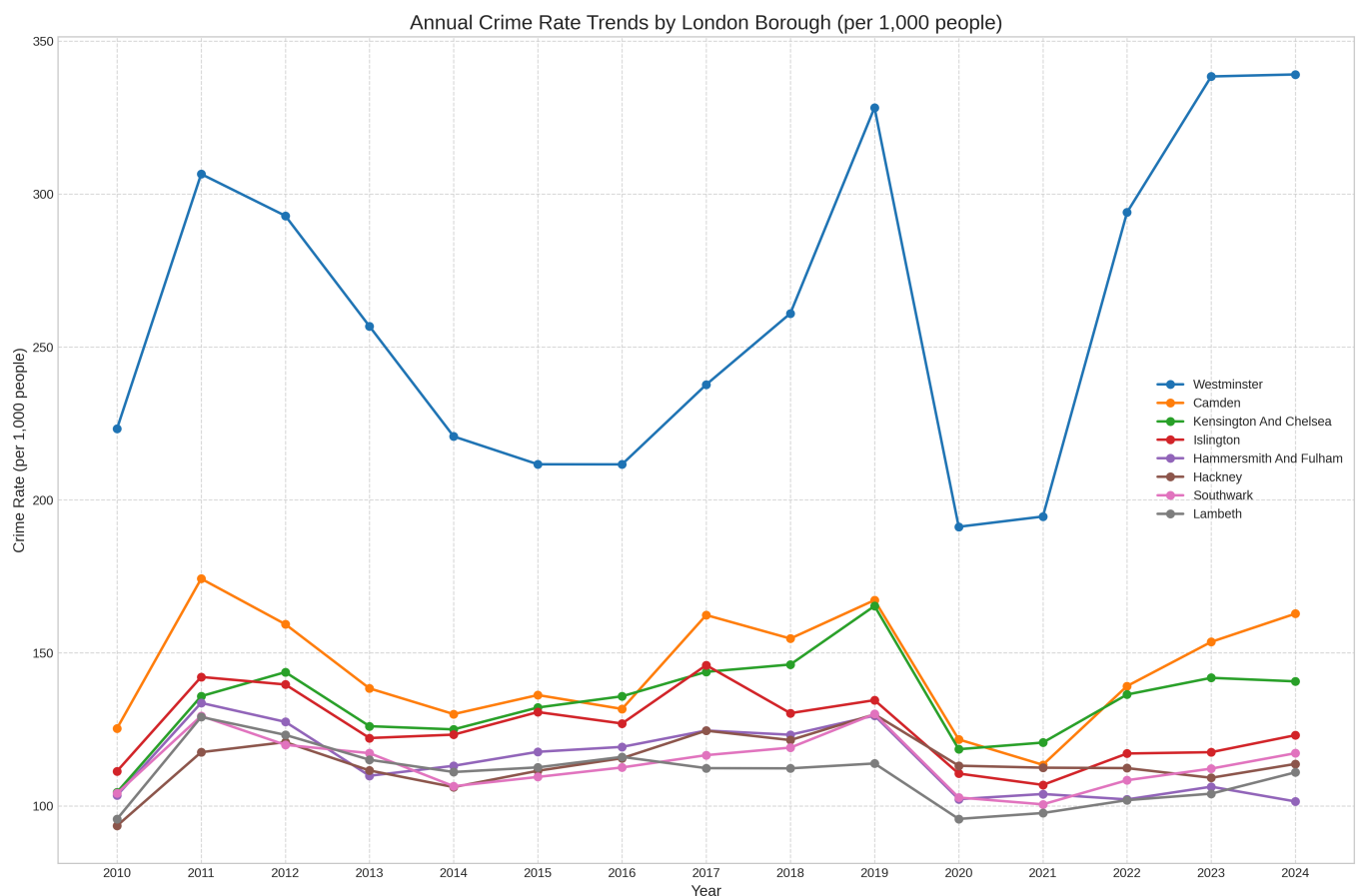
plt.savefig("data/london_borough_crime_rates.png", dpi=300)

from IPython.display import Image, display

display(Image("data/london_borough_crime_rates.png"))

plt.close()
```

Showing top 8 boroughs with highest average crime rates: Westminster, Camden, Kensington And Chelsea, Islington, Hammersmith And Fulham, Hackney, Southwark, Lambeth



From the line chart above, We could notice that Westminster consistently stands out with significantly higher rates compared to other boroughs. Other boroughs like Camden, Kensington and Chelsea, and Southwark show moderate increases over time, while areas such as Hackney and Islington have remained relatively stable.

Feature Extraction for K-means

To support LSTM-based crime forecasting, we extracted features from monthly and yearly crime data to capture each London borough's crime profile.

The feature extraction for K-means clustering captures each London borough's crime profile through 24 features, including trend slopes, seasonal strength, COVID-19 impact, and crime type

distribution and diversity, which summarizing both temporal patterns and crime composition across boroughs.

```
In [9]: def extract_clustering_features():
        """
        Extract features from London crime data for K-means clustering.
        These features capture various aspects of crime patterns across boroughs.
        """

        # Load the relevant datasets
        monthly_borough = pd.read_csv("data/london_monthly_borough.csv")
        yearly_crime_rates = pd.read_csv("data/london_yearly_crime_rates.csv")
        crime_type_rates = pd.read_csv("data/london_yearly_crime_type_rates.csv")

        monthly_borough["Date"] = pd.to_datetime(monthly_borough["Date"])

        # Extract trend features
        trend_features = []

        for borough in monthly_borough["BoroughName"].unique():
            borough_data = monthly_borough[
                monthly_borough["BoroughName"] == borough
            ].sort_values("Date")

            if len(borough_data) >= 24: # Need enough data points
                # Get borough time series
                borough_ts = pd.Series(
                    borough_data["CrimeRate_per_1000"].values,
                    index=pd.to_datetime(borough_data["Date"]),
                )

                # Calculate trend slope using linear regression
                x = np.arange(len(borough_ts))
                if len(x) > 0:
                    slope = np.polyfit(x, borough_ts, 1)[0]
                else:
                    slope = 0

                # Calculate time series statistics
                avg_rate = borough_ts.mean()
                volatility = borough_ts.std()
                max_rate = borough_ts.max()
                min_rate = borough_ts.min()
                range_rate = max_rate - min_rate

                # Try to calculate seasonal features
                try:
                    # Decompose time series
                    decomposition = seasonal_decompose(
                        borough_ts, model="multiplicative", period=12
                    )
                    seasonal = decomposition.seasonal
                    residual = decomposition.resid

                    # Calculate seasonal strength
                    if (seasonal.dropna() + residual.dropna()).var() > 0:
                        seasonal_strength = 1 - (
                            residual.dropna().var()
                            / (seasonal.dropna() + residual.dropna()).var()
                        )
```

```

else:
    seasonal_strength = 0

    # Calculate monthly patterns
    monthly_avg = borough_data.groupby("Month")["CrimeRate_per_1000"].mean()
    peak_month = monthly_avg.idxmax()
    monthly_variation = (
        monthly_avg.std() / monthly_avg.mean()
        if monthly_avg.mean() > 0
        else 0
    )

except Exception as e:
    print(f"Error calculating seasonal features for {borough}: {e}")
    seasonal_strength = 0
    peak_month = 0
    monthly_variation = 0

# Calculate COVID impact
pre_covid = borough_data[borough_data["Date"] < "2020-03-01"]["CrimeRate_per_1000"]
pre_covid = pre_covid.mean()
during_covid = borough_data[
    (borough_data["Date"] >= "2020-03-01")
    & (borough_data["Date"] <= "2021-07-01")
]["CrimeRate_per_1000"]
during_covid = during_covid.mean()
post_covid = borough_data[borough_data["Date"] > "2021-07-01"]["CrimeRate_per_1000"]
post_covid = post_covid.mean()

covid_impact = (
    (during_covid - pre_covid) / pre_covid if pre_covid > 0 else 0
)
recovery_rate = (
    (post_covid - during_covid) / during_covid if during_covid > 0 else 0
)

# Calculate recent trend (last 2 years)
recent_data = borough_data[borough_data["Date"] >= "2021-01-01"]
if len(recent_data) > 12:
    x_recent = np.arange(len(recent_data))
    recent_rates = recent_data["CrimeRate_per_1000"].values
    recent_slope = np.polyfit(x_recent, recent_rates, 1)[0]
else:
    recent_slope = 0

# Add features to the list
trend_features.append(
    {
        "BoroughName": borough,
        "trend_slope": recent_slope,
        "recent_trend": recent_slope,
        "avg_crime_rate": avg_rate,
        "volatility": volatility,
        "range_crime_rate": range_rate,
        "seasonal_strength": seasonal_strength,
        "peak_month": peak_month,
        "monthly_variation": monthly_variation,
        "covid_impact": covid_impact,
        "recovery_rate": recovery_rate,
    }
)

```



```

    )

    # Create a dataframe of trend features
    trend_df = pd.DataFrame(trend_features)

    # Extract crime type distribution features
    # Calculate percentage of each crime type per borough
    type_distribution = (
        crime_type_rates.groupby(["BoroughName", "MajorText"])["CrimeCount"]
        .sum()
        .reset_index()
    )
    type_distribution["total_crimes"] = type_distribution.groupby("BoroughName")["CrimeCount"]
    ].transform("sum")
    type_distribution["percentage"] = (
        type_distribution["CrimeCount"] / type_distribution["total_crimes"] * 100
    )

    type_pivot = type_distribution.pivot(
        index="BoroughName", columns="MajorText", values="percentage"
    ).fillna(0)

    # Calculate crime type diversity (entropy)
    def entropy(row):
        non_zero = row[row > 0] / 100
        if len(non_zero) > 0:
            return -sum(non_zero * np.log2(non_zero))
        return 0

    type_pivot["crime_diversity"] = type_pivot.apply(entropy, axis=1)

    # Combine all features
    clustering_features = pd.merge(
        trend_df, type_pivot.reset_index(), on="BoroughName", how="left"
    )

    # Handle any remaining NaN values
    clustering_features = clustering_features.fillna(0)

    clustering_features.to_csv("data/borough_features_for_clustering.csv", index=False)

    print("Features saved to: data/borough_features_for_clustering.csv")

    return clustering_features

if __name__ == "__main__":
    # Run feature extraction
    features = extract_clustering_features()

    # Show summary of extracted features
    print("\nExtracted features summary:")
    print(f"Number of boroughs: {len(features)}")
    print(f"Number of features: {features.shape[1] - 1}") # Excluding BoroughName

    # Display the first few rows
    print("\nSample features (first 5 boroughs):")
    print(
        features[
            [

```

```
        "BoroughName",
        "trend_slope",
        "avg_crime_rate",
        "seasonal_strength",
        "crime_diversity",
    ]
].head()
)
```

Features saved to: data/borough_features_for_clustering.csv

Extracted features summary:

Number of boroughs: 32

Number of features: 24

Sample features (first 5 boroughs):

	BoroughName	trend_slope	avg_crime_rate	seasonal_strength	\
0	Barking And Dagenham	-0.008001	7.877738	0.255284	
1	Barnet	-0.006141	6.290617	0.334462	
2	Bexley	0.004469	5.064671	0.239705	
3	Brent	-0.004740	7.711720	0.364080	
4	Bromley	-0.003130	6.090528	0.242077	

	crime_diversity
0	2.956873
1	2.868944
2	2.925747
3	2.971684
4	2.896793

Methodology

[\[go back to the top \]](#)

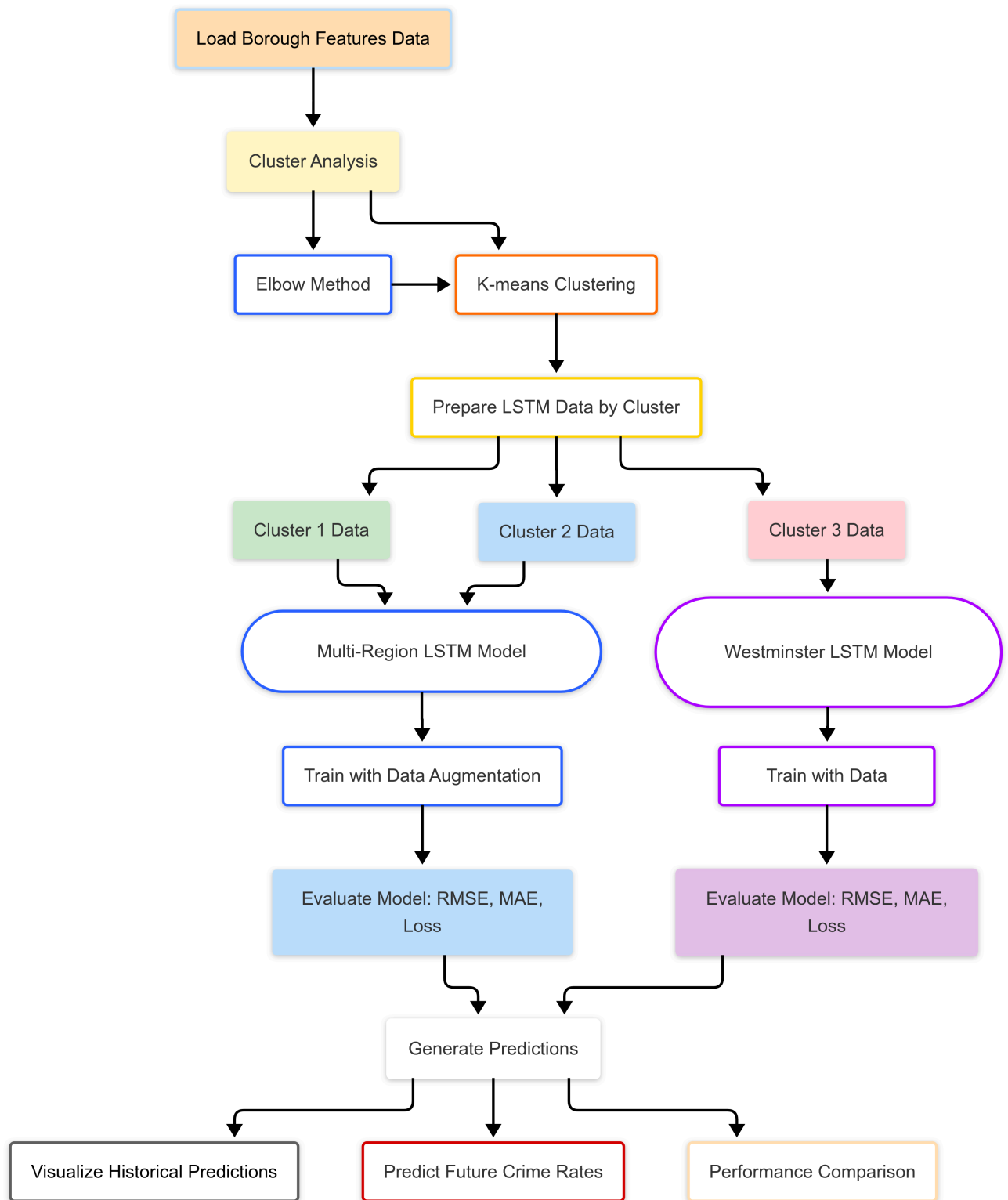
K-means Clustering

To identify regions exhibiting similar temporal crime patterns and spatial characteristics, K-Means clustering was applied to features extracted from historical crime data. Temporal features included metrics like mean, variance, and seasonality, while spatial features captured location and urban context. The optimal number of clusters was selected using the elbow method.

LSTM Model

Following the clustering process, a dedicated Long Short-Term Memory (LSTM) model was developed for each identified region group to capture the temporal dependencies in crime rates more effectively. LSTM models use a gate mechanism to retain relevant information over time, allowing them to learn patterns in sequential data as it progresses through each step (Mishra et al., 2024). This study trained LSTM models on monthly crime data from January 2010 to March 2023, using a sliding window to retain temporal structure. Data were normalized before training to improve convergence. Model performance was assessed on the period from April 2023 to March 2025 using RMSE and MAE.

- Workflow of Analysis



Analysis

Elbow Method

To determine the optimal number of clusters for K-Means, the Elbow Method was applied by computing the within-cluster sum of squares (inertia) for k values ranging from 1 to 15. The resulting curve helped identify points where adding more clusters yields diminishing returns in variance reduction. These “elbow” points, estimated via second-order differences, suggest an efficient trade-off between model complexity and compactness.

```

In [10]: os.makedirs("data/visualizations", exist_ok=True)

print("Loading and preparing data...")
features_df = pd.read_csv("data/borough_features_for_clustering.csv")
borough_names = features_df["BoroughName"].values
# Prepare features for clustering
features_for_clustering = features_df.drop("BoroughName", axis=1)
feature_names = features_for_clustering.columns
# Standardize features
scaler = StandardScaler()
features_scaled = scaler.fit_transform(features_for_clustering)
# Calculate inertia (within-cluster sum of squares) for different k values
print("Calculating inertia for k=1 to k=15...")
inertia_values = []
k_range = range(1, 16)
for k in k_range:
    if k == 1:
        inertia = np.sum(np.square(features_scaled - np.mean(features_scaled, axis=0)))
        inertia_values.append(inertia)
    else:
        kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
        kmeans.fit(features_scaled)
        inertia_values.append(kmeans.inertia_)
    print(f"For k={k}, inertia: {inertia_values[-1]:.2f}")
# Create enhanced Elbow plot
plt.figure(figsize=(12, 8))
sns.set_style("whitegrid")
# Plot inertia values with attractive styling
plt.plot(
    k_range,
    inertia_values,
    "o-",
    linewidth=2.5,
    markersize=10,
    color="#2980b9",
    markerfacecolor="#e74c3c",
    markeredgecolor="white",
    markeredgewidth=2,
)
# Add annotations for key points
deltas = np.diff(inertia_values)
delta_deltas = np.diff(deltas)
elbow_indices = np.argsort(np.abs(delta_deltas))[:3] + 1 # +1 due to double diff
for idx in elbow_indices:
    plt.annotate(
        f"k={k_range[idx]}\n({inertia_values[idx]:.1f})",
        (k_range[idx], inertia_values[idx]),
        textcoords="offset points",
        xytext=(0, 15),
        ha="center",
        fontsize=10,
        fontweight="bold",
        bbox=dict(boxstyle="round,pad=0.3", fc="yellow", alpha=0.3),
    )
plt.title("Elbow Method for Optimal Number of Clusters", fontsize=16, pad=20)
plt.xlabel("Number of Clusters (k)", fontsize=14, labelpad=10)
plt.ylabel("Inertia (Within-Cluster Sum of Squares)", fontsize=14, labelpad=10)
plt.xticks(k_range, fontsize=12)
plt.yticks(fontsize=12)
plt.grid(True, alpha=0.3, linestyle="--")
# Add vertical line ONLY at k=3 (chosen cluster)

```

```

plt.axvline(x=3, color="gray", linestyle="--", alpha=0.5)
# Add a note about interpretation
plt.figtext(
    0.5,
    0.01,
    "",
    ha="center",
    fontsize=12,
    bbox={"facecolor": "lightgrey", "alpha": 0.5, "pad": 5},
)
plt.tight_layout()
plt.savefig(
    "data/visualizations/elbow_method_analysis.png", dpi=300, bbox_inches="tight"
)
print("Elbow method analysis saved to: data/visualizations/elbow_method_analysis.png")

```

Loading and preparing data...

Calculating inertia for k=1 to k=15...

For k=1, inertia: 768.00

For k=2, inertia: 524.92

For k=3, inertia: 407.09

For k=4, inertia: 346.22

For k=5, inertia: 309.23

For k=6, inertia: 272.01

For k=7, inertia: 248.02

For k=8, inertia: 220.93

For k=9, inertia: 201.09

For k=10, inertia: 174.31

For k=11, inertia: 155.82

For k=12, inertia: 140.97

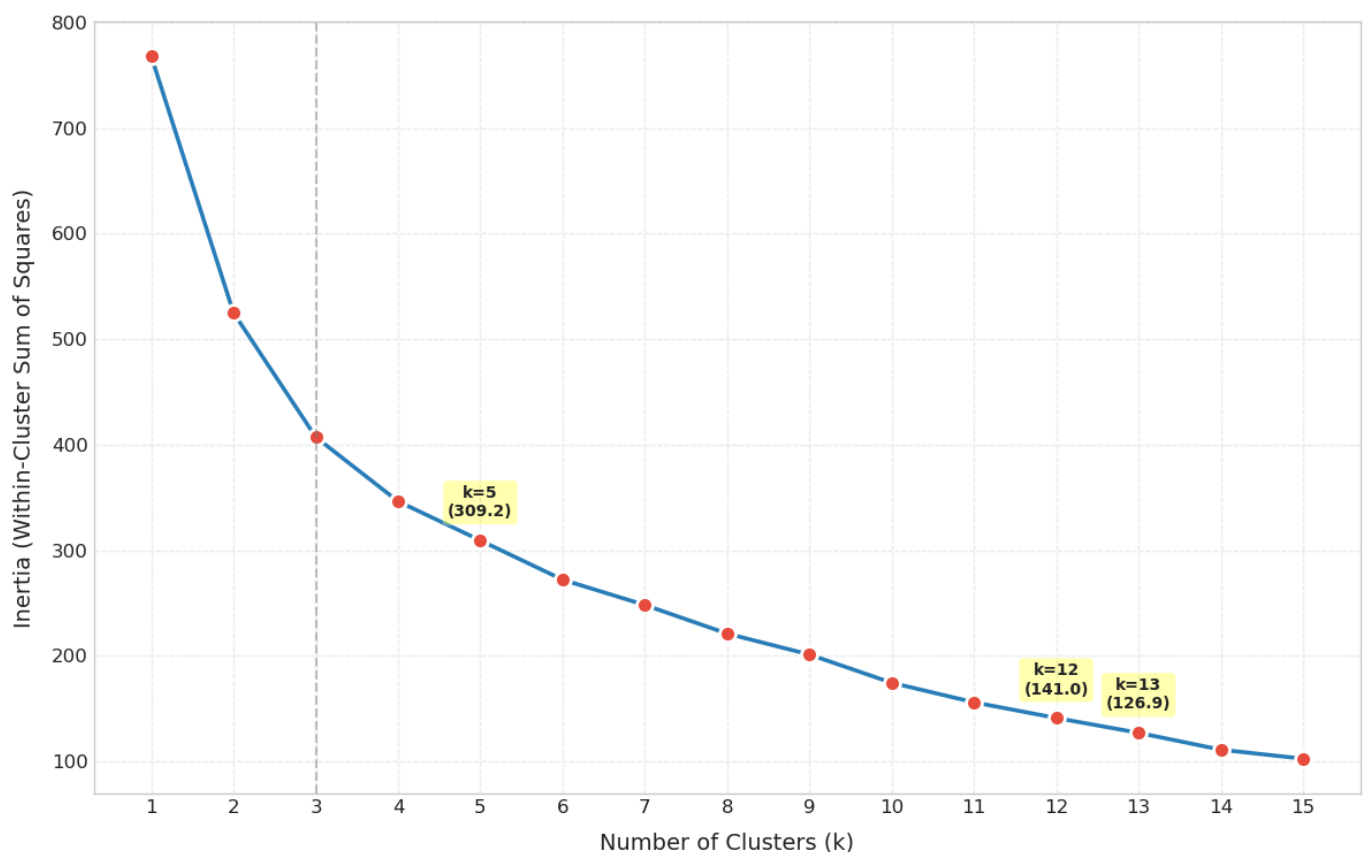
For k=13, inertia: 126.90

For k=14, inertia: 110.90

For k=15, inertia: 102.39

Elbow method analysis saved to: data/visualizations/elbow_method_analysis.png

Elbow Method for Optimal Number of Clusters



According to the Elbow Method, $k=3$ was selected as the optimal number of clusters. While lower inertia values were observed with higher k , the rate of decrease significantly diminished beyond $k=3$, indicating that additional clusters contributed marginal explanatory power.

Perform K-means Clustering

To perform K-means clustering, a mix of time-series and structural features was capucured to represent each borough's crime profile. This included long-term and recent trends in crime rates, seasonal strength, and how crime changed during and after the COVID-19 period. Also, the composition of crime types in each borough were considered, calculating the percentage share of each major category and an overall diversity score using entropy.

```
In [11]: # Set cluster count – change this based on your analysis
K = 3

# Load and prepare data
print(f"Performing K-means clustering with k={K}...")
features_df = pd.read_csv("data/borough_features_for_clustering.csv")
borough_names = features_df["BoroughName"].values

# Prepare and standardize features
features = features_df.drop("BoroughName", axis=1)
feature_names = features.columns
scaler = StandardScaler()
features_scaled = scaler.fit_transform(features)

# Perform K-means clustering
kmeans = KMeans(n_clusters=K, random_state=42, n_init=10)
cluster_labels = kmeans.fit_predict(features_scaled)

# Add cluster labels to features dataframe
clustered_data = features_df.copy()
clustered_data["cluster"] = cluster_labels

# Create borough-to-cluster mapping
cluster_mapping = pd.DataFrame(
    {"BoroughName": borough_names, "cluster": cluster_labels}
)

# Save clustering results
os.makedirs("data/visualizations", exist_ok=True)
clustered_data.to_csv("data/london_borough_clusters.csv", index=False)
cluster_mapping.to_csv("data/london_cluster_mapping.csv", index=False)

# Save cluster centers for interpretation
cluster_centers_original = pd.DataFrame(
    scaler.inverse_transform(kmeans.cluster_centers_), columns=feature_names
)
cluster_centers_original.to_csv("data/london_cluster_centers.csv", index=False)

# Create PCA visualization
pca = PCA(n_components=2)
features_pca = pca.fit_transform(features_scaled)
variance_explained = pca.explained_variance_ratio_ * 100

plt.figure(figsize=(12, 10))
colors = plt.cm.viridis(np.linspace(0, 1, K))
```

```

# Plot clusters
for i in range(K):
    cluster_points = features_pca[cluster_labels == i]
    plt.scatter(
        cluster_points[:, 0],
        cluster_points[:, 1],
        s=80,
        alpha=0.8,
        c=[colors[i]],
        edgecolor="white",
        linewidth=1,
        label=f"Cluster {i+1} ({np.sum(cluster_labels == i)} boroughs)",
    )

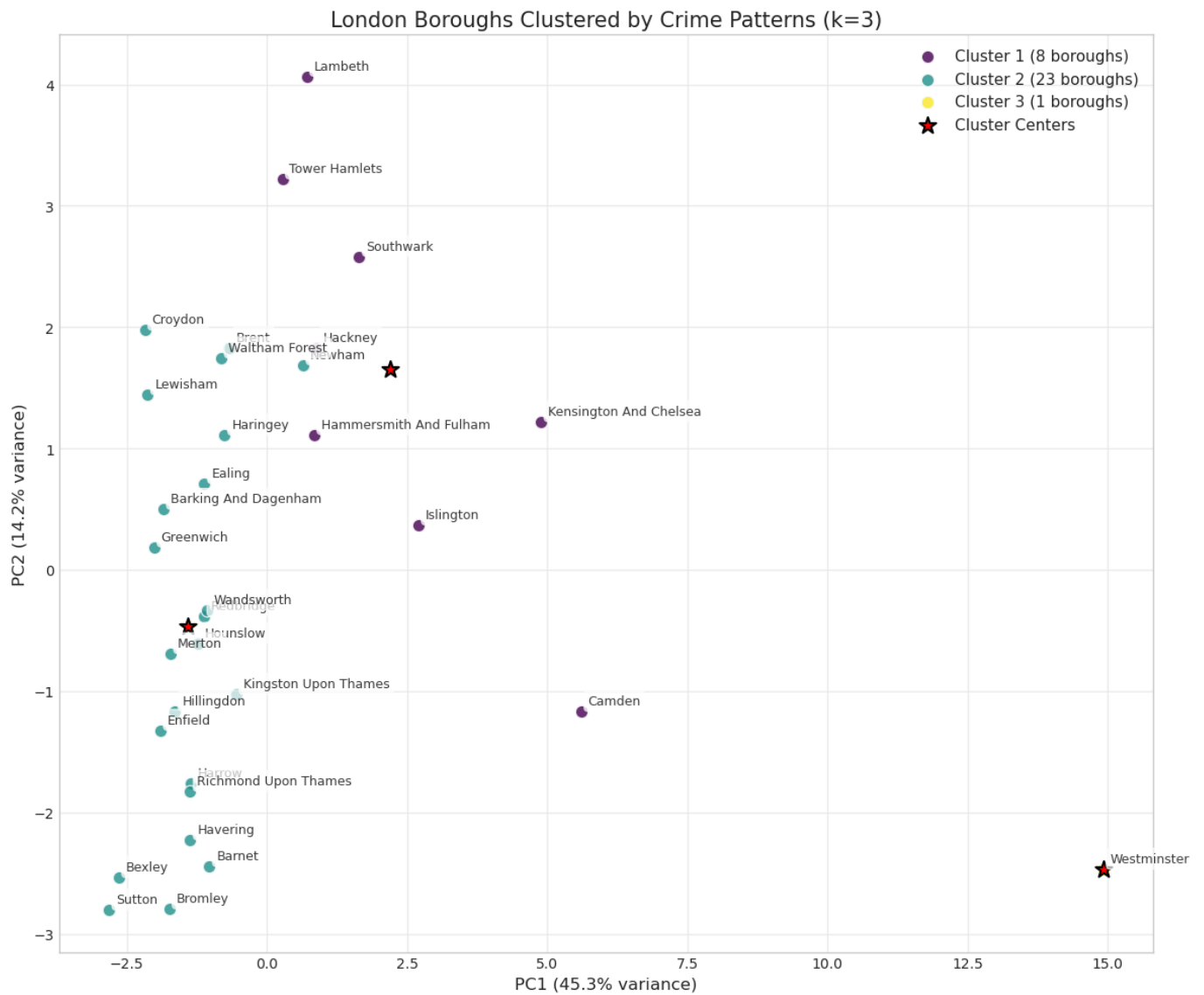
# Add borough labels
for i, borough in enumerate(borough_names):
    plt.annotate(
        borough,
        (features_pca[i, 0], features_pca[i, 1]),
        fontsize=9,
        alpha=0.9,
        xytext=(5, 5),
        textcoords="offset points",
        bbox=dict(boxstyle="round,pad=0.3", fc="white", alpha=0.7),
    )

# Plot cluster centers
centers_pca = pca.transform(kmeans.cluster_centers_)
plt.scatter(
    centers_pca[:, 0],
    centers_pca[:, 1],
    s=150,
    marker="*",
    c="red",
    edgecolor="black",
    linewidth=1.5,
    label="Cluster Centers",
)

plt.title(f"London Boroughs Clustered by Crime Patterns (k={K})", fontsize=15)
plt.xlabel(f"PC1 ({variance_explained[0]:.1f}% variance)", fontsize=12)
plt.ylabel(f"PC2 ({variance_explained[1]:.1f}% variance)", fontsize=12)
plt.legend(fontsize=11)
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("data/visualizations/borough_clusters_pca.png", dpi=300)

```

Performing K-means clustering with k=3...



```
In [12]: # Load shapefile and cluster mapping
print("Creating map visualization with swapped blue and green colors...")
london_shape = gpd.read_file("data/ESRI/London_Borough_Excluding_MHW.shp")
clusters = pd.read_csv("data/london_cluster_mapping.csv")

# Standardize borough names for matching
london_shape["NAME_lower"] = london_shape["NAME"].str.lower().str.strip()
clusters["BoroughName_lower"] = clusters["BoroughName"].str.lower().str.strip()

# Merge data
london_clusters = london_shape.merge(
    clusters, left_on="NAME_lower", right_on="BoroughName_lower", how="left"
)

# Check for unmatched boroughs
unmatched = london_clusters[london_clusters["cluster"].isna()]
if not unmatched.empty:
    print(f"Warning: {len(unmatched)} unmatched boroughs: {unmatched['NAME'].tolist()}")

# Create directory for output
os.makedirs("data/visualizations", exist_ok=True)

# Create visually appealing map
fig, ax = plt.subplots(figsize=(15, 12))

# Get number of clusters
```



```

n_clusters = london_clusters["cluster"].nunique()

# Create a color palette with swapped blue and green regions
elegant_distinct_colors = [
    "#c3d7b3",
    "#acd0e9",
    "#e4c1c1",
    "#8bbcaa",
    "#d1c39f",
    "#728fb8",
    "#b5a0c9",
    "#dab89f",
    "#6d9ea3",
    "#c99f9f",
]

# Ensure we have enough colors for all clusters
while len(elegant_distinct_colors) < n_clusters:
    elegant_distinct_colors.extend(elegant_distinct_colors)
elegant_distinct_colors = elegant_distinct_colors[:n_clusters]

cmap = LinearSegmentedColormap.from_list(
    "elegant_distinct_swapped", elegant_distinct_colors, N=n_clusters
)

london_clusters.plot(
    column="cluster",
    cmap=cmap,
    linewidth=0.8,
    edgecolor="white",
    ax=ax,
    legend=True,
    categorical=True,
    legend_kwds={
        "title": "Crime Pattern Clusters",
        "bbox_to_anchor": (1.05, 0.95),
        "loc": "upper left",
        "fontsize": 12,
        "labels": ["1", "2", "3"], # Modified labels: 0→1, 1→2, 2→3
    },
)

for idx, row in london_clusters.iterrows():
    centroid = row["geometry"].centroid

    ax.text(
        centroid.x,
        centroid.y,
        row["NAME"],
        fontsize=8,
        ha="center",
        va="center",
        fontweight="bold",
        bbox=dict(boxstyle="round,pad=0.3", fc="white", ec="#a0a0a0", alpha=0.75),
    )

# Add basemap for context
try:
    ctx.add_basemap(
        ax,
        crs=london_clusters.crs.to_string(),

```

```

        source=ctx.providers.CartoDB.Positron,
        alpha=0.5,
    )
except Exception as e:
    print(f"Could not add basemap: {e}")

plt.title("London Borough Crime Pattern Clusters", fontsize=18, pad=20)
plt.axis("off")
plt.tight_layout()

output_path = "data/visualizations/london_clusters_map_swapped_colors.png"
plt.savefig(output_path, dpi=300, bbox_inches="tight")
print(f"Map with swapped blue and green colors saved to: {output_path}")

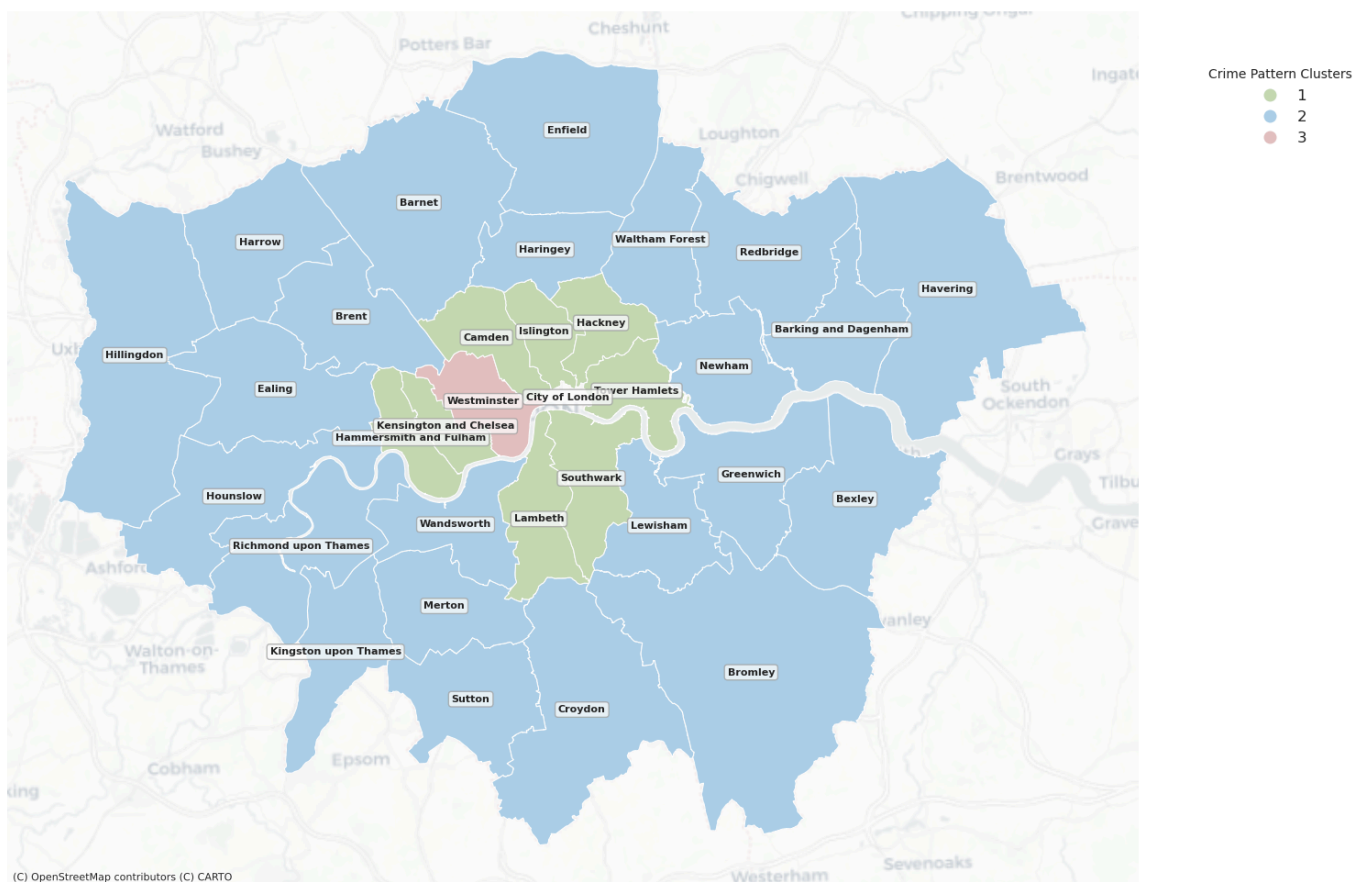
```

Creating map visualization with swapped blue and green colors...

Warning: 1 unmatched boroughs: ['City of London']

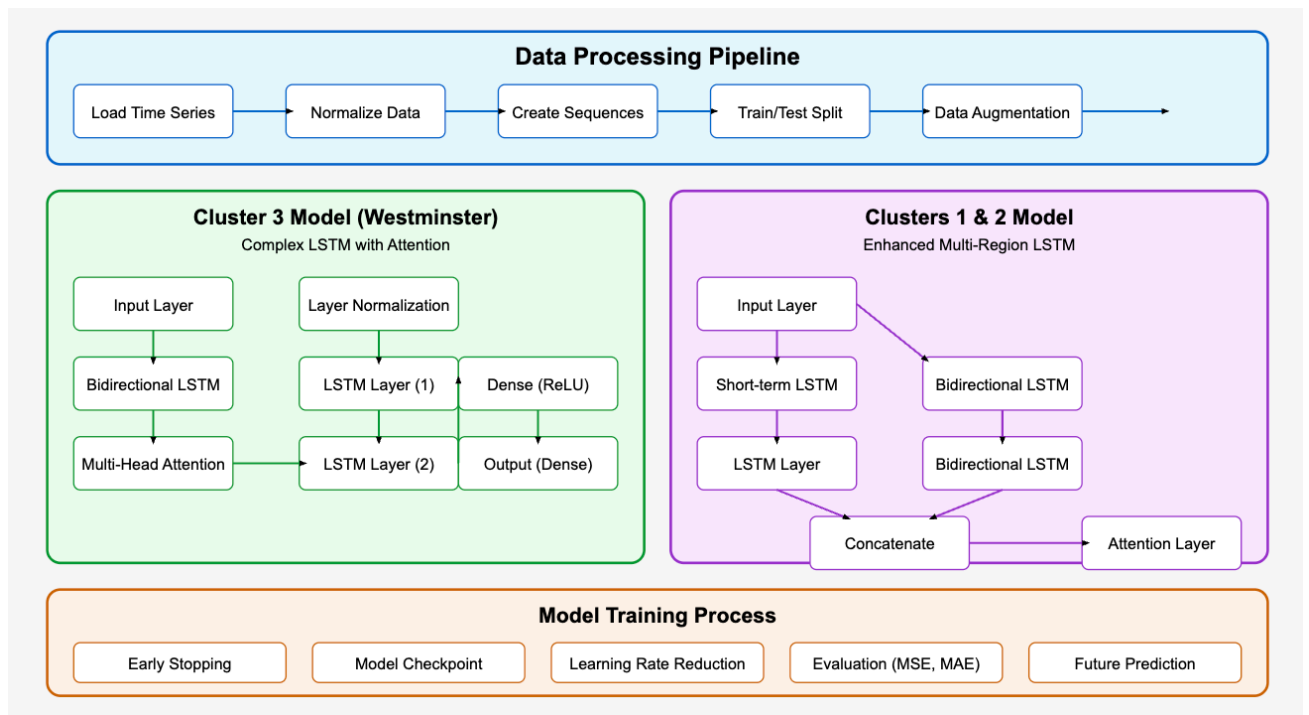
Map with swapped blue and green colors saved to: data/visualizations/london_clusters_map_swapped_colors.png

London Borough Crime Pattern Clusters



The K-means clustering analysis of London boroughs revealed three distinct crime pattern clusters. Cluster 1 comprising eight inner London boroughs characterized by dense urban environments; Cluster 2 containing 23 outer London boroughs showing relatively homogeneous crime patterns despite their geographical spread; and Cluster 3 consisting solely of Westminster, which displays unique crime characteristics. Based on the initial data, Westminster exhibits significantly higher crime rates compared to other London areas.

LSTM Model



LSTM data preparation

To model temporal crime patterns, monthly crime rates were averaged within each spatial cluster to form representative time series. A 12-month sliding window was used to predict the following month. Models were trained on data from April 2010 to March 2023 and tested on April 2023 to March 2025 using a rolling forecast.

```

In [13]: def prepare_lstm_data(
    train_data, test_data, output_dir, name_prefix, value_column, seq_length=12
):
    """
    Prepare sequence data for LSTM model with complete test sequence coverage

    Args:
        train_data: Training data DataFrame
        test_data: Testing data DataFrame
        output_dir: Output directory
        name_prefix: File name prefix
        value_column: Value column name
        seq_length: Sequence length (default is 12 months)
    """
    print(f"Train data shape: {train_data.shape}, Test data shape: {test_data.shape}")

    # Use MinMaxScaler for data normalization
    scaler = MinMaxScaler()
    train_scaled = pd.DataFrame(
        scaler.fit_transform(train_data[[value_column]]), columns=[value_column]
    )
    test_scaled = pd.DataFrame(
        scaler.transform(test_data[[value_column]]), columns=[value_column]
    )

    # Create training sequences using traditional method
    def create_train_sequences(data, seq_length=12):
        """Create sequence data for LSTM training"""
        X, y = [], []
  
```

```

data_array = data[value_column].values
for i in range(len(data_array) - seq_length):
    X.append(data_array[i : i + seq_length])
    y.append(data_array[i + seq_length])
return np.array(X), np.array(y)

# Create test sequences using a rolling forecast approach
def create_rolling_test_sequences(train_data, test_data, seq_length=12):
    """
    Create test sequences using rolling forecast approach
    This ensures we have predictions for ALL test months
    """
    train_values = train_data[value_column].values
    test_values = test_data[value_column].values

    X, y = [], []
    input_sequence = train_values[-seq_length:]

    # For each test value, create a prediction sequence and then update the sequence
    for i in range(len(test_values)):
        X.append(input_sequence.copy())

        y.append(test_values[i])

        input_sequence = np.append(input_sequence[1:], test_values[i])

    return np.array(X), np.array(y)

X_train, y_train = create_train_sequences(train_scaled, seq_length)

X_test, y_test = create_rolling_test_sequences(
    train_scaled, test_scaled, seq_length
)

print(f"X_train shape: {X_train.shape}, y_train shape: {y_train.shape}")
print(f"X_test shape: {X_test.shape}, y_test shape: {y_test.shape}")
print(f"Number of test months: {len(test_data)}")
print(f"Number of test sequences created: {len(X_test)}")

# Verify that we have a sequence for each test month
if len(X_test) != len(test_data):
    print(
        f"WARNING: Number of test sequences ({len(X_test)}) does not match number
    )
else:
    print(
        f"SUCCESS: Created a sequence for each test month ({len(test_data)} month
    )

# Check if training data is sufficient
if len(X_train) == 0:
    print(f"Warning: Not enough training data for {name_prefix}")
    return

np.save(f"{output_dir}/X_train.npy", X_train)
np.save(f"{output_dir}/y_train.npy", y_train)
np.save(f"{output_dir}/X_test.npy", X_test)
np.save(f"{output_dir}/y_test.npy", y_test)

# Save scaler for later denormalization
with open(f"{output_dir}/scaler.pkl", "wb") as f:

```

```

        pickle.dump(scaler, f)

    # Save the original test data for date reference
    test_data.to_csv(f"{output_dir}/original_test.csv", index=False)

    # Save sequence details for debugging
    with open(f"{output_dir}/sequence_details.txt", "w") as f:
        f.write(
            f"Train period: {train_data['Date'].min()} to {train_data['Date'].max()}, {l
        )
        f.write(
            f"Test period: {test_data['Date'].min()} to {test_data['Date'].max()}, {l
        )
        f.write(f"X_train shape: {X_train.shape}, y_train shape: {y_train.shape}\n")
        f.write(f"X_test shape: {X_test.shape}, y_test shape: {y_test.shape}\n")

    # Save first few sequences for verification
    f.write("\nFirst 3 test sequences (if available):\n")
    for i in range(min(3, len(X_test))):
        sequence_dates = test_data["Date"].iloc[i : i + 1].tolist()
        f.write(f"Sequence {i}, Target date: {sequence_dates[0]}\n")

    # Save last few sequences for verification
    f.write("\nLast 3 test sequences (if available):\n")
    for i in range(max(0, len(X_test) - 3), len(X_test)):
        sequence_dates = test_data["Date"].iloc[i : i + 1].tolist()
        f.write(f"Sequence {i}, Target date: {sequence_dates[0]}\n")

# Main program starts
# Load time series data (organized by borough and crime type)
ts_data = pd.read_csv("data/london_crime_ts_data.csv")
ts_data["Date"] = pd.to_datetime(ts_data["Date"])

# Load clustering results
clusters_data = pd.read_csv("data/london_borough_clusters.csv")
# Get number of clusters
K = clusters_data["cluster"].nunique()
print(f"Found {K} borough clusters")

# Create LSTM data directory
lstm_dir = "data/lstm_by_cluster"
os.makedirs(lstm_dir, exist_ok=True)

# Ensure we have data through March 2025
max_date = ts_data["Date"].max()
expected_end_date = pd.to_datetime("2025-03-01")

if max_date < expected_end_date:
    print(
        f"Warning: Data only goes up to {max_date.strftime('%Y-%m-%d')}, not {expecte
    )

# Prepare data for each cluster
print("Preparing LSTM data by cluster...")
for cluster_id in range(K):
    print(f"Processing cluster {cluster_id}...")

    cluster_boroughs = clusters_data[clusters_data["cluster"] == cluster_id][
        "BoroughName"
    ].tolist()

```

```

cluster_dir = f"{lstm_dir}/cluster_{cluster_id+1}"
os.makedirs(cluster_dir, exist_ok=True)

cluster_data = ts_data[ts_data["BoroughName"].isin(cluster_boroughs)]

agg_data = cluster_data.groupby("Date")["TotalCrimeRate"].mean().reset_index()
agg_data.rename(columns={"TotalCrimeRate": "CrimeRate"}, inplace=True)

# Split into training and testing data - use April 2023 as the split date
split_date = pd.to_datetime("2023-04-01")
train_data = agg_data[agg_data["Date"] < split_date]
test_data = agg_data[agg_data["Date"] >= split_date]

print(
    f"Train period: {train_data['Date'].min()} to {train_data['Date'].max()}, {len(train_data)} days"
)
print(
    f"Test period: {test_data['Date'].min()} to {test_data['Date'].max()}, {len(test_data)} days"
)

# Save overall data for this cluster
agg_data.to_csv(f"{cluster_dir}/total_crime_rate.csv", index=False)
train_data.to_csv(f"{cluster_dir}/train.csv", index=False)
test_data.to_csv(f"{cluster_dir}/test.csv", index=False)

# Create sequence data for LSTM
prepare_lstm_data(train_data, test_data, cluster_dir, "total", "CrimeRate", 12)

print(f"LSTM data for cluster {cluster_id+1} prepared successfully")

print("\nCluster Summary:")
for cluster_id in range(K):
    cluster_boroughs = clusters_data[clusters_data["cluster"] == cluster_id][
        "BoroughName"
    ].tolist()
    print(f"Cluster {cluster_id+1}: {len(cluster_boroughs)} boroughs")
    print(f"    {' '.join(cluster_boroughs)}")

print(f"\nLSTM model data saved to {lstm_dir}/")

```

```
Found 3 borough clusters
Preparing LSTM data by cluster...
Processing cluster 0...
Train period: 2010-04-01 00:00:00 to 2023-03-01 00:00:00, 156 months
Test period: 2023-04-01 00:00:00 to 2025-03-01 00:00:00, 24 months
Train data shape: (156, 2), Test data shape: (24, 2)
X_train shape: (144, 12), y_train shape: (144,)
X_test shape: (24, 12), y_test shape: (24,)
Number of test months: 24
Number of test sequences created: 24
SUCCESS: Created a sequence for each test month (24 months)
LSTM data for cluster 1 prepared successfully
Processing cluster 1...
Train period: 2010-04-01 00:00:00 to 2023-03-01 00:00:00, 156 months
Test period: 2023-04-01 00:00:00 to 2025-03-01 00:00:00, 24 months
Train data shape: (156, 2), Test data shape: (24, 2)
X_train shape: (144, 12), y_train shape: (144,)
X_test shape: (24, 12), y_test shape: (24,)
Number of test months: 24
Number of test sequences created: 24
SUCCESS: Created a sequence for each test month (24 months)
LSTM data for cluster 2 prepared successfully
Processing cluster 2...
Train period: 2010-04-01 00:00:00 to 2023-03-01 00:00:00, 156 months
Test period: 2023-04-01 00:00:00 to 2025-03-01 00:00:00, 24 months
Train data shape: (156, 2), Test data shape: (24, 2)
X_train shape: (144, 12), y_train shape: (144,)
X_test shape: (24, 12), y_test shape: (24,)
Number of test months: 24
Number of test sequences created: 24
SUCCESS: Created a sequence for each test month (24 months)
LSTM data for cluster 3 prepared successfully
```

Cluster Summary:

Cluster 1: 8 boroughs

Camden, Hackney, Hammersmith And Fulham, Islington, Kensington And Chelsea, Lambeth, Southwark, Tower Hamlets

Cluster 2: 23 boroughs

Barking And Dagenham, Barnet, Bexley, Brent, Bromley, Croydon, Ealing, Enfield, Greenwich, Haringey, Harrow, Havering, Hillingdon, Hounslow, Kingston Upon Thames, Lewisham, Merton, Newham, Redbridge, Richmond Upon Thames, Sutton, Waltham Forest, Wandsworth

Cluster 3: 1 boroughs

Westminster

LSTM model data saved to data/lstm_by_cluster/

Tranning LSTM model

To improve temporal prediction accuracy, separate LSTM models were trained for each cluster.

Cluster 3 (Westminster) uses a deeper architecture with bidirectional LSTM and attention mechanisms to better capture complex temporal dynamics. For Clusters 1 and 2, a multi-path LSTM design was applied to account for both short-term and long-term patterns.

Data augmentation techniques such as noise injection and temporal shifts—were used in clusters with limited or unstable data to enhance model robustness. While the models focus on time series, clustering ensures that regional differences are indirectly considered through tailored training for each group.

```

In [14]: # Set random seed for reproducibility
np.random.seed(42)
tf.random.set_seed(42)

# Apply seaborn style for more beautiful plots
sns.set_style("whitegrid")
plt.rcParams.update(
    {
        "font.family": "sans-serif",
        "font.sans-serif": ["Arial", "DejaVu Sans"],
        "figure.figsize": (14, 7),
        "axes.labelsize": 12,
        "axes.titlesize": 14,
        "xtick.labelsize": 10,
        "ytick.labelsize": 10,
        "legend.fontsize": 10,
    }
)

# Enhanced model architectures with further specialization for each cluster
def build_multi_region_lstm_model(input_shape, units=96, dropout_rate=0.25):
    """
    Enhanced specialized model for multi-region clusters (Clusters 1 & 2)
    Designed to better capture volatility patterns in crime data
    """
    inputs = Input(shape=input_shape)

    # Parallel processing paths to capture different patterns
    # Captures short-term fluctuations
    short_term = LSTM(
        units,
        return_sequences=True,
        recurrent_regularizer=tf.keras.regularizers.l2(0.0005),
    )(inputs)
    short_term = LSTM(units // 2, return_sequences=False, recurrent_dropout=0.2)(
        short_term
    )
    short_term = Dropout(dropout_rate)(short_term)

    # Captures broader patterns with bidirectional processing
    broad_pattern = Bidirectional(LSTM(units // 2, return_sequences=True))(inputs)
    broad_pattern = Bidirectional(LSTM(units // 4))(broad_pattern)
    broad_pattern = Dropout(dropout_rate)(broad_pattern)

    # Combine the paths
    combined = Concatenate()([short_term, broad_pattern])

    # Add attention mechanism to focus on important features
    attention_layer = Dense(units, activation="tanh")(combined)
    attention_weights = Dense(1, activation="sigmoid")(attention_layer)
    attention_output = Multiply()([combined, attention_weights])

    # Feature extraction layers
    x = Dense(
        64,
        activation="relu",
        kernel_regularizer=tf.keras.regularizers.l1_l2(l1=0.0005, l2=0.0005),
    )(attention_output)
    x = BatchNormalization()(x)
    x = Dropout(0.3)(x)

```



```

# Add residual connection to preserve important features
x = Dense(32, activation="relu")(x)
x = Add()(
    [x, Dense(32, activation="linear")(attention_output)]
) # Residual connection

# Final prediction layer
outputs = Dense(1)(x)

model = Model(inputs=inputs, outputs=outputs)

optimizer = tf.keras.optimizers.Adam(learning_rate=0.001)
model.compile(optimizer=optimizer, loss="mse", metrics=["mae"])

return model

# MODEL
def build_lstm_model(input_shape, units=128, dropout_rate=0.2):
    """
    Complex LSTM model with attention mechanism – for Cluster 3 (Westminster)
    This is your original model that works well for Cluster 3
    """
    # Using Functional API for Attention mechanism
    inputs = Input(shape=input_shape)

    # Bidirectional LSTM
    x = Bidirectional(LSTM(units, return_sequences=True))(inputs)

    # Self-attention mechanism (MultiHeadAttention)
    query = x
    key = x
    value = x
    attention_output = MultiHeadAttention(num_heads=4, key_dim=units)(query, value, k

    # Residual connection and layer normalization
    x = Concatenate()([x, attention_output])
    x = LayerNormalization()(x)

    # Additional LSTM layers
    x = LSTM(units, return_sequences=True)(x)
    x = Dropout(dropout_rate)(x)
    x = LSTM(units)(x)
    x = Dense(32, activation="relu")(x)
    outputs = Dense(1)(x)

    model = Model(inputs=inputs, outputs=outputs)
    model.compile(optimizer="adam", loss="mse", metrics=["mae"])
    return model

def train_and_evaluate(cluster_id, model_dir='data/lstm_by_cluster'):
    """
    Train and evaluate LSTM model with region-aware parameters
    """
    try:
        # Build paths
        cluster_dir = f'{model_dir}/cluster_{cluster_id}'

        X_train = np.load(f'{cluster_dir}/X_train.npy')

```

```

y_train = np.load(f'{cluster_dir}/y_train.npy')
X_test = np.load(f'{cluster_dir}/X_test.npy')
y_test = np.load(f'{cluster_dir}/y_test.npy')

# Load original test data for dates
test_data = pd.read_csv(f'{cluster_dir}/original_test.csv')
test_data['Date'] = pd.to_datetime(test_data['Date'])

print(f"X_train shape: {X_train.shape}, y_train shape: {y_train.shape}")
print(f"X_test shape: {X_test.shape}, y_test shape: {y_test.shape}")

with open(f'{cluster_dir}/scaler.pkl', 'rb') as f:
    scaler = pickle.load(f)

X_train = X_train.reshape(X_train.shape[0], X_train.shape[1], 1)
X_test = X_test.reshape(X_test.shape[0], X_test.shape[1], 1)

# CLUSTER 3
if cluster_id == 3:
    # ===== CODE FOR CLUSTER 3 =====
    print(f"Using complex attention model for Cluster 3 (Westminster)")
    model = build_lstm_model((X_train.shape[1], 1))

    # parameters for Cluster 3
    epochs = 50
    batch_size = 32
    patience = 10
    val_split = 0.2

    print("Using standard training for Cluster 3 (Westminster)")

    import time
    unique_id = int(time.time())

    # callbacks for Cluster 3
    callbacks = [
        EarlyStopping(monitor='val_loss', patience=patience, restore_best_weights=True),
        ModelCheckpoint(
            filepath=f'{cluster_dir}/best_model_{unique_id}.keras',
            monitor='val_loss',
            save_best_only=True,
            verbose=1
        )
    ]

    # Ensure proper shape for y_train
    if len(y_train.shape) == 1:
        y_train = y_train.reshape(-1, 1)

    # Regular training for Cluster 3
    history = model.fit(
        X_train, y_train,
        epochs=epochs,
        batch_size=batch_size,
        validation_split=val_split,
        callbacks=callbacks,
        verbose=2
    )

# CLUSTERS 1 & 2
else:
    print(f"Using enhanced multi-region specialized model for Cluster {cluster_id}")

```

```

model = build_multi_region_lstm_model((X_train.shape[1], 1))

epochs = 120
batch_size = 4
patience = 25
val_split = 0.1

print("Using specialized training for multi-region cluster")

import time
unique_id = int(time.time())

callbacks = [
    EarlyStopping(
        monitor='val_loss',
        patience=patience,
        restore_best_weights=True,
        min_delta=0.0001
    ),
    ModelCheckpoint(
        filepath=f'{cluster_dir}/best_model_{unique_id}.keras',
        monitor='val_loss',
        save_best_only=True,
        verbose=1
    ),
    ReduceLROnPlateau(
        monitor='val_loss',
        factor=0.5,
        patience=patience//3,
        min_lr=0.00001,
        verbose=1
    )
]

# Enhanced data augmentation for multi-region clusters
X_train_aug = X_train.copy()

# Ensure proper shape for y_train
if len(y_train.shape) == 1:
    y_train = y_train.reshape(-1, 1)

y_train_aug = y_train.copy()

# Standard noise augmentation
for i in range(2):
    noise_factor = 0.03 if i == 0 else 0.05
    noise = np.random.normal(0, noise_factor, X_train.shape)
    X_train_aug = np.vstack([X_train_aug, X_train + noise])
    y_train_aug = np.vstack([y_train_aug, y_train])

# Targeted high-volatility augmentation
volatility_noise = np.zeros_like(X_train)
for i in range(X_train.shape[0]):
    local_std = np.std(X_train[i])
    volatility_noise[i] = np.random.normal(0, local_std * 0.5, X_train.sh

X_train_aug = np.vstack([X_train_aug, X_train + volatility_noise])
y_train_aug = np.vstack([y_train_aug, y_train])

# Add small random time shifts
for shift in [1, 2]:

```

```

X_shifted = np.zeros_like(X_train)
for i in range(X_train.shape[0]):
    X_shifted[i, shift:, :] = X_train[i, :-shift, :]
    X_shifted[i, :shift, :] = X_train[i, 0:1, :]

X_train_aug = np.vstack([X_train_aug, X_shifted])
y_train_aug = np.vstack([y_train_aug, y_train])

print(f"Advanced augmented training data: {X_train.shape} → {X_train_aug.shape}")
print(f"Augmented target data: {y_train.shape} → {y_train_aug.shape}")

# Verify shapes match before model.fit
if X_train_aug.shape[0] != y_train_aug.shape[0]:
    print(f"ERROR: Shape mismatch after augmentation! X: {X_train_aug.shape}, y: {y_train_aug.shape}")
    min_samples = min(X_train_aug.shape[0], y_train_aug.shape[0])
    X_train_aug = X_train_aug[:min_samples]
    y_train_aug = y_train_aug[:min_samples]
    print(f"Adjusted shapes to match: X: {X_train_aug.shape[0]}, y: {y_train_aug.shape[0]}")

# Train with augmented data
history = model.fit(
    X_train_aug, y_train_aug,
    epochs=epochs,
    batch_size=batch_size,
    validation_split=val_split,
    callbacks=callbacks,
    verbose=2,
    shuffle=True
)

# Ensure y_test is properly shaped for evaluation
if len(y_test.shape) == 1:
    y_test = y_test.reshape(-1, 1)

# Evaluate model
print(f"Evaluating cluster {cluster_id} LSTM model...")
loss, mae = model.evaluate(X_test, y_test, verbose=0)
print(f"Test Loss: {loss:.4f}")
print(f"Test MAE: {mae:.4f}")

# Make predictions
train_predictions = model.predict(X_train, verbose=0)
test_predictions = model.predict(X_test, verbose=0)

# Ensure shapes match for inverse_transform
if len(y_train.shape) == 1:
    y_train = y_train.reshape(-1, 1)
if len(y_test.shape) == 1:
    y_test = y_test.reshape(-1, 1)

# Denormalize predictions
train_predictions_original = scaler.inverse_transform(train_predictions)
test_predictions_original = scaler.inverse_transform(test_predictions)
y_train_original = scaler.inverse_transform(y_train)
y_test_original = scaler.inverse_transform(y_test)

# Calculate RMSE
train_rmse = np.sqrt(np.mean((train_predictions_original - y_train_original)**2))
test_rmse = np.sqrt(np.mean((test_predictions_original - y_test_original)**2))
print(f"Training RMSE: {train_rmse:.4f}")
print(f"Testing RMSE: {test_rmse:.4f}")

```

```

# Create the main prediction visualization
visualize_predictions(
    train_predictions_original, y_train_original,
    test_predictions_original, y_test_original,
    cluster_id, cluster_dir, test_data
)

# Save training history
pd.DataFrame(history.history).to_csv(f'{cluster_dir}/training_history.csv', i

# Create evaluation dictionary
evaluation = {
    'test_loss': loss,
    'test_mae': mae,
    'train_rmse': train_rmse,
    'test_rmse': test_rmse,
    'model_type': "Enhanced Multi-Region Model" if cluster_id in [1, 2] else
}

future_predictions = predict_future(
    model, X_test, scaler, cluster_id, cluster_dir, test_data
)

final_model_path = f'{cluster_dir}/final_model_{unique_id}.keras'
try:
    model.save(final_model_path)
    print(f"Final model saved to {final_model_path}")
except Exception as e:
    print(f"Warning: Could not save final model: {str(e)}")

return model, history, evaluation

finally:
    import gc
    from tensorflow.keras import backend as K

    print(f"Cleaning up resources for cluster {cluster_id}...")
    K.clear_session()
    gc.collect()
    import time
    time.sleep(2)

def get_model_type_name(cluster_id):
    """Helper function to get model type name for reporting"""
    if cluster_id == 3:
        return "Complex (Bidirectional LSTM with Attention)"
    elif cluster_id == 1:
        return "Simple (Single LSTM)"
    elif cluster_id == 2:
        return "Medium (Stacked LSTM)"
    else:
        return "Medium (Default)"

def visualize_predictions(
    train_pred, train_true, test_pred, test_true, cluster_id, output_dir, test_data
):
    """
    Create main prediction visualization with proper date handling

```

```

"""
# Create figure
plt.figure(figsize=(14, 7))

# Load training data to get dates
train_data = pd.read_csv(f"{output_dir}/train.csv")
train_data["Date"] = pd.to_datetime(train_data["Date"])

# Set the exact date boundaries using actual data
split_date = test_data["Date"].min()
end_date = test_data["Date"].max()

# For training dates:
# Training predictions start after the sequence_length
sequence_length = 12 # This is our sequence length
train_dates = train_data["Date"].iloc[
    sequence_length : sequence_length + len(train_pred)
]

# Ensure we have the right number of training dates
if len(train_dates) != len(train_pred):
    print(
        f"WARNING: Adjusting training dates: {len(train_dates)} dates vs {len(train_pred)} predictions"
    )
    if len(train_dates) > len(train_pred):
        train_dates = train_dates[: len(train_pred)]
    else:
        # Create a date range to match the number of predictions
        first_date = train_data["Date"].iloc[sequence_length]
        train_dates = pd.date_range(
            start=first_date, periods=len(train_pred), freq="MS"
        )

# For test dates: Use the actual dates from test data
test_dates_array = test_data["Date"].values

# Ensure we have the right number of test dates
if len(test_dates_array) != len(test_pred):
    print(
        f"WARNING: Test dates ({len(test_dates_array)}) and predictions ({len(test_pred)}) do not match"
    )
    if len(test_dates_array) > len(test_pred):
        test_dates_array = test_dates_array[: len(test_pred)]
    else:
        # Extend with a date range if needed
        last_date = test_dates_array[-1]
        missing_dates = pd.date_range(
            start=pd.to_datetime(last_date) + pd.DateOffset(months=1),
            periods=len(test_pred) - len(test_dates_array),
            freq="MS",
        )
        test_dates_array = np.append(test_dates_array, missing_dates)

actual_start_date = (
    train_dates.iloc[0] if isinstance(train_dates, pd.Series) else train_dates[0]
)
print(f"Actual data start date: {actual_start_date}")

# Plot training set with dates
plt.plot(train_dates, train_true, "b-", label="Training Actual", linewidth=1.5)
plt.plot(train_dates, train_pred, "g--", label="Training Predicted", linewidth=2)

```

```

# Plot test set with dates – ensure one-to-one mapping between dates and predicti
plt.plot(test_dates_array, test_true, "r-", label="Test Actual", linewidth=1.5)
plt.plot(
    test_dates_array,
    test_pred,
    "orange",
    linestyle="--",
    label="Test Predicted",
    linewidth=2,
)

# Add training/test boundary line – use exact split_date
plt.axvline(x=split_date, color="gray", linestyle="--")
y_min, y_max = plt.ylim()
plt.text(
    split_date + pd.Timedelta(days=15),
    y_max * 0.95,
    "Test Data Start",
    color="gray",
)

# Format x-axis to show dates nicely
plt.gcf().autofmt_xdate()
ax = plt.gca()
ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y-%m"))

# Use YearLocator for the major ticks
ax.xaxis.set_major_locator(mdates.YearLocator())

# Use MonthLocator for the minor ticks
ax.xaxis.set_minor_locator(mdates.MonthLocator(interval=3))

buffer_time = pd.Timedelta(days=31)
plt.xlim(actual_start_date, end_date + buffer_time)

# Add chart title and labels
plt.title(
    f"Cluster {cluster_id} Crime Rate Prediction", fontweight="bold", fontsize=16
)
plt.xlabel("Date", fontsize=12)
plt.ylabel("Crime Rate", fontsize=12)
plt.legend(loc="best")
plt.grid(True, alpha=0.3)

# Add text annotation about test sequences
plt.text(
    0.02,
    0.02,
    f"Number of test months: {len(test_data)}\nNumber of test predictions: {len(t
transform=plt.gca().transAxes,
    fontsize=9,
    bbox=dict(facecolor="white", alpha=0.8, boxstyle="round,pad=0.5"),
)

# Save figure
plt.tight_layout()
plt.savefig(f"{output_dir}/prediction_plot.png", dpi=300, bbox_inches="tight")

# Display figure
plt.show()

```

```

# Save predictions to CSV for external use
predictions_df = pd.DataFrame(
    {
        "Date": np.concatenate([train_dates, test_dates_array]),
        "Actual": np.concatenate([train_true.flatten(), test_true.flatten()]),
        "Predicted": np.concatenate([train_pred.flatten(), test_pred.flatten()]),
        "DataType": ["Train"] * len(train_true) + ["Test"] * len(test_true),
    }
)
predictions_df.to_csv(f"{output_dir}/all_predictions.csv", index=False)

# Return figure for programmatic use if needed
return plt.gcf()

def predict_future(
    model, X_test, scaler, cluster_id, output_dir, test_data, n_steps=12
):
    """
    Predict future values and create visualization
    """
    # Get the last sequence from test data for forecasting
    last_sequence = X_test[-1:].reshape(1, X_test.shape[1], 1)

    # Predict future steps
    future_predictions = []
    current_sequence = last_sequence.copy()

    # Predict next n_steps
    for _ in range(n_steps):
        next_pred = model.predict(current_sequence, verbose=0)
        future_predictions.append(next_pred[0, 0])

        # Update sequence for next prediction
        current_sequence = np.roll(current_sequence, -1, axis=1)
        current_sequence[0, -1, 0] = next_pred[0, 0]

    # Denormalize predictions
    future_predictions = np.array(future_predictions).reshape(-1, 1)
    future_predictions = scaler.inverse_transform(future_predictions).flatten()

    # Get test predictions for visualization
    test_predictions = model.predict(
        X_test.reshape(X_test.shape[0], X_test.shape[1], 1), verbose=0
    )
    test_predictions_original = scaler.inverse_transform(test_predictions).flatten()

    # Load actual test values
    y_test = np.load(f"{output_dir}/y_test.npy")
    y_test_original = scaler.inverse_transform(y_test.reshape(-1, 1)).flatten()

    # Get last test date and create future dates
    last_test_date = test_data["Date"].max()
    future_dates = pd.date_range(
        start=last_test_date + pd.DateOffset(months=1), periods=n_steps, freq="MS"
    )

    # Print date range information
    print(f"Last test date: {last_test_date}")
    print(f"Future date range: {future_dates[0]} to {future_dates[-1]}")

```



```

# Use only the last 12 months of test data for visualization clarity
recent_test_length = min(12, len(test_predictions_original))
recent_indices = range(
    len(test_predictions_original) - recent_test_length,
    len(test_predictions_original),
)

recent_test_pred = test_predictions_original[recent_indices]
recent_test_actual = y_test_original[recent_indices]
recent_test_dates = test_data["Date"].values[-recent_test_length:]

# Create visualization with dates
plt.figure(figsize=(14, 7))

# Plot recent test data
plt.plot(
    recent_test_dates,
    recent_test_actual,
    "b-",
    label="Recent Actual",
    linewidth=1.5,
)

plt.plot(
    recent_test_dates,
    recent_test_pred,
    "g--",
    label="Recent Predicted",
    linewidth=2,
)

# Get the last predicted value to connect with future predictions
last_pred_value = test_predictions_original[-1]
last_pred_date = test_data["Date"].max()

# Create a connection point between test predictions and future predictions
connection_dates = [last_pred_date, future_dates[0]]
connection_values = [last_pred_value, future_predictions[0]]

# Plot the connection line
plt.plot(connection_dates, connection_values, "r-", alpha=0.5, linewidth=1)

# Plot future predictions
plt.plot(
    future_dates,
    future_predictions,
    "r-o",
    label="Future Predictions",
    linewidth=2,
    markersize=5,
)

# Add boundary line
plt.axvline(x=last_test_date, color="gray", linestyle="--")
y_min, y_max = plt.ylim()
plt.text(
    last_test_date + pd.Timedelta(days=5),
    y_max * 0.95,
    "Forecast Start",
    color="gray",
)

```

```

# Format x-axis
plt.gcf().autofmt_xdate()
ax = plt.gca()
ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y-%m"))
ax.xaxis.set_major_locator(mdates.MonthLocator(interval=2))

# Ensure we see the full date range with some padding
# Important: Start exactly at the first recent test date with no left-side empty
buffer_time = pd.Timedelta(days=31) # About a month of buffer
plt.xlim(recent_test_dates[0], future_dates[-1] + buffer_time)

# Calculate suitable y-axis range
all_values = np.concatenate(
    [recent_test_actual, recent_test_pred, future_predictions]
)
y_min = max(0, np.min(all_values) * 0.95) # Prevent negative values
y_max = np.max(all_values) * 1.05
plt.ylim(y_min, y_max)

# Add chart styling
plt.title(
    f"Cluster {cluster_id} - Future Crime Rate Forecast",
    fontweight="bold",
    fontsize=16,
)
plt.xlabel("Date", fontsize=12)
plt.ylabel("Crime Rate", fontsize=12)
plt.legend(loc="best")
plt.grid(True, alpha=0.3)

# Save and display
plt.tight_layout()
plt.savefig(f"{output_dir}/future_forecast.png", dpi=300, bbox_inches="tight")

# Display the figure
plt.show()

# Save future predictions to CSV
future_df = pd.DataFrame(
    {"Date": future_dates, "PredictedCrimeRate": future_predictions}
)
future_df.to_csv(f"{output_dir}/future_predictions.csv", index=False)

# Return the figure for programmatic use if needed
return future_predictions, plt.gcf()

def get_model_type_name(cluster_id):
    """Helper function to get model type name for reporting"""
    if cluster_id == 3:
        return "Complex (Bidirectional LSTM with Attention)"
    elif cluster_id == 1:
        return "Simple (Single LSTM)"
    elif cluster_id == 2:
        return "Medium (Stacked LSTM)"
    else:
        return "Medium (Default)"

def create_performance_comparison(results_df, output_dir):

```

```
"""
```

Create a visualization comparing model performance across clusters

Parameters:

results_df : pandas DataFrame

DataFrame containing model evaluation results

output_dir : str

Directory to save the output visualization

```
"""
```

Ensure proper styling

```
sns.set_style("whitegrid")
```

```
plt.rcParams.update({
```

```
{
```

```
    "font.family": "sans-serif",
```

```
    "font.sans-serif": ["Arial", "DejaVu Sans"],
```

```
    "figure.figsize": (14, 8),
```

```
    "axes.labelsize": 12,
```

```
    "axes.titlesize": 14,
```

```
    "xtick.labelsize": 10,
```

```
    "ytick.labelsize": 10,
```

```
    "legend.fontsize": 10,
```

```
    }
```

```
)
```

Create figure with two subplots

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 7))
```

Get cluster IDs and model types

```
cluster_ids = results_df["ClusterID"].tolist()
```

```
model_types = [get_model_type_name(c_id) for c_id in cluster_ids]
```

Add model type to results_df

```
results_df["ModelType"] = model_types
```

Sort by cluster ID

```
results_df = results_df.sort_values("ClusterID")
```

Define color map

```
colors = ["#3498db", "#2ecc71", "#e74c3c"]
```

1. RMSE Comparison (Left subplot)

```
bar_width = 0.35
```

```
x = np.arange(len(cluster_ids))
```

Plot Training RMSE

```
trainBars = ax1.bar(
```

```
    x - bar_width / 2,
```

```
    results_df["TrainRMSE"],
```

```
    bar_width,
```

```
    label="Training RMSE",
```

```
    color=[colors[i - 1] for i in results_df["ClusterID"]],
```

```
    alpha=0.7,
```

```
    edgecolor="black",
```

```
    linewidth=1,
```

```
)
```

Plot Test RMSE

```
testBars = ax1.bar(
```

```
    x + bar_width / 2,
```

```
    results_df["TestRMSE"],
```

```

        bar_width,
        label="Test RMSE",
        color=[colors[i - 1] for i in results_df["ClusterID"]],
        alpha=1.0,
        edgecolor="black",
        linewidth=1,
    )

# Customize RMSE plot
ax1.set_xlabel("Cluster", fontweight="bold")
ax1.set_ylabel("RMSE", fontweight="bold")
ax1.set_title("RMSE Comparison by Cluster", fontsize=14, fontweight="bold", pad=2)
ax1.set_xticks(x)
ax1.set_xticklabels([f"Cluster {cid}" for cid in results_df["ClusterID"]])
ax1.legend()

# Add value labels on bars
def add_labels(bars):
    for bar in bars:
        height = bar.get_height()
        ax1.annotate(
            f"{height:.2f}",
            xy=(bar.get_x() + bar.get_width() / 2, height),
            xytext=(0, 3), # 3 points vertical offset
            textcoords="offset points",
            ha="center",
            va="bottom",
            fontsize=9,
        )

add_labels(train_bars)
add_labels(test_bars)

# 2. Metrics Table (Right subplot)
ax2.axis("tight")
ax2.axis("off")

# Format the table data
table_data = []
header = ["Cluster", "Model Type", "Test RMSE", "Test MAE", "Train RMSE"]

for _, row in results_df.iterrows():
    table_data.append(
        [
            f"Cluster {row['ClusterID']}",
            row["ModelType"],
            f"{row['TestRMSE']:.4f}",
            f"{row['TestMAE']:.4f}",
            f"{row['TrainRMSE']:.4f}",
        ]
    )

# Create table
table = ax2.table(
    cellText=table_data,
    colLabels=header,
    loc="center",
    cellLoc="center",
    colColours=["#f0f0f0"] * 5,
)

```

```

# Style the table
table.auto_set_font_size(False)
table.set_fontsize(11)
table.scale(1.2, 1.5)

# Adjust table column widths
table.auto_set_column_width([0, 1, 2, 3, 4])

# Add title for the table
ax2.set_title("Model Performance Metrics", fontsize=14, fontweight="bold", pad=20)

# Overall title for the figure
plt.suptitle(
    "LSTM Model Performance Comparison Across Clusters",
    fontsize=16,
    fontweight="bold",
    y=0.98,
)

# Adjust layout
plt.tight_layout(rect=[0, 0, 1, 0.95])

# Save figure
plt.savefig(
    f"{output_dir}/model_performance_comparison.png", dpi=300, bbox_inches="tight"
)
plt.show()

# Create a detailed performance table as CSV
detailed_table = results_df.copy()
detailed_table["ModelArchitecture"] = detailed_table["ModelType"]
detailed_table = detailed_table.sort_values("TestRMSE")

# Add ranking
detailed_table["PerformanceRank"] = range(1, len(detailed_table) + 1)

# Select and reorganize columns
detailed_table = detailed_table[
    [
        "PerformanceRank",
        "ClusterID",
        "ModelArchitecture",
        "TestRMSE",
        "TestMAE",
        "TrainRMSE",
    ]
]

# Save detailed table
detailed_table.to_csv(
    f"{output_dir}/detailed_performance_comparison.csv", index=False
)

return fig

# Main function
def main():
    # Set results directory
    results_dir = "data/lstm_results"
    os.makedirs(results_dir, exist_ok=True)

```

```

# Get cluster directories
model_dir = "data/lstm_by_cluster"
cluster_dirs = [d for d in os.listdir(model_dir) if d.startswith("cluster_")]

# Create results summary DataFrame
results_summary = []

# Process each cluster
for cluster_dir in cluster_dirs:
    # Extract cluster ID
    cluster_id = int(cluster_dir.split("_")[1])
    print(f"\n{'='*50}")
    print(f"Processing Cluster {cluster_id}")
    print(f"{'='*50}")

    try:
        # Train and evaluate model
        model, history, evaluation = train_and_evaluate(cluster_id)

        # Add to results summary
        results_summary.append(
            {
                "ClusterID": cluster_id,
                "TestRMSE": evaluation["test_rmse"],
                "TestMAE": evaluation["test_mae"],
                "TestLoss": evaluation["test_loss"],
                "TrainRMSE": evaluation["train_rmse"],
            }
        )

    except Exception as e:
        print(f"Error processing cluster {cluster_id}: {str(e)}")
        import traceback

        traceback.print_exc()

# Create results summary file
if results_summary:
    results_df = pd.DataFrame(results_summary)
    results_df.to_csv(f"{results_dir}/model_evaluation_summary.csv", index=False)

    create_performance_comparison(results_df, results_dir)

    print("\nModel Evaluation Summary:")
    print(results_df)

    print("\nLSTM model training and evaluation complete!")

if __name__ == "__main__":
    main()

```

=====
Processing Cluster 1
=====

X_train shape: (144, 12), y_train shape: (144,)

X_test shape: (24, 12), y_test shape: (24,)

Using enhanced multi-region specialized model for Cluster 1

Using specialized training for multi-region cluster

Advanced augmented training data: (144, 12, 1) → (864, 12, 1)

Augmented target data: (144, 1) → (864, 1)

Epoch 1/120

Epoch 1: val_loss improved from inf to 0.31071, saving model to data/lstm_by_cluster/c
luster_1/best_model_1745208288.keras

195/195 - 5s - 26ms/step - loss: 0.3930 - mae: 0.1871 - val_loss: 0.3107 - val_mae: 0.
1510 - learning_rate: 1.0000e-03

Epoch 2/120

Epoch 2: val_loss improved from 0.31071 to 0.23867, saving model to data/lstm_by_clust
er/cluster_1/best_model_1745208288.keras

195/195 - 2s - 8ms/step - loss: 0.2659 - mae: 0.1269 - val_loss: 0.2387 - val_mae: 0.1
421 - learning_rate: 1.0000e-03

Epoch 3/120

Epoch 3: val_loss improved from 0.23867 to 0.17633, saving model to data/lstm_by_clust
er/cluster_1/best_model_1745208288.keras

195/195 - 2s - 9ms/step - loss: 0.1973 - mae: 0.1166 - val_loss: 0.1763 - val_mae: 0.1
312 - learning_rate: 1.0000e-03

Epoch 4/120

Epoch 4: val_loss improved from 0.17633 to 0.12918, saving model to data/lstm_by_clust
er/cluster_1/best_model_1745208288.keras

195/195 - 2s - 8ms/step - loss: 0.1423 - mae: 0.1137 - val_loss: 0.1292 - val_mae: 0.1
233 - learning_rate: 1.0000e-03

Epoch 5/120

Epoch 5: val_loss improved from 0.12918 to 0.09635, saving model to data/lstm_by_clust
er/cluster_1/best_model_1745208288.keras

195/195 - 2s - 8ms/step - loss: 0.1034 - mae: 0.1131 - val_loss: 0.0963 - val_mae: 0.1
253 - learning_rate: 1.0000e-03

Epoch 6/120

Epoch 6: val_loss improved from 0.09635 to 0.08092, saving model to data/lstm_by_clust
er/cluster_1/best_model_1745208288.keras

195/195 - 2s - 8ms/step - loss: 0.0751 - mae: 0.1119 - val_loss: 0.0809 - val_mae: 0.1
346 - learning_rate: 1.0000e-03

Epoch 7/120

Epoch 7: val_loss improved from 0.08092 to 0.06542, saving model to data/lstm_by_clust
er/cluster_1/best_model_1745208288.keras

195/195 - 2s - 8ms/step - loss: 0.0544 - mae: 0.1079 - val_loss: 0.0654 - val_mae: 0.1
335 - learning_rate: 1.0000e-03

Epoch 8/120

Epoch 8: val_loss improved from 0.06542 to 0.06286, saving model to data/lstm_by_clust
er/cluster_1/best_model_1745208288.keras

195/195 - 2s - 8ms/step - loss: 0.0416 - mae: 0.1085 - val_loss: 0.0629 - val_mae: 0.1
480 - learning_rate: 1.0000e-03

Epoch 9/120

Epoch 9: val_loss improved from 0.06286 to 0.04476, saving model to data/lstm_by_clust
er/cluster_1/best_model_1745208288.keras

195/195 - 2s - 8ms/step - loss: 0.0327 - mae: 0.1073 - val_loss: 0.0448 - val_mae: 0.1237 - learning_rate: 1.0000e-03
Epoch 10/120

Epoch 10: val_loss improved from 0.04476 to 0.04205, saving model to data/lstm_by_cluster/cluster_1/best_model_1745208288.keras

195/195 - 2s - 8ms/step - loss: 0.0273 - mae: 0.1045 - val_loss: 0.0421 - val_mae: 0.1281 - learning_rate: 1.0000e-03
Epoch 11/120

Epoch 11: val_loss did not improve from 0.04205

195/195 - 2s - 8ms/step - loss: 0.0243 - mae: 0.1041 - val_loss: 0.0421 - val_mae: 0.1281 - learning_rate: 1.0000e-03
Epoch 12/120

Epoch 12: val_loss improved from 0.04205 to 0.04047, saving model to data/lstm_by_cluster/cluster_1/best_model_1745208288.keras

195/195 - 2s - 8ms/step - loss: 0.0225 - mae: 0.1008 - val_loss: 0.0405 - val_mae: 0.1302 - learning_rate: 1.0000e-03
Epoch 13/120

Epoch 13: val_loss did not improve from 0.04047

195/195 - 1s - 8ms/step - loss: 0.0214 - mae: 0.1000 - val_loss: 0.0435 - val_mae: 0.1340 - learning_rate: 1.0000e-03
Epoch 14/120

Epoch 14: val_loss improved from 0.04047 to 0.03707, saving model to data/lstm_by_cluster/cluster_1/best_model_1745208288.keras

195/195 - 2s - 8ms/step - loss: 0.0197 - mae: 0.0948 - val_loss: 0.0371 - val_mae: 0.1298 - learning_rate: 1.0000e-03
Epoch 15/120

Epoch 15: val_loss did not improve from 0.03707

195/195 - 2s - 8ms/step - loss: 0.0189 - mae: 0.0945 - val_loss: 0.0382 - val_mae: 0.1268 - learning_rate: 1.0000e-03
Epoch 16/120

Epoch 16: val_loss improved from 0.03707 to 0.03700, saving model to data/lstm_by_cluster/cluster_1/best_model_1745208288.keras

195/195 - 2s - 8ms/step - loss: 0.0175 - mae: 0.0915 - val_loss: 0.0370 - val_mae: 0.1297 - learning_rate: 1.0000e-03
Epoch 17/120

Epoch 17: val_loss did not improve from 0.03700

195/195 - 2s - 8ms/step - loss: 0.0171 - mae: 0.0920 - val_loss: 0.0394 - val_mae: 0.1356 - learning_rate: 1.0000e-03
Epoch 18/120

Epoch 18: val_loss improved from 0.03700 to 0.03453, saving model to data/lstm_by_cluster/cluster_1/best_model_1745208288.keras

195/195 - 2s - 8ms/step - loss: 0.0164 - mae: 0.0902 - val_loss: 0.0345 - val_mae: 0.1252 - learning_rate: 1.0000e-03
Epoch 19/120

Epoch 19: val_loss did not improve from 0.03453

195/195 - 2s - 8ms/step - loss: 0.0160 - mae: 0.0895 - val_loss: 0.0363 - val_mae: 0.1258 - learning_rate: 1.0000e-03
Epoch 20/120

Epoch 20: val_loss did not improve from 0.03453

195/195 - 2s - 8ms/step - loss: 0.0160 - mae: 0.0907 - val_loss: 0.0380 - val_mae: 0.1

308 - learning_rate: 1.0000e-03
Epoch 21/120

Epoch 21: val_loss did not improve from 0.03453
195/195 - 1s - 8ms/step - loss: 0.0154 - mae: 0.0889 - val_loss: 0.0357 - val_mae: 0.1
279 - learning_rate: 1.0000e-03
Epoch 22/120

Epoch 22: val_loss did not improve from 0.03453
195/195 - 2s - 8ms/step - loss: 0.0149 - mae: 0.0882 - val_loss: 0.0351 - val_mae: 0.1
283 - learning_rate: 1.0000e-03
Epoch 23/120

Epoch 23: val_loss improved from 0.03453 to 0.03328, saving model to data/lstm_by_cluster/cluster_1/best_model_1745208288.keras
195/195 - 2s - 8ms/step - loss: 0.0147 - mae: 0.0885 - val_loss: 0.0333 - val_mae: 0.1
232 - learning_rate: 1.0000e-03
Epoch 24/120

Epoch 24: val_loss did not improve from 0.03328
195/195 - 2s - 8ms/step - loss: 0.0150 - mae: 0.0892 - val_loss: 0.0337 - val_mae: 0.1
250 - learning_rate: 1.0000e-03
Epoch 25/120

Epoch 25: val_loss did not improve from 0.03328
195/195 - 1s - 8ms/step - loss: 0.0141 - mae: 0.0878 - val_loss: 0.0363 - val_mae: 0.1
324 - learning_rate: 1.0000e-03
Epoch 26/120

Epoch 26: val_loss did not improve from 0.03328
195/195 - 1s - 8ms/step - loss: 0.0140 - mae: 0.0876 - val_loss: 0.0369 - val_mae: 0.1
334 - learning_rate: 1.0000e-03
Epoch 27/120

Epoch 27: val_loss did not improve from 0.03328
195/195 - 2s - 8ms/step - loss: 0.0143 - mae: 0.0880 - val_loss: 0.0388 - val_mae: 0.1
388 - learning_rate: 1.0000e-03
Epoch 28/120

Epoch 28: val_loss did not improve from 0.03328
195/195 - 2s - 8ms/step - loss: 0.0140 - mae: 0.0886 - val_loss: 0.0353 - val_mae: 0.1
309 - learning_rate: 1.0000e-03
Epoch 29/120

Epoch 29: val_loss did not improve from 0.03328
195/195 - 2s - 8ms/step - loss: 0.0136 - mae: 0.0865 - val_loss: 0.0334 - val_mae: 0.1
260 - learning_rate: 1.0000e-03
Epoch 30/120

Epoch 30: val_loss did not improve from 0.03328
195/195 - 1s - 8ms/step - loss: 0.0133 - mae: 0.0850 - val_loss: 0.0363 - val_mae: 0.1
335 - learning_rate: 1.0000e-03
Epoch 31/120

Epoch 31: val_loss improved from 0.03328 to 0.03319, saving model to data/lstm_by_cluster/cluster_1/best_model_1745208288.keras

Epoch 31: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
195/195 - 2s - 8ms/step - loss: 0.0136 - mae: 0.0866 - val_loss: 0.0332 - val_mae: 0.1
262 - learning_rate: 1.0000e-03
Epoch 32/120

Epoch 32: val_loss improved from 0.03319 to 0.03160, saving model to data/lstm_by_cluster/cluster_1/best_model_1745208288.keras
195/195 - 2s - 8ms/step - loss: 0.0129 - mae: 0.0834 - val_loss: 0.0316 - val_mae: 0.1248 - learning_rate: 5.0000e-04
Epoch 33/120

Epoch 33: val_loss improved from 0.03160 to 0.03072, saving model to data/lstm_by_cluster/cluster_1/best_model_1745208288.keras
195/195 - 2s - 9ms/step - loss: 0.0126 - mae: 0.0824 - val_loss: 0.0307 - val_mae: 0.1227 - learning_rate: 5.0000e-04
Epoch 34/120

Epoch 34: val_loss did not improve from 0.03072
195/195 - 2s - 8ms/step - loss: 0.0126 - mae: 0.0828 - val_loss: 0.0319 - val_mae: 0.1270 - learning_rate: 5.0000e-04
Epoch 35/120

Epoch 35: val_loss did not improve from 0.03072
195/195 - 2s - 8ms/step - loss: 0.0127 - mae: 0.0831 - val_loss: 0.0316 - val_mae: 0.1264 - learning_rate: 5.0000e-04
Epoch 36/120

Epoch 36: val_loss did not improve from 0.03072
195/195 - 2s - 8ms/step - loss: 0.0121 - mae: 0.0806 - val_loss: 0.0320 - val_mae: 0.1272 - learning_rate: 5.0000e-04
Epoch 37/120

Epoch 37: val_loss improved from 0.03072 to 0.02970, saving model to data/lstm_by_cluster/cluster_1/best_model_1745208288.keras
195/195 - 2s - 8ms/step - loss: 0.0121 - mae: 0.0811 - val_loss: 0.0297 - val_mae: 0.1196 - learning_rate: 5.0000e-04
Epoch 38/120

Epoch 38: val_loss did not improve from 0.02970
195/195 - 2s - 9ms/step - loss: 0.0121 - mae: 0.0815 - val_loss: 0.0298 - val_mae: 0.1211 - learning_rate: 5.0000e-04
Epoch 39/120

Epoch 39: val_loss did not improve from 0.02970
195/195 - 2s - 8ms/step - loss: 0.0121 - mae: 0.0806 - val_loss: 0.0309 - val_mae: 0.1248 - learning_rate: 5.0000e-04
Epoch 40/120

Epoch 40: val_loss improved from 0.02970 to 0.02859, saving model to data/lstm_by_cluster/cluster_1/best_model_1745208288.keras
195/195 - 2s - 8ms/step - loss: 0.0116 - mae: 0.0798 - val_loss: 0.0286 - val_mae: 0.1173 - learning_rate: 5.0000e-04
Epoch 41/120

Epoch 41: val_loss did not improve from 0.02859
195/195 - 2s - 8ms/step - loss: 0.0123 - mae: 0.0819 - val_loss: 0.0310 - val_mae: 0.1253 - learning_rate: 5.0000e-04
Epoch 42/120

Epoch 42: val_loss did not improve from 0.02859
195/195 - 2s - 8ms/step - loss: 0.0119 - mae: 0.0802 - val_loss: 0.0320 - val_mae: 0.1265 - learning_rate: 5.0000e-04
Epoch 43/120

Epoch 43: val_loss did not improve from 0.02859

195/195 - 2s - 8ms/step - loss: 0.0121 - mae: 0.0804 - val_loss: 0.0327 - val_mae: 0.1299 - learning_rate: 5.0000e-04
Epoch 44/120

Epoch 44: val_loss did not improve from 0.02859
195/195 - 2s - 8ms/step - loss: 0.0117 - mae: 0.0794 - val_loss: 0.0297 - val_mae: 0.1229 - learning_rate: 5.0000e-04
Epoch 45/120

Epoch 45: val_loss did not improve from 0.02859
195/195 - 2s - 8ms/step - loss: 0.0119 - mae: 0.0803 - val_loss: 0.0308 - val_mae: 0.1251 - learning_rate: 5.0000e-04
Epoch 46/120

Epoch 46: val_loss did not improve from 0.02859
195/195 - 2s - 8ms/step - loss: 0.0119 - mae: 0.0805 - val_loss: 0.0298 - val_mae: 0.1245 - learning_rate: 5.0000e-04
Epoch 47/120

Epoch 47: val_loss did not improve from 0.02859
195/195 - 2s - 8ms/step - loss: 0.0120 - mae: 0.0805 - val_loss: 0.0310 - val_mae: 0.1256 - learning_rate: 5.0000e-04
Epoch 48/120

Epoch 48: val_loss did not improve from 0.02859

Epoch 48: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.
195/195 - 2s - 8ms/step - loss: 0.0117 - mae: 0.0804 - val_loss: 0.0311 - val_mae: 0.1270 - learning_rate: 5.0000e-04
Epoch 49/120

Epoch 49: val_loss improved from 0.02859 to 0.02852, saving model to data/lstm_by_cluster/cluster_1/best_model_1745208288.keras
195/195 - 2s - 8ms/step - loss: 0.0115 - mae: 0.0789 - val_loss: 0.0285 - val_mae: 0.1219 - learning_rate: 2.5000e-04
Epoch 50/120

Epoch 50: val_loss did not improve from 0.02852
195/195 - 2s - 8ms/step - loss: 0.0111 - mae: 0.0782 - val_loss: 0.0288 - val_mae: 0.1232 - learning_rate: 2.5000e-04
Epoch 51/120

Epoch 51: val_loss improved from 0.02852 to 0.02792, saving model to data/lstm_by_cluster/cluster_1/best_model_1745208288.keras
195/195 - 2s - 8ms/step - loss: 0.0112 - mae: 0.0788 - val_loss: 0.0279 - val_mae: 0.1206 - learning_rate: 2.5000e-04
Epoch 52/120

Epoch 52: val_loss did not improve from 0.02792
195/195 - 2s - 8ms/step - loss: 0.0112 - mae: 0.0777 - val_loss: 0.0292 - val_mae: 0.1232 - learning_rate: 2.5000e-04
Epoch 53/120

Epoch 53: val_loss did not improve from 0.02792
195/195 - 2s - 8ms/step - loss: 0.0108 - mae: 0.0771 - val_loss: 0.0298 - val_mae: 0.1248 - learning_rate: 2.5000e-04
Epoch 54/120

Epoch 54: val_loss improved from 0.02792 to 0.02769, saving model to data/lstm_by_cluster/cluster_1/best_model_1745208288.keras
195/195 - 2s - 8ms/step - loss: 0.0110 - mae: 0.0770 - val_loss: 0.0277 - val_mae: 0.1

189 - learning_rate: 2.5000e-04
Epoch 55/120

Epoch 55: val_loss improved from 0.02769 to 0.02748, saving model to data/lstm_by_cluster/cluster_1/best_model_1745208288.keras
195/195 - 2s - 9ms/step - loss: 0.0113 - mae: 0.0781 - val_loss: 0.0275 - val_mae: 0.1
194 - learning_rate: 2.5000e-04
Epoch 56/120

Epoch 56: val_loss improved from 0.02748 to 0.02691, saving model to data/lstm_by_cluster/cluster_1/best_model_1745208288.keras
195/195 - 2s - 8ms/step - loss: 0.0107 - mae: 0.0768 - val_loss: 0.0269 - val_mae: 0.1
180 - learning_rate: 2.5000e-04
Epoch 57/120

Epoch 57: val_loss did not improve from 0.02691
195/195 - 2s - 9ms/step - loss: 0.0111 - mae: 0.0781 - val_loss: 0.0286 - val_mae: 0.1
238 - learning_rate: 2.5000e-04
Epoch 58/120

Epoch 58: val_loss did not improve from 0.02691
195/195 - 2s - 8ms/step - loss: 0.0109 - mae: 0.0778 - val_loss: 0.0270 - val_mae: 0.1
191 - learning_rate: 2.5000e-04
Epoch 59/120

Epoch 59: val_loss did not improve from 0.02691
195/195 - 2s - 8ms/step - loss: 0.0105 - mae: 0.0760 - val_loss: 0.0283 - val_mae: 0.1
209 - learning_rate: 2.5000e-04
Epoch 60/120

Epoch 60: val_loss did not improve from 0.02691
195/195 - 2s - 8ms/step - loss: 0.0107 - mae: 0.0767 - val_loss: 0.0270 - val_mae: 0.1
199 - learning_rate: 2.5000e-04
Epoch 61/120

Epoch 61: val_loss did not improve from 0.02691
195/195 - 2s - 8ms/step - loss: 0.0108 - mae: 0.0765 - val_loss: 0.0282 - val_mae: 0.1
226 - learning_rate: 2.5000e-04
Epoch 62/120

Epoch 62: val_loss did not improve from 0.02691
195/195 - 2s - 8ms/step - loss: 0.0106 - mae: 0.0757 - val_loss: 0.0275 - val_mae: 0.1
203 - learning_rate: 2.5000e-04
Epoch 63/120

Epoch 63: val_loss did not improve from 0.02691
195/195 - 2s - 8ms/step - loss: 0.0107 - mae: 0.0765 - val_loss: 0.0279 - val_mae: 0.1
212 - learning_rate: 2.5000e-04
Epoch 64/120

Epoch 64: val_loss did not improve from 0.02691

Epoch 64: ReduceLROnPlateau reducing learning rate to 0.0001250000059371814.
195/195 - 2s - 8ms/step - loss: 0.0110 - mae: 0.0773 - val_loss: 0.0291 - val_mae: 0.1
249 - learning_rate: 2.5000e-04
Epoch 65/120

Epoch 65: val_loss improved from 0.02691 to 0.02691, saving model to data/lstm_by_cluster/cluster_1/best_model_1745208288.keras
195/195 - 2s - 8ms/step - loss: 0.0109 - mae: 0.0777 - val_loss: 0.0269 - val_mae: 0.1
193 - learning_rate: 1.2500e-04

Epoch 66/120

Epoch 66: val_loss did not improve from 0.02691

195/195 - 2s - 8ms/step - loss: 0.0105 - mae: 0.0755 - val_loss: 0.0272 - val_mae: 0.1179 - learning_rate: 1.2500e-04

Epoch 67/120

Epoch 67: val_loss improved from 0.02691 to 0.02654, saving model to data/lstm_by_cluster/cluster_1/best_model_1745208288.keras

195/195 - 2s - 8ms/step - loss: 0.0103 - mae: 0.0751 - val_loss: 0.0265 - val_mae: 0.1184 - learning_rate: 1.2500e-04

Epoch 68/120

Epoch 68: val_loss did not improve from 0.02654

195/195 - 2s - 8ms/step - loss: 0.0107 - mae: 0.0763 - val_loss: 0.0267 - val_mae: 0.1169 - learning_rate: 1.2500e-04

Epoch 69/120

Epoch 69: val_loss improved from 0.02654 to 0.02597, saving model to data/lstm_by_cluster/cluster_1/best_model_1745208288.keras

195/195 - 2s - 8ms/step - loss: 0.0106 - mae: 0.0758 - val_loss: 0.0260 - val_mae: 0.1156 - learning_rate: 1.2500e-04

Epoch 70/120

Epoch 70: val_loss did not improve from 0.02597

195/195 - 2s - 8ms/step - loss: 0.0108 - mae: 0.0774 - val_loss: 0.0264 - val_mae: 0.1166 - learning_rate: 1.2500e-04

Epoch 71/120

Epoch 71: val_loss did not improve from 0.02597

195/195 - 2s - 8ms/step - loss: 0.0103 - mae: 0.0753 - val_loss: 0.0264 - val_mae: 0.1172 - learning_rate: 1.2500e-04

Epoch 72/120

Epoch 72: val_loss improved from 0.02597 to 0.02572, saving model to data/lstm_by_cluster/cluster_1/best_model_1745208288.keras

195/195 - 2s - 9ms/step - loss: 0.0107 - mae: 0.0766 - val_loss: 0.0257 - val_mae: 0.1143 - learning_rate: 1.2500e-04

Epoch 73/120

Epoch 73: val_loss did not improve from 0.02572

195/195 - 2s - 8ms/step - loss: 0.0107 - mae: 0.0765 - val_loss: 0.0274 - val_mae: 0.1203 - learning_rate: 1.2500e-04

Epoch 74/120

Epoch 74: val_loss did not improve from 0.02572

195/195 - 2s - 8ms/step - loss: 0.0105 - mae: 0.0758 - val_loss: 0.0270 - val_mae: 0.1187 - learning_rate: 1.2500e-04

Epoch 75/120

Epoch 75: val_loss did not improve from 0.02572

195/195 - 2s - 8ms/step - loss: 0.0107 - mae: 0.0772 - val_loss: 0.0260 - val_mae: 0.1156 - learning_rate: 1.2500e-04

Epoch 76/120

Epoch 76: val_loss did not improve from 0.02572

195/195 - 2s - 8ms/step - loss: 0.0107 - mae: 0.0766 - val_loss: 0.0263 - val_mae: 0.1175 - learning_rate: 1.2500e-04

Epoch 77/120

Epoch 77: val_loss did not improve from 0.02572

195/195 - 1s - 8ms/step - loss: 0.0102 - mae: 0.0750 - val_loss: 0.0261 - val_mae: 0.161 - learning_rate: 1.2500e-04
Epoch 78/120

Epoch 78: val_loss did not improve from 0.02572
195/195 - 2s - 8ms/step - loss: 0.0105 - mae: 0.0765 - val_loss: 0.0267 - val_mae: 0.191 - learning_rate: 1.2500e-04
Epoch 79/120

Epoch 79: val_loss did not improve from 0.02572
195/195 - 2s - 8ms/step - loss: 0.0105 - mae: 0.0761 - val_loss: 0.0271 - val_mae: 0.193 - learning_rate: 1.2500e-04
Epoch 80/120

Epoch 80: val_loss improved from 0.02572 to 0.02546, saving model to data/lstm_by_cluster/cluster_1/best_model_1745208288.keras
195/195 - 2s - 8ms/step - loss: 0.0101 - mae: 0.0748 - val_loss: 0.0255 - val_mae: 0.170 - learning_rate: 1.2500e-04
Epoch 81/120

Epoch 81: val_loss did not improve from 0.02546
195/195 - 2s - 8ms/step - loss: 0.0103 - mae: 0.0752 - val_loss: 0.0259 - val_mae: 0.163 - learning_rate: 1.2500e-04
Epoch 82/120

Epoch 82: val_loss did not improve from 0.02546
195/195 - 2s - 8ms/step - loss: 0.0104 - mae: 0.0750 - val_loss: 0.0264 - val_mae: 0.184 - learning_rate: 1.2500e-04
Epoch 83/120

Epoch 83: val_loss did not improve from 0.02546
195/195 - 2s - 8ms/step - loss: 0.0103 - mae: 0.0757 - val_loss: 0.0266 - val_mae: 0.192 - learning_rate: 1.2500e-04
Epoch 84/120

Epoch 84: val_loss improved from 0.02546 to 0.02539, saving model to data/lstm_by_cluster/cluster_1/best_model_1745208288.keras
195/195 - 2s - 8ms/step - loss: 0.0104 - mae: 0.0753 - val_loss: 0.0254 - val_mae: 0.142 - learning_rate: 1.2500e-04
Epoch 85/120

Epoch 85: val_loss did not improve from 0.02539
195/195 - 2s - 8ms/step - loss: 0.0104 - mae: 0.0761 - val_loss: 0.0259 - val_mae: 0.168 - learning_rate: 1.2500e-04
Epoch 86/120

Epoch 86: val_loss did not improve from 0.02539
195/195 - 2s - 8ms/step - loss: 0.0105 - mae: 0.0755 - val_loss: 0.0257 - val_mae: 0.165 - learning_rate: 1.2500e-04
Epoch 87/120

Epoch 87: val_loss did not improve from 0.02539
195/195 - 2s - 8ms/step - loss: 0.0105 - mae: 0.0761 - val_loss: 0.0260 - val_mae: 0.163 - learning_rate: 1.2500e-04
Epoch 88/120

Epoch 88: val_loss did not improve from 0.02539

Epoch 88: ReduceLROnPlateau reducing learning rate to 6.25000029685907e-05.
195/195 - 2s - 8ms/step - loss: 0.0101 - mae: 0.0748 - val_loss: 0.0258 - val_mae: 0.164 - learning_rate: 1.2500e-04

Epoch 89/120

Epoch 89: val_loss did not improve from 0.02539

195/195 - 1s - 8ms/step - loss: 0.0101 - mae: 0.0751 - val_loss: 0.0261 - val_mae: 0.163 - learning_rate: 6.2500e-05

Epoch 90/120

Epoch 90: val_loss did not improve from 0.02539

195/195 - 2s - 8ms/step - loss: 0.0102 - mae: 0.0749 - val_loss: 0.0264 - val_mae: 0.168 - learning_rate: 6.2500e-05

Epoch 91/120

Epoch 91: val_loss did not improve from 0.02539

195/195 - 2s - 8ms/step - loss: 0.0099 - mae: 0.0742 - val_loss: 0.0256 - val_mae: 0.147 - learning_rate: 6.2500e-05

Epoch 92/120

Epoch 92: val_loss did not improve from 0.02539

195/195 - 2s - 8ms/step - loss: 0.0101 - mae: 0.0750 - val_loss: 0.0255 - val_mae: 0.154 - learning_rate: 6.2500e-05

Epoch 93/120

Epoch 93: val_loss did not improve from 0.02539

195/195 - 2s - 8ms/step - loss: 0.0103 - mae: 0.0754 - val_loss: 0.0257 - val_mae: 0.144 - learning_rate: 6.2500e-05

Epoch 94/120

Epoch 94: val_loss improved from 0.02539 to 0.02536, saving model to data/lstm_by_cluster/cluster_1/best_model_1745208288.keras

195/195 - 2s - 8ms/step - loss: 0.0102 - mae: 0.0750 - val_loss: 0.0254 - val_mae: 0.144 - learning_rate: 6.2500e-05

Epoch 95/120

Epoch 95: val_loss did not improve from 0.02536

195/195 - 2s - 8ms/step - loss: 0.0098 - mae: 0.0738 - val_loss: 0.0256 - val_mae: 0.152 - learning_rate: 6.2500e-05

Epoch 96/120

Epoch 96: val_loss did not improve from 0.02536

Epoch 96: ReduceLRonPlateau reducing learning rate to 3.125000148429535e-05.

195/195 - 2s - 8ms/step - loss: 0.0100 - mae: 0.0745 - val_loss: 0.0257 - val_mae: 0.151 - learning_rate: 6.2500e-05

Epoch 97/120

Epoch 97: val_loss did not improve from 0.02536

195/195 - 1s - 8ms/step - loss: 0.0096 - mae: 0.0736 - val_loss: 0.0255 - val_mae: 0.145 - learning_rate: 3.1250e-05

Epoch 98/120

Epoch 98: val_loss improved from 0.02536 to 0.02534, saving model to data/lstm_by_cluster/cluster_1/best_model_1745208288.keras

195/195 - 2s - 8ms/step - loss: 0.0098 - mae: 0.0745 - val_loss: 0.0253 - val_mae: 0.142 - learning_rate: 3.1250e-05

Epoch 99/120

Epoch 99: val_loss did not improve from 0.02534

195/195 - 2s - 8ms/step - loss: 0.0101 - mae: 0.0751 - val_loss: 0.0254 - val_mae: 0.142 - learning_rate: 3.1250e-05

Epoch 100/120

Epoch 100: val_loss did not improve from 0.02534
195/195 - 2s - 8ms/step - loss: 0.0100 - mae: 0.0744 - val_loss: 0.0255 - val_mae: 0.147 - learning_rate: 3.1250e-05
Epoch 101/120

Epoch 101: val_loss improved from 0.02534 to 0.02518, saving model to data/lstm_by_cluster/cluster_1/best_model_1745208288.keras
195/195 - 2s - 8ms/step - loss: 0.0098 - mae: 0.0738 - val_loss: 0.0252 - val_mae: 0.142 - learning_rate: 3.1250e-05
Epoch 102/120

Epoch 102: val_loss did not improve from 0.02518
195/195 - 2s - 8ms/step - loss: 0.0100 - mae: 0.0747 - val_loss: 0.0254 - val_mae: 0.144 - learning_rate: 3.1250e-05
Epoch 103/120

Epoch 103: val_loss improved from 0.02518 to 0.02506, saving model to data/lstm_by_cluster/cluster_1/best_model_1745208288.keras
195/195 - 2s - 8ms/step - loss: 0.0096 - mae: 0.0732 - val_loss: 0.0251 - val_mae: 0.141 - learning_rate: 3.1250e-05
Epoch 104/120

Epoch 104: val_loss did not improve from 0.02506
195/195 - 2s - 8ms/step - loss: 0.0103 - mae: 0.0754 - val_loss: 0.0254 - val_mae: 0.144 - learning_rate: 3.1250e-05
Epoch 105/120

Epoch 105: val_loss improved from 0.02506 to 0.02501, saving model to data/lstm_by_cluster/cluster_1/best_model_1745208288.keras
195/195 - 2s - 9ms/step - loss: 0.0099 - mae: 0.0748 - val_loss: 0.0250 - val_mae: 0.135 - learning_rate: 3.1250e-05
Epoch 106/120

Epoch 106: val_loss did not improve from 0.02501
195/195 - 2s - 8ms/step - loss: 0.0097 - mae: 0.0736 - val_loss: 0.0250 - val_mae: 0.135 - learning_rate: 3.1250e-05
Epoch 107/120

Epoch 107: val_loss did not improve from 0.02501
195/195 - 2s - 8ms/step - loss: 0.0101 - mae: 0.0748 - val_loss: 0.0253 - val_mae: 0.143 - learning_rate: 3.1250e-05
Epoch 108/120

Epoch 108: val_loss did not improve from 0.02501
195/195 - 2s - 8ms/step - loss: 0.0096 - mae: 0.0741 - val_loss: 0.0252 - val_mae: 0.140 - learning_rate: 3.1250e-05
Epoch 109/120

Epoch 109: val_loss did not improve from 0.02501
195/195 - 2s - 8ms/step - loss: 0.0100 - mae: 0.0738 - val_loss: 0.0254 - val_mae: 0.136 - learning_rate: 3.1250e-05
Epoch 110/120

Epoch 110: val_loss did not improve from 0.02501
195/195 - 2s - 8ms/step - loss: 0.0097 - mae: 0.0741 - val_loss: 0.0254 - val_mae: 0.143 - learning_rate: 3.1250e-05
Epoch 111/120

Epoch 111: val_loss did not improve from 0.02501
Epoch 111: ReduceLROnPlateau reducing learning rate to 1.5625000742147677e-05.

195/195 - 2s - 8ms/step - loss: 0.0098 - mae: 0.0734 - val_loss: 0.0253 - val_mae: 0.137 - learning_rate: 3.1250e-05
Epoch 112/120

Epoch 112: val_loss did not improve from 0.02501
195/195 - 2s - 8ms/step - loss: 0.0101 - mae: 0.0744 - val_loss: 0.0254 - val_mae: 0.140 - learning_rate: 1.5625e-05
Epoch 113/120

Epoch 113: val_loss did not improve from 0.02501
195/195 - 2s - 8ms/step - loss: 0.0098 - mae: 0.0740 - val_loss: 0.0255 - val_mae: 0.141 - learning_rate: 1.5625e-05
Epoch 114/120

Epoch 114: val_loss did not improve from 0.02501
195/195 - 2s - 8ms/step - loss: 0.0097 - mae: 0.0733 - val_loss: 0.0256 - val_mae: 0.147 - learning_rate: 1.5625e-05
Epoch 115/120

Epoch 115: val_loss did not improve from 0.02501
195/195 - 2s - 8ms/step - loss: 0.0099 - mae: 0.0746 - val_loss: 0.0254 - val_mae: 0.144 - learning_rate: 1.5625e-05
Epoch 116/120

Epoch 116: val_loss did not improve from 0.02501
195/195 - 1s - 8ms/step - loss: 0.0097 - mae: 0.0739 - val_loss: 0.0253 - val_mae: 0.142 - learning_rate: 1.5625e-05
Epoch 117/120

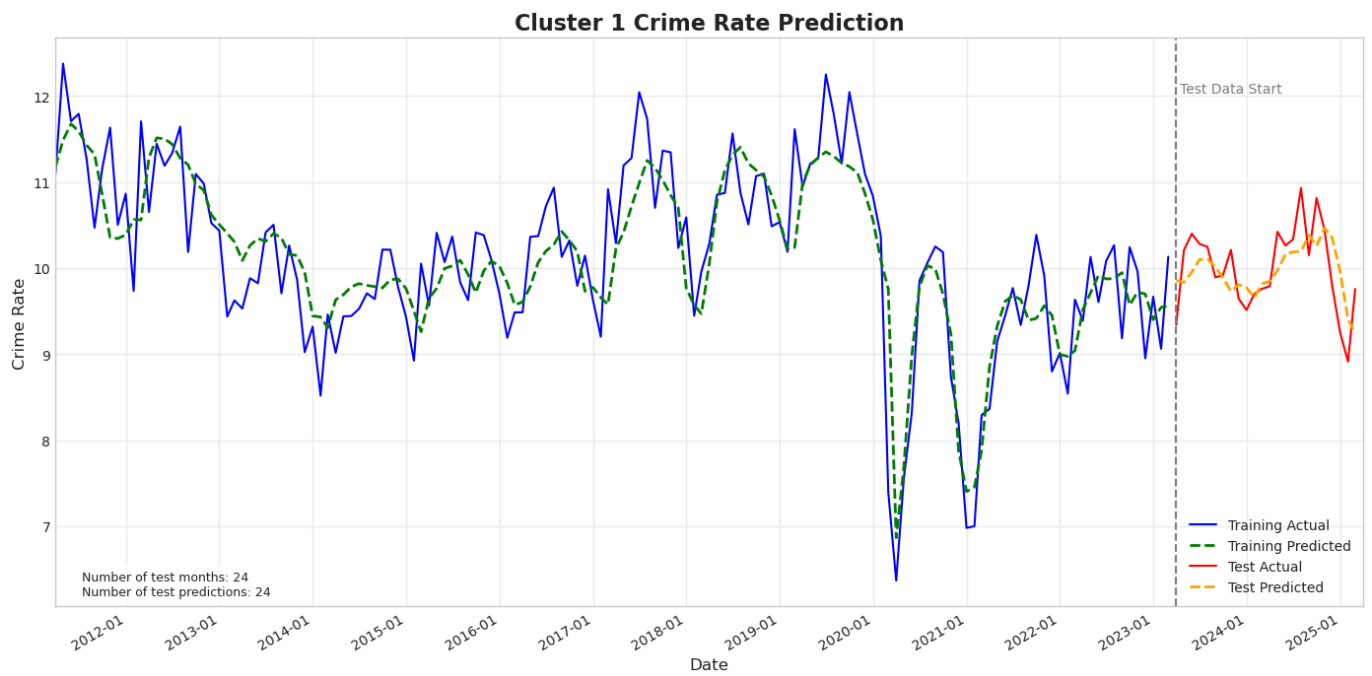
Epoch 117: val_loss did not improve from 0.02501
195/195 - 2s - 8ms/step - loss: 0.0101 - mae: 0.0748 - val_loss: 0.0251 - val_mae: 0.139 - learning_rate: 1.5625e-05
Epoch 118/120

Epoch 118: val_loss did not improve from 0.02501
195/195 - 2s - 8ms/step - loss: 0.0100 - mae: 0.0750 - val_loss: 0.0252 - val_mae: 0.142 - learning_rate: 1.5625e-05
Epoch 119/120

Epoch 119: val_loss did not improve from 0.02501

Epoch 119: ReduceLROnPlateau reducing learning rate to 1e-05.
195/195 - 2s - 8ms/step - loss: 0.0096 - mae: 0.0727 - val_loss: 0.0254 - val_mae: 0.144 - learning_rate: 1.5625e-05
Epoch 120/120

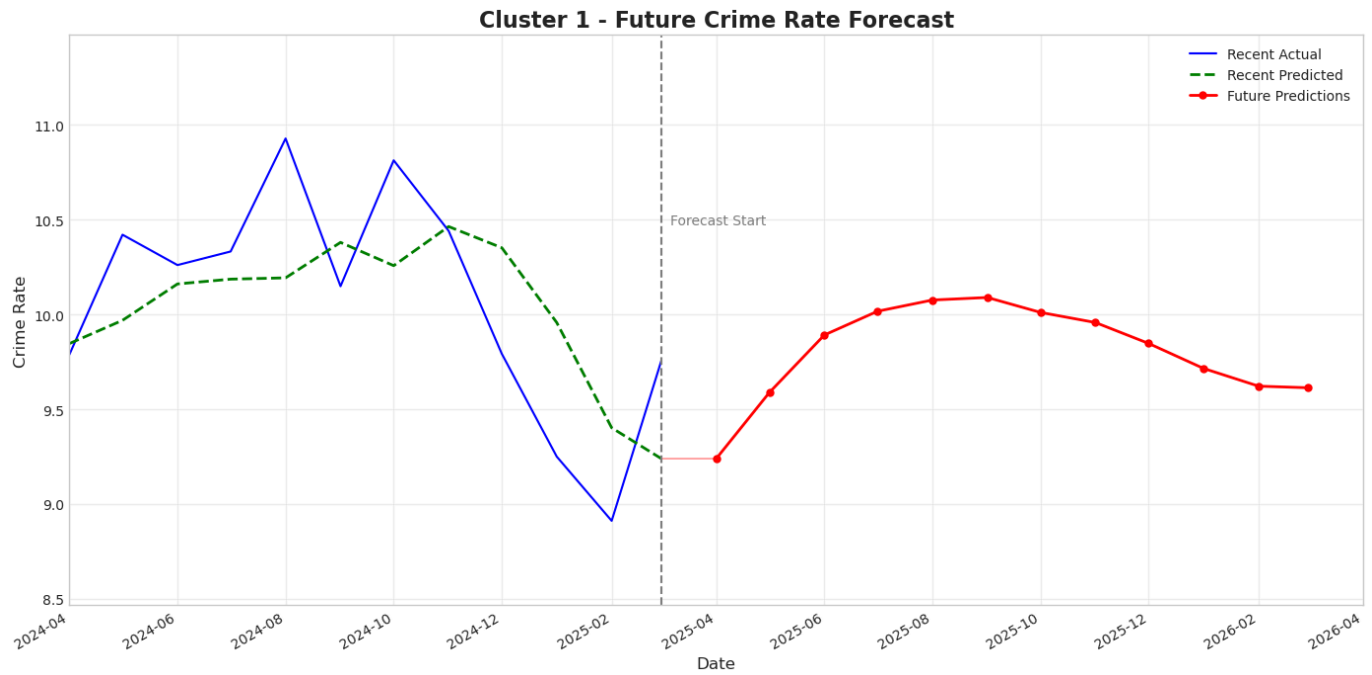
Epoch 120: val_loss did not improve from 0.02501
195/195 - 2s - 8ms/step - loss: 0.0094 - mae: 0.0733 - val_loss: 0.0253 - val_mae: 0.142 - learning_rate: 1.0000e-05
Evaluating cluster 1 LSTM model...
Test Loss: 0.0040
Test MAE: 0.0513
Training RMSE: 0.5350
Testing RMSE: 0.3802
Actual data start date: 2011-04-01 00:00:00



Last test date: 2025-03-01 00:00:00

Future date range: 2025-04-01 00:00:00 to 2026-03-01 00:00:00

<Figure size 1400x700 with 0 Axes>



Final model saved to data/lstm_by_cluster/cluster_1/final_model_1745208288.keras
Cleaning up resources for cluster 1...

=====
Processing Cluster 2
=====

X_train shape: (144, 12), y_train shape: (144,)
X_test shape: (24, 12), y_test shape: (24,)
Using enhanced multi-region specialized model for Cluster 2
Using specialized training for multi-region cluster
Advanced augmented training data: (144, 12, 1) → (864, 12, 1)
Augmented target data: (144, 1) → (864, 1)
Epoch 1/120

Epoch 1: val_loss improved from inf to 0.28687, saving model to data/lstm_by_cluster/cluster_2/best_model_1745208484.keras
195/195 - 5s - 25ms/step - loss: 0.3563 - mae: 0.1702 - val_loss: 0.2869 - val_mae: 0.1606 - learning_rate: 1.0000e-03
Epoch 2/120

Epoch 2: val_loss improved from 0.28687 to 0.20359, saving model to data/lstm_by_cluster/cluster_2/best_model_1745208484.keras
195/195 - 2s - 8ms/step - loss: 0.2354 - mae: 0.1299 - val_loss: 0.2036 - val_mae: 0.1474 - learning_rate: 1.0000e-03
Epoch 3/120

Epoch 3: val_loss improved from 0.20359 to 0.13796, saving model to data/lstm_by_cluster/cluster_2/best_model_1745208484.keras
195/195 - 2s - 8ms/step - loss: 0.1631 - mae: 0.1222 - val_loss: 0.1380 - val_mae: 0.1168 - learning_rate: 1.0000e-03
Epoch 4/120

Epoch 4: val_loss improved from 0.13796 to 0.09684, saving model to data/lstm_by_cluster/cluster_2/best_model_1745208484.keras
195/195 - 2s - 8ms/step - loss: 0.1117 - mae: 0.1202 - val_loss: 0.0968 - val_mae: 0.1148 - learning_rate: 1.0000e-03
Epoch 5/120

Epoch 5: val_loss improved from 0.09684 to 0.07112, saving model to data/lstm_by_cluster/cluster_2/best_model_1745208484.keras
195/195 - 2s - 8ms/step - loss: 0.0777 - mae: 0.1161 - val_loss: 0.0711 - val_mae: 0.1091 - learning_rate: 1.0000e-03
Epoch 6/120

Epoch 6: val_loss improved from 0.07112 to 0.05907, saving model to data/lstm_by_cluster/cluster_2/best_model_1745208484.keras
195/195 - 2s - 8ms/step - loss: 0.0555 - mae: 0.1144 - val_loss: 0.0591 - val_mae: 0.1127 - learning_rate: 1.0000e-03
Epoch 7/120

Epoch 7: val_loss improved from 0.05907 to 0.04309, saving model to data/lstm_by_cluster/cluster_2/best_model_1745208484.keras
195/195 - 2s - 8ms/step - loss: 0.0432 - mae: 0.1169 - val_loss: 0.0431 - val_mae: 0.1091 - learning_rate: 1.0000e-03
Epoch 8/120

Epoch 8: val_loss improved from 0.04309 to 0.04260, saving model to data/lstm_by_cluster/cluster_2/best_model_1745208484.keras
195/195 - 2s - 8ms/step - loss: 0.0334 - mae: 0.1129 - val_loss: 0.0426 - val_mae: 0.1152 - learning_rate: 1.0000e-03
Epoch 9/120

Epoch 9: val_loss improved from 0.04260 to 0.03656, saving model to data/lstm_by_cluster/cluster_2/best_model_1745208484.keras
195/195 - 2s - 8ms/step - loss: 0.0291 - mae: 0.1138 - val_loss: 0.0366 - val_mae: 0.1104 - learning_rate: 1.0000e-03
Epoch 10/120

Epoch 10: val_loss did not improve from 0.03656
195/195 - 2s - 8ms/step - loss: 0.0264 - mae: 0.1117 - val_loss: 0.0373 - val_mae: 0.1133 - learning_rate: 1.0000e-03
Epoch 11/120

Epoch 11: val_loss did not improve from 0.03656
195/195 - 2s - 8ms/step - loss: 0.0248 - mae: 0.1111 - val_loss: 0.0374 - val_mae: 0.1153 - learning_rate: 1.0000e-03
Epoch 12/120

Epoch 12: val_loss improved from 0.03656 to 0.03586, saving model to data/lstm_by_cluster/cluster_2/best_model_1745208484.keras
195/195 - 2s - 8ms/step - loss: 0.0242 - mae: 0.1122 - val_loss: 0.0359 - val_mae: 0.1134 - learning_rate: 1.0000e-03
Epoch 13/120

Epoch 13: val_loss did not improve from 0.03586
195/195 - 2s - 9ms/step - loss: 0.0233 - mae: 0.1101 - val_loss: 0.0359 - val_mae: 0.1142 - learning_rate: 1.0000e-03
Epoch 14/120

Epoch 14: val_loss improved from 0.03586 to 0.03464, saving model to data/lstm_by_cluster/cluster_2/best_model_1745208484.keras
195/195 - 2s - 8ms/step - loss: 0.0233 - mae: 0.1103 - val_loss: 0.0346 - val_mae: 0.1132 - learning_rate: 1.0000e-03
Epoch 15/120

Epoch 15: val_loss did not improve from 0.03464
195/195 - 2s - 8ms/step - loss: 0.0227 - mae: 0.1084 - val_loss: 0.0369 - val_mae: 0.1175 - learning_rate: 1.0000e-03
Epoch 16/120

Epoch 16: val_loss did not improve from 0.03464
195/195 - 2s - 8ms/step - loss: 0.0224 - mae: 0.1082 - val_loss: 0.0357 - val_mae: 0.1161 - learning_rate: 1.0000e-03
Epoch 17/120

Epoch 17: val_loss improved from 0.03464 to 0.03456, saving model to data/lstm_by_cluster/cluster_2/best_model_1745208484.keras
195/195 - 2s - 10ms/step - loss: 0.0220 - mae: 0.1084 - val_loss: 0.0346 - val_mae: 0.1168 - learning_rate: 1.0000e-03
Epoch 18/120

Epoch 18: val_loss improved from 0.03456 to 0.03441, saving model to data/lstm_by_cluster/cluster_2/best_model_1745208484.keras
195/195 - 2s - 9ms/step - loss: 0.0221 - mae: 0.1091 - val_loss: 0.0344 - val_mae: 0.1166 - learning_rate: 1.0000e-03
Epoch 19/120

Epoch 19: val_loss did not improve from 0.03441
195/195 - 2s - 9ms/step - loss: 0.0218 - mae: 0.1069 - val_loss: 0.0351 - val_mae: 0.1172 - learning_rate: 1.0000e-03
Epoch 20/120

Epoch 20: val_loss did not improve from 0.03441
195/195 - 2s - 8ms/step - loss: 0.0216 - mae: 0.1064 - val_loss: 0.0356 - val_mae: 0.1205 - learning_rate: 1.0000e-03
Epoch 21/120

Epoch 21: val_loss did not improve from 0.03441
195/195 - 2s - 8ms/step - loss: 0.0215 - mae: 0.1062 - val_loss: 0.0352 - val_mae: 0.1193 - learning_rate: 1.0000e-03
Epoch 22/120

Epoch 22: val_loss did not improve from 0.03441
195/195 - 2s - 11ms/step - loss: 0.0206 - mae: 0.1051 - val_loss: 0.0367 - val_mae: 0.1220 - learning_rate: 1.0000e-03
Epoch 23/120

Epoch 23: val_loss did not improve from 0.03441
195/195 - 2s - 8ms/step - loss: 0.0209 - mae: 0.1055 - val_loss: 0.0346 - val_mae: 0.1207 - learning_rate: 1.0000e-03
Epoch 24/120

Epoch 24: val_loss did not improve from 0.03441
195/195 - 2s - 10ms/step - loss: 0.0203 - mae: 0.1044 - val_loss: 0.0374 - val_mae: 0.1234 - learning_rate: 1.0000e-03
Epoch 25/120

Epoch 25: val_loss did not improve from 0.03441
195/195 - 2s - 8ms/step - loss: 0.0202 - mae: 0.1053 - val_loss: 0.0357 - val_mae: 0.1223 - learning_rate: 1.0000e-03
Epoch 26/120

Epoch 26: val_loss improved from 0.03441 to 0.03408, saving model to data/lstm_by_cluster/cluster_2/best_model_1745208484.keras
195/195 - 2s - 8ms/step - loss: 0.0201 - mae: 0.1050 - val_loss: 0.0341 - val_mae: 0.1205 - learning_rate: 1.0000e-03
Epoch 27/120

Epoch 27: val_loss did not improve from 0.03408
195/195 - 2s - 8ms/step - loss: 0.0195 - mae: 0.1022 - val_loss: 0.0346 - val_mae: 0.1217 - learning_rate: 1.0000e-03
Epoch 28/120

Epoch 28: val_loss did not improve from 0.03408
195/195 - 2s - 8ms/step - loss: 0.0197 - mae: 0.1034 - val_loss: 0.0350 - val_mae: 0.1208 - learning_rate: 1.0000e-03
Epoch 29/120

Epoch 29: val_loss did not improve from 0.03408
195/195 - 2s - 8ms/step - loss: 0.0198 - mae: 0.1045 - val_loss: 0.0360 - val_mae: 0.1224 - learning_rate: 1.0000e-03
Epoch 30/120

Epoch 30: val_loss did not improve from 0.03408
195/195 - 2s - 8ms/step - loss: 0.0198 - mae: 0.1028 - val_loss: 0.0350 - val_mae: 0.1193 - learning_rate: 1.0000e-03
Epoch 31/120

Epoch 31: val_loss did not improve from 0.03408
195/195 - 2s - 8ms/step - loss: 0.0205 - mae: 0.1047 - val_loss: 0.0362 - val_mae: 0.1236 - learning_rate: 1.0000e-03
Epoch 32/120

Epoch 32: val_loss did not improve from 0.03408
195/195 - 2s - 8ms/step - loss: 0.0195 - mae: 0.1041 - val_loss: 0.0367 - val_mae: 0.1221 - learning_rate: 1.0000e-03
Epoch 33/120

Epoch 33: val_loss did not improve from 0.03408
195/195 - 2s - 8ms/step - loss: 0.0191 - mae: 0.1013 - val_loss: 0.0377 - val_mae: 0.1253 - learning_rate: 1.0000e-03
Epoch 34/120

Epoch 34: val_loss did not improve from 0.03408

Epoch 34: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
195/195 - 2s - 8ms/step - loss: 0.0190 - mae: 0.1013 - val_loss: 0.0347 - val_mae: 0.1230 - learning_rate: 1.0000e-03
Epoch 35/120

Epoch 35: val_loss did not improve from 0.03408
195/195 - 2s - 8ms/step - loss: 0.0179 - mae: 0.1001 - val_loss: 0.0347 - val_mae: 0.1291 - learning_rate: 5.0000e-04
Epoch 36/120

Epoch 36: val_loss did not improve from 0.03408
195/195 - 2s - 8ms/step - loss: 0.0174 - mae: 0.0983 - val_loss: 0.0355 - val_mae: 0.1310 - learning_rate: 5.0000e-04
Epoch 37/120

Epoch 37: val_loss did not improve from 0.03408
195/195 - 2s - 8ms/step - loss: 0.0178 - mae: 0.0991 - val_loss: 0.0346 - val_mae: 0.1285 - learning_rate: 5.0000e-04
Epoch 38/120

Epoch 38: val_loss did not improve from 0.03408
195/195 - 2s - 8ms/step - loss: 0.0174 - mae: 0.0982 - val_loss: 0.0351 - val_mae: 0.1300 - learning_rate: 5.0000e-04
Epoch 39/120

Epoch 39: val_loss did not improve from 0.03408
195/195 - 1s - 8ms/step - loss: 0.0175 - mae: 0.0989 - val_loss: 0.0351 - val_mae: 0.1309 - learning_rate: 5.0000e-04
Epoch 40/120

Epoch 40: val_loss did not improve from 0.03408
195/195 - 2s - 8ms/step - loss: 0.0174 - mae: 0.0983 - val_loss: 0.0352 - val_mae: 0.1317 - learning_rate: 5.0000e-04
Epoch 41/120

Epoch 41: val_loss did not improve from 0.03408
195/195 - 2s - 8ms/step - loss: 0.0173 - mae: 0.0985 - val_loss: 0.0349 - val_mae: 0.1309 - learning_rate: 5.0000e-04
Epoch 42/120

Epoch 42: val_loss did not improve from 0.03408

Epoch 42: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.
195/195 - 2s - 8ms/step - loss: 0.0175 - mae: 0.0987 - val_loss: 0.0358 - val_mae: 0.1327 - learning_rate: 5.0000e-04
Epoch 43/120

Epoch 43: val_loss did not improve from 0.03408
195/195 - 2s - 8ms/step - loss: 0.0166 - mae: 0.0970 - val_loss: 0.0343 - val_mae: 0.1

309 - learning_rate: 2.5000e-04
Epoch 44/120

Epoch 44: val_loss did not improve from 0.03408
195/195 - 2s - 8ms/step - loss: 0.0169 - mae: 0.0980 - val_loss: 0.0344 - val_mae: 0.1
323 - learning_rate: 2.5000e-04
Epoch 45/120

Epoch 45: val_loss did not improve from 0.03408
195/195 - 2s - 8ms/step - loss: 0.0166 - mae: 0.0972 - val_loss: 0.0356 - val_mae: 0.1
338 - learning_rate: 2.5000e-04
Epoch 46/120

Epoch 46: val_loss improved from 0.03408 to 0.03364, saving model to data/lstm_by_cluster/cluster_2/best_model_1745208484.keras
195/195 - 2s - 8ms/step - loss: 0.0170 - mae: 0.0978 - val_loss: 0.0336 - val_mae: 0.1
304 - learning_rate: 2.5000e-04
Epoch 47/120

Epoch 47: val_loss did not improve from 0.03364
195/195 - 2s - 9ms/step - loss: 0.0167 - mae: 0.0975 - val_loss: 0.0341 - val_mae: 0.1
306 - learning_rate: 2.5000e-04
Epoch 48/120

Epoch 48: val_loss did not improve from 0.03364
195/195 - 2s - 8ms/step - loss: 0.0168 - mae: 0.0978 - val_loss: 0.0338 - val_mae: 0.1
307 - learning_rate: 2.5000e-04
Epoch 49/120

Epoch 49: val_loss did not improve from 0.03364
195/195 - 1s - 8ms/step - loss: 0.0163 - mae: 0.0963 - val_loss: 0.0338 - val_mae: 0.1
307 - learning_rate: 2.5000e-04
Epoch 50/120

Epoch 50: val_loss improved from 0.03364 to 0.03337, saving model to data/lstm_by_cluster/cluster_2/best_model_1745208484.keras
195/195 - 2s - 8ms/step - loss: 0.0164 - mae: 0.0956 - val_loss: 0.0334 - val_mae: 0.1
293 - learning_rate: 2.5000e-04
Epoch 51/120

Epoch 51: val_loss improved from 0.03337 to 0.03270, saving model to data/lstm_by_cluster/cluster_2/best_model_1745208484.keras
195/195 - 2s - 8ms/step - loss: 0.0166 - mae: 0.0960 - val_loss: 0.0327 - val_mae: 0.1
279 - learning_rate: 2.5000e-04
Epoch 52/120

Epoch 52: val_loss did not improve from 0.03270
195/195 - 2s - 8ms/step - loss: 0.0160 - mae: 0.0944 - val_loss: 0.0329 - val_mae: 0.1
286 - learning_rate: 2.5000e-04
Epoch 53/120

Epoch 53: val_loss improved from 0.03270 to 0.03190, saving model to data/lstm_by_cluster/cluster_2/best_model_1745208484.keras
195/195 - 2s - 8ms/step - loss: 0.0156 - mae: 0.0933 - val_loss: 0.0319 - val_mae: 0.1
264 - learning_rate: 2.5000e-04
Epoch 54/120

Epoch 54: val_loss improved from 0.03190 to 0.03135, saving model to data/lstm_by_cluster/cluster_2/best_model_1745208484.keras
195/195 - 2s - 8ms/step - loss: 0.0161 - mae: 0.0944 - val_loss: 0.0313 - val_mae: 0.1
243 - learning_rate: 2.5000e-04

Epoch 55/120

Epoch 55: val_loss did not improve from 0.03135

195/195 - 2s - 8ms/step - loss: 0.0156 - mae: 0.0933 - val_loss: 0.0323 - val_mae: 0.1254 - learning_rate: 2.5000e-04

Epoch 56/120

Epoch 56: val_loss did not improve from 0.03135

195/195 - 2s - 9ms/step - loss: 0.0160 - mae: 0.0930 - val_loss: 0.0318 - val_mae: 0.1252 - learning_rate: 2.5000e-04

Epoch 57/120

Epoch 57: val_loss did not improve from 0.03135

195/195 - 2s - 8ms/step - loss: 0.0151 - mae: 0.0905 - val_loss: 0.0314 - val_mae: 0.1247 - learning_rate: 2.5000e-04

Epoch 58/120

Epoch 58: val_loss did not improve from 0.03135

195/195 - 2s - 10ms/step - loss: 0.0155 - mae: 0.0919 - val_loss: 0.0314 - val_mae: 0.1223 - learning_rate: 2.5000e-04

Epoch 59/120

Epoch 59: val_loss improved from 0.03135 to 0.03096, saving model to data/lstm_by_cluster/cluster_2/best_model_1745208484.keras

195/195 - 2s - 9ms/step - loss: 0.0157 - mae: 0.0921 - val_loss: 0.0310 - val_mae: 0.1207 - learning_rate: 2.5000e-04

Epoch 60/120

Epoch 60: val_loss improved from 0.03096 to 0.03081, saving model to data/lstm_by_cluster/cluster_2/best_model_1745208484.keras

195/195 - 2s - 8ms/step - loss: 0.0153 - mae: 0.0903 - val_loss: 0.0308 - val_mae: 0.1198 - learning_rate: 2.5000e-04

Epoch 61/120

Epoch 61: val_loss did not improve from 0.03081

195/195 - 2s - 8ms/step - loss: 0.0153 - mae: 0.0903 - val_loss: 0.0311 - val_mae: 0.1210 - learning_rate: 2.5000e-04

Epoch 62/120

Epoch 62: val_loss improved from 0.03081 to 0.03057, saving model to data/lstm_by_cluster/cluster_2/best_model_1745208484.keras

195/195 - 2s - 8ms/step - loss: 0.0153 - mae: 0.0909 - val_loss: 0.0306 - val_mae: 0.1215 - learning_rate: 2.5000e-04

Epoch 63/120

Epoch 63: val_loss did not improve from 0.03057

195/195 - 1s - 8ms/step - loss: 0.0153 - mae: 0.0907 - val_loss: 0.0311 - val_mae: 0.1232 - learning_rate: 2.5000e-04

Epoch 64/120

Epoch 64: val_loss did not improve from 0.03057

195/195 - 2s - 8ms/step - loss: 0.0151 - mae: 0.0897 - val_loss: 0.0316 - val_mae: 0.1240 - learning_rate: 2.5000e-04

Epoch 65/120

Epoch 65: val_loss did not improve from 0.03057

195/195 - 2s - 8ms/step - loss: 0.0149 - mae: 0.0884 - val_loss: 0.0318 - val_mae: 0.1237 - learning_rate: 2.5000e-04

Epoch 66/120

Epoch 66: val_loss did not improve from 0.03057

195/195 - 1s - 8ms/step - loss: 0.0148 - mae: 0.0874 - val_loss: 0.0306 - val_mae: 0.1202 - learning_rate: 2.5000e-04
Epoch 67/120

Epoch 67: val_loss improved from 0.03057 to 0.03012, saving model to data/lstm_by_cluster/cluster_2/best_model_1745208484.keras

195/195 - 2s - 8ms/step - loss: 0.0147 - mae: 0.0871 - val_loss: 0.0301 - val_mae: 0.1202 - learning_rate: 2.5000e-04
Epoch 68/120

Epoch 68: val_loss did not improve from 0.03012

195/195 - 2s - 8ms/step - loss: 0.0145 - mae: 0.0861 - val_loss: 0.0307 - val_mae: 0.1201 - learning_rate: 2.5000e-04
Epoch 69/120

Epoch 69: val_loss did not improve from 0.03012

195/195 - 2s - 8ms/step - loss: 0.0150 - mae: 0.0872 - val_loss: 0.0308 - val_mae: 0.1199 - learning_rate: 2.5000e-04
Epoch 70/120

Epoch 70: val_loss did not improve from 0.03012

195/195 - 2s - 8ms/step - loss: 0.0146 - mae: 0.0866 - val_loss: 0.0309 - val_mae: 0.1207 - learning_rate: 2.5000e-04
Epoch 71/120

Epoch 71: val_loss did not improve from 0.03012

195/195 - 2s - 8ms/step - loss: 0.0144 - mae: 0.0844 - val_loss: 0.0316 - val_mae: 0.1229 - learning_rate: 2.5000e-04
Epoch 72/120

Epoch 72: val_loss did not improve from 0.03012

195/195 - 2s - 8ms/step - loss: 0.0143 - mae: 0.0846 - val_loss: 0.0313 - val_mae: 0.1224 - learning_rate: 2.5000e-04
Epoch 73/120

Epoch 73: val_loss did not improve from 0.03012

195/195 - 1s - 8ms/step - loss: 0.0143 - mae: 0.0855 - val_loss: 0.0304 - val_mae: 0.1202 - learning_rate: 2.5000e-04
Epoch 74/120

Epoch 74: val_loss did not improve from 0.03012

195/195 - 1s - 8ms/step - loss: 0.0141 - mae: 0.0842 - val_loss: 0.0301 - val_mae: 0.1200 - learning_rate: 2.5000e-04
Epoch 75/120

Epoch 75: val_loss did not improve from 0.03012

Epoch 75: ReduceLROnPlateau reducing learning rate to 0.0001250000059371814.

195/195 - 2s - 8ms/step - loss: 0.0139 - mae: 0.0834 - val_loss: 0.0303 - val_mae: 0.1185 - learning_rate: 2.5000e-04
Epoch 76/120

Epoch 76: val_loss did not improve from 0.03012

195/195 - 2s - 8ms/step - loss: 0.0136 - mae: 0.0833 - val_loss: 0.0311 - val_mae: 0.1210 - learning_rate: 1.2500e-04
Epoch 77/120

Epoch 77: val_loss did not improve from 0.03012

195/195 - 2s - 8ms/step - loss: 0.0139 - mae: 0.0840 - val_loss: 0.0302 - val_mae: 0.1194 - learning_rate: 1.2500e-04
Epoch 78/120

Epoch 78: val_loss did not improve from 0.03012
195/195 - 1s - 7ms/step - loss: 0.0135 - mae: 0.0828 - val_loss: 0.0301 - val_mae: 0.1191 - learning_rate: 1.2500e-04
Epoch 79/120

Epoch 79: val_loss did not improve from 0.03012
195/195 - 1s - 8ms/step - loss: 0.0134 - mae: 0.0817 - val_loss: 0.0309 - val_mae: 0.1213 - learning_rate: 1.2500e-04
Epoch 80/120

Epoch 80: val_loss did not improve from 0.03012
195/195 - 1s - 8ms/step - loss: 0.0134 - mae: 0.0825 - val_loss: 0.0306 - val_mae: 0.1200 - learning_rate: 1.2500e-04
Epoch 81/120

Epoch 81: val_loss did not improve from 0.03012
195/195 - 1s - 8ms/step - loss: 0.0135 - mae: 0.0821 - val_loss: 0.0306 - val_mae: 0.1202 - learning_rate: 1.2500e-04
Epoch 82/120

Epoch 82: val_loss improved from 0.03012 to 0.02975, saving model to data/lstm_by_cluster/cluster_2/best_model_1745208484.keras
195/195 - 2s - 8ms/step - loss: 0.0138 - mae: 0.0846 - val_loss: 0.0298 - val_mae: 0.1188 - learning_rate: 1.2500e-04
Epoch 83/120

Epoch 83: val_loss did not improve from 0.02975
195/195 - 2s - 8ms/step - loss: 0.0136 - mae: 0.0826 - val_loss: 0.0304 - val_mae: 0.1200 - learning_rate: 1.2500e-04
Epoch 84/120

Epoch 84: val_loss did not improve from 0.02975
195/195 - 2s - 8ms/step - loss: 0.0134 - mae: 0.0820 - val_loss: 0.0301 - val_mae: 0.1187 - learning_rate: 1.2500e-04
Epoch 85/120

Epoch 85: val_loss improved from 0.02975 to 0.02923, saving model to data/lstm_by_cluster/cluster_2/best_model_1745208484.keras
195/195 - 2s - 8ms/step - loss: 0.0132 - mae: 0.0812 - val_loss: 0.0292 - val_mae: 0.1169 - learning_rate: 1.2500e-04
Epoch 86/120

Epoch 86: val_loss did not improve from 0.02923
195/195 - 2s - 8ms/step - loss: 0.0136 - mae: 0.0833 - val_loss: 0.0296 - val_mae: 0.1187 - learning_rate: 1.2500e-04
Epoch 87/120

Epoch 87: val_loss did not improve from 0.02923
195/195 - 2s - 8ms/step - loss: 0.0133 - mae: 0.0811 - val_loss: 0.0293 - val_mae: 0.1172 - learning_rate: 1.2500e-04
Epoch 88/120

Epoch 88: val_loss did not improve from 0.02923
195/195 - 2s - 8ms/step - loss: 0.0135 - mae: 0.0822 - val_loss: 0.0302 - val_mae: 0.1208 - learning_rate: 1.2500e-04
Epoch 89/120

Epoch 89: val_loss did not improve from 0.02923
195/195 - 2s - 8ms/step - loss: 0.0133 - mae: 0.0818 - val_loss: 0.0303 - val_mae: 0.1202 - learning_rate: 1.2500e-04

Epoch 90/120

Epoch 90: val_loss did not improve from 0.02923

195/195 - 1s - 7ms/step - loss: 0.0132 - mae: 0.0821 - val_loss: 0.0301 - val_mae: 0.191 - learning_rate: 1.2500e-04

Epoch 91/120

Epoch 91: val_loss did not improve from 0.02923

195/195 - 1s - 8ms/step - loss: 0.0131 - mae: 0.0809 - val_loss: 0.0300 - val_mae: 0.192 - learning_rate: 1.2500e-04

Epoch 92/120

Epoch 92: val_loss did not improve from 0.02923

195/195 - 1s - 8ms/step - loss: 0.0134 - mae: 0.0817 - val_loss: 0.0306 - val_mae: 0.1203 - learning_rate: 1.2500e-04

Epoch 93/120

Epoch 93: val_loss did not improve from 0.02923

Epoch 93: ReduceLROnPlateau reducing learning rate to 6.25000029685907e-05.

195/195 - 1s - 8ms/step - loss: 0.0133 - mae: 0.0819 - val_loss: 0.0300 - val_mae: 0.187 - learning_rate: 1.2500e-04

Epoch 94/120

Epoch 94: val_loss did not improve from 0.02923

195/195 - 2s - 8ms/step - loss: 0.0133 - mae: 0.0820 - val_loss: 0.0296 - val_mae: 0.165 - learning_rate: 6.2500e-05

Epoch 95/120

Epoch 95: val_loss did not improve from 0.02923

195/195 - 2s - 8ms/step - loss: 0.0130 - mae: 0.0806 - val_loss: 0.0298 - val_mae: 0.165 - learning_rate: 6.2500e-05

Epoch 96/120

Epoch 96: val_loss did not improve from 0.02923

195/195 - 1s - 8ms/step - loss: 0.0129 - mae: 0.0805 - val_loss: 0.0294 - val_mae: 0.154 - learning_rate: 6.2500e-05

Epoch 97/120

Epoch 97: val_loss did not improve from 0.02923

195/195 - 2s - 8ms/step - loss: 0.0133 - mae: 0.0816 - val_loss: 0.0296 - val_mae: 0.162 - learning_rate: 6.2500e-05

Epoch 98/120

Epoch 98: val_loss did not improve from 0.02923

195/195 - 1s - 7ms/step - loss: 0.0129 - mae: 0.0809 - val_loss: 0.0298 - val_mae: 0.172 - learning_rate: 6.2500e-05

Epoch 99/120

Epoch 99: val_loss did not improve from 0.02923

195/195 - 2s - 8ms/step - loss: 0.0132 - mae: 0.0809 - val_loss: 0.0298 - val_mae: 0.163 - learning_rate: 6.2500e-05

Epoch 100/120

Epoch 100: val_loss did not improve from 0.02923

195/195 - 1s - 7ms/step - loss: 0.0130 - mae: 0.0811 - val_loss: 0.0300 - val_mae: 0.171 - learning_rate: 6.2500e-05

Epoch 101/120

Epoch 101: val_loss improved from 0.02923 to 0.02918, saving model to data/lstm_by_cluster/cluster_2/best_model_1745208484.keras

Epoch 101: ReduceLROnPlateau reducing learning rate to 3.125000148429535e-05.
195/195 - 2s - 8ms/step - loss: 0.0131 - mae: 0.0800 - val_loss: 0.0292 - val_mae: 0.148 - learning_rate: 6.2500e-05
Epoch 102/120

Epoch 102: val_loss did not improve from 0.02918
195/195 - 2s - 8ms/step - loss: 0.0131 - mae: 0.0809 - val_loss: 0.0292 - val_mae: 0.132 - learning_rate: 3.1250e-05
Epoch 103/120

Epoch 103: val_loss improved from 0.02918 to 0.02906, saving model to data/lstm_by_cluster/cluster_2/best_model_1745208484.keras
195/195 - 2s - 8ms/step - loss: 0.0130 - mae: 0.0800 - val_loss: 0.0291 - val_mae: 0.133 - learning_rate: 3.1250e-05
Epoch 104/120

Epoch 104: val_loss improved from 0.02906 to 0.02891, saving model to data/lstm_by_cluster/cluster_2/best_model_1745208484.keras
195/195 - 2s - 8ms/step - loss: 0.0128 - mae: 0.0797 - val_loss: 0.0289 - val_mae: 0.125 - learning_rate: 3.1250e-05
Epoch 105/120

Epoch 105: val_loss did not improve from 0.02891
195/195 - 2s - 8ms/step - loss: 0.0126 - mae: 0.0796 - val_loss: 0.0291 - val_mae: 0.129 - learning_rate: 3.1250e-05
Epoch 106/120

Epoch 106: val_loss did not improve from 0.02891
195/195 - 2s - 8ms/step - loss: 0.0130 - mae: 0.0805 - val_loss: 0.0293 - val_mae: 0.141 - learning_rate: 3.1250e-05
Epoch 107/120

Epoch 107: val_loss did not improve from 0.02891
195/195 - 1s - 8ms/step - loss: 0.0130 - mae: 0.0806 - val_loss: 0.0291 - val_mae: 0.137 - learning_rate: 3.1250e-05
Epoch 108/120

Epoch 108: val_loss did not improve from 0.02891
195/195 - 2s - 8ms/step - loss: 0.0129 - mae: 0.0804 - val_loss: 0.0291 - val_mae: 0.136 - learning_rate: 3.1250e-05
Epoch 109/120

Epoch 109: val_loss did not improve from 0.02891
195/195 - 1s - 8ms/step - loss: 0.0132 - mae: 0.0812 - val_loss: 0.0291 - val_mae: 0.134 - learning_rate: 3.1250e-05
Epoch 110/120

Epoch 110: val_loss did not improve from 0.02891
195/195 - 1s - 8ms/step - loss: 0.0132 - mae: 0.0814 - val_loss: 0.0291 - val_mae: 0.136 - learning_rate: 3.1250e-05
Epoch 111/120

Epoch 111: val_loss did not improve from 0.02891
195/195 - 1s - 8ms/step - loss: 0.0131 - mae: 0.0816 - val_loss: 0.0290 - val_mae: 0.130 - learning_rate: 3.1250e-05
Epoch 112/120

Epoch 112: val_loss did not improve from 0.02891

Epoch 112: ReduceLROnPlateau reducing learning rate to 1.5625000742147677e-05.

195/195 - 2s - 8ms/step - loss: 0.0129 - mae: 0.0800 - val_loss: 0.0291 - val_mae: 0.130 - learning_rate: 3.1250e-05
Epoch 113/120

Epoch 113: val_loss did not improve from 0.02891
195/195 - 2s - 8ms/step - loss: 0.0127 - mae: 0.0803 - val_loss: 0.0291 - val_mae: 0.128 - learning_rate: 1.5625e-05
Epoch 114/120

Epoch 114: val_loss did not improve from 0.02891
195/195 - 1s - 8ms/step - loss: 0.0131 - mae: 0.0806 - val_loss: 0.0292 - val_mae: 0.132 - learning_rate: 1.5625e-05
Epoch 115/120

Epoch 115: val_loss did not improve from 0.02891
195/195 - 2s - 8ms/step - loss: 0.0126 - mae: 0.0801 - val_loss: 0.0294 - val_mae: 0.133 - learning_rate: 1.5625e-05
Epoch 116/120

Epoch 116: val_loss did not improve from 0.02891
195/195 - 2s - 8ms/step - loss: 0.0126 - mae: 0.0795 - val_loss: 0.0294 - val_mae: 0.137 - learning_rate: 1.5625e-05
Epoch 117/120

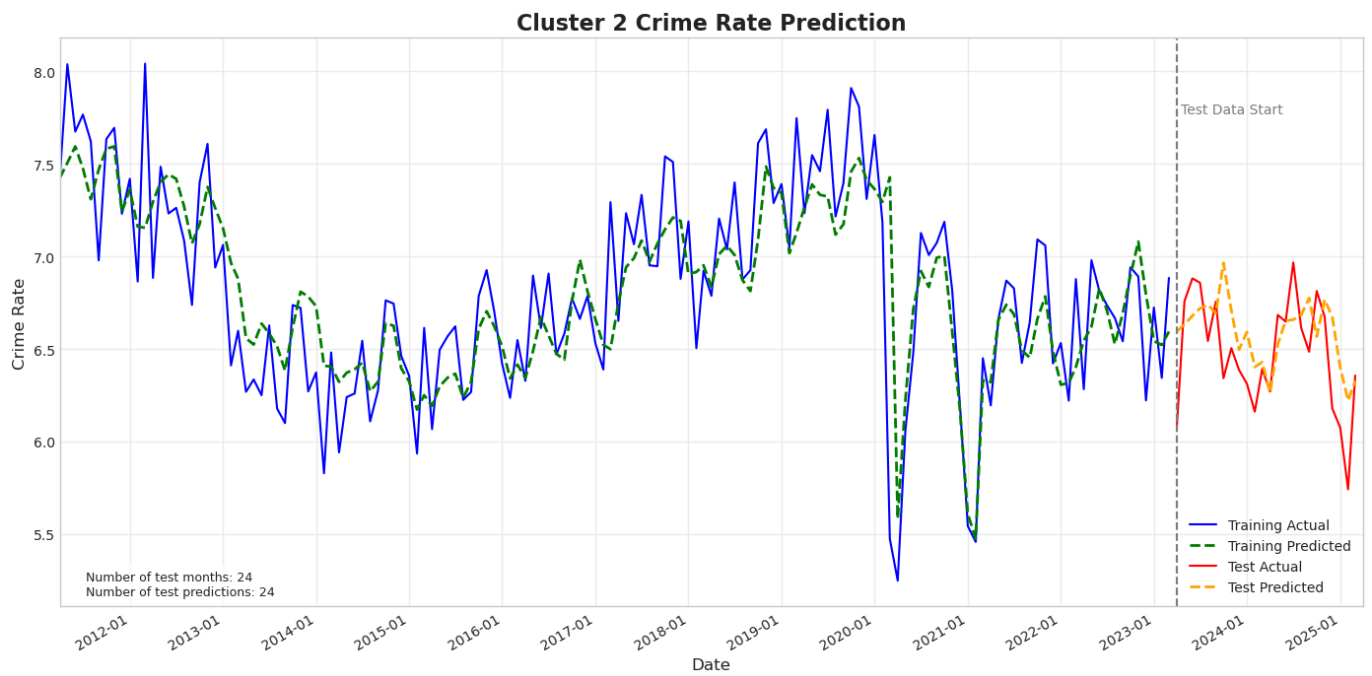
Epoch 117: val_loss did not improve from 0.02891
195/195 - 1s - 7ms/step - loss: 0.0128 - mae: 0.0808 - val_loss: 0.0293 - val_mae: 0.133 - learning_rate: 1.5625e-05
Epoch 118/120

Epoch 118: val_loss did not improve from 0.02891
195/195 - 1s - 8ms/step - loss: 0.0129 - mae: 0.0797 - val_loss: 0.0292 - val_mae: 0.132 - learning_rate: 1.5625e-05
Epoch 119/120

Epoch 119: val_loss did not improve from 0.02891
195/195 - 1s - 7ms/step - loss: 0.0129 - mae: 0.0807 - val_loss: 0.0290 - val_mae: 0.123 - learning_rate: 1.5625e-05
Epoch 120/120

Epoch 120: val_loss did not improve from 0.02891

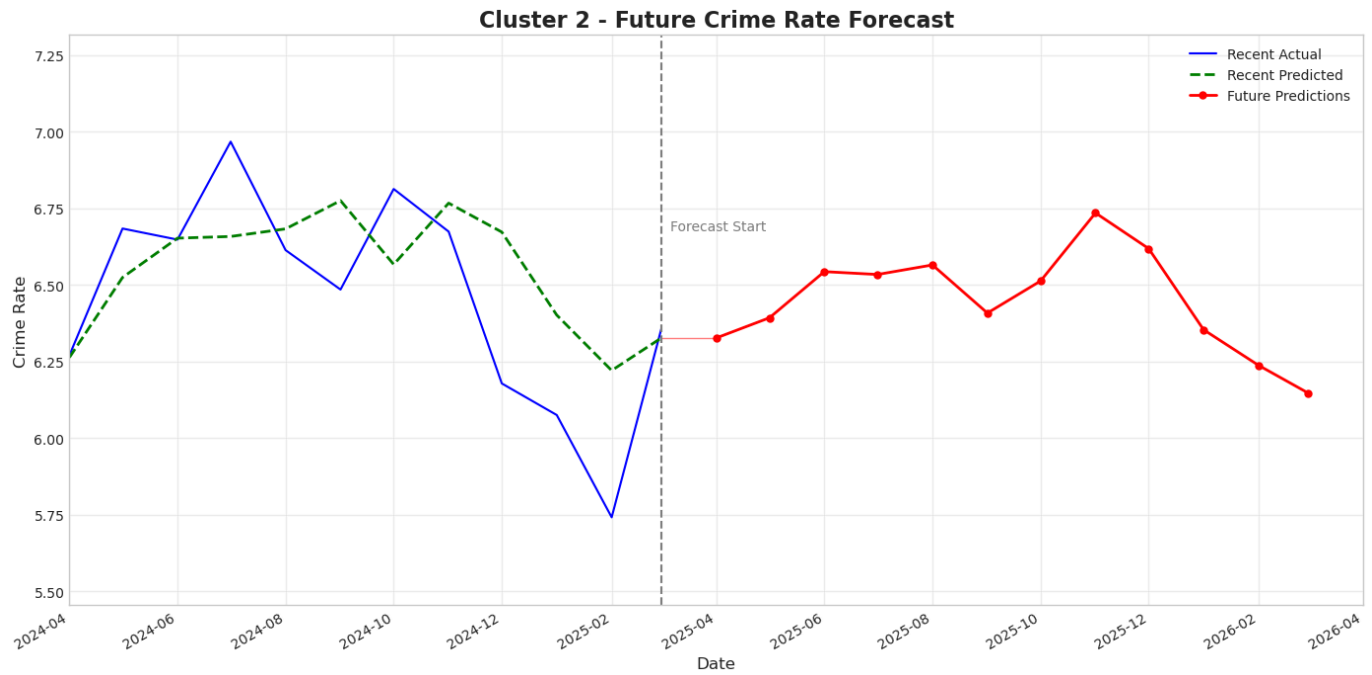
Epoch 120: ReduceLROnPlateau reducing learning rate to 1e-05.
195/195 - 2s - 8ms/step - loss: 0.0127 - mae: 0.0794 - val_loss: 0.0291 - val_mae: 0.125 - learning_rate: 1.5625e-05
Evaluating cluster 2 LSTM model...
Test Loss: 0.0091
Test MAE: 0.0753
Training RMSE: 0.3075
Testing RMSE: 0.2759
Actual data start date: 2011-04-01 00:00:00
<Figure size 1400x700 with 0 Axes>



Last test date: 2025-03-01 00:00:00

Future date range: 2025-04-01 00:00:00 to 2026-03-01 00:00:00

<Figure size 1400x700 with 0 Axes>



Final model saved to data/lstm_by_cluster/cluster_2/final_model_1745208484.keras
Cleaning up resources for cluster 2...

=====
Processing Cluster 3
=====

X_train shape: (144, 12), y_train shape: (144,)
X_test shape: (24, 12), y_test shape: (24,)
Using complex attention model for Cluster 3 (Westminster)
Using standard training for Cluster 3 (Westminster)
Epoch 1/50

Epoch 1: val_loss improved from inf to 0.05063, saving model to data/lstm_by_cluster/cluster_3/best_model_1745208679.keras
4/4 - 3s - 649ms/step - loss: 0.3351 - mae: 0.4853 - val_loss: 0.0506 - val_mae: 0.1771
Epoch 2/50

Epoch 2: val_loss did not improve from 0.05063
4/4 - 0s - 52ms/step - loss: 0.0422 - mae: 0.1716 - val_loss: 0.0780 - val_mae: 0.2428
Epoch 3/50

Epoch 3: val_loss did not improve from 0.05063
4/4 - 0s - 53ms/step - loss: 0.0387 - mae: 0.1522 - val_loss: 0.0639 - val_mae: 0.1918
Epoch 4/50

Epoch 4: val_loss did not improve from 0.05063
4/4 - 0s - 46ms/step - loss: 0.0321 - mae: 0.1490 - val_loss: 0.0593 - val_mae: 0.1980
Epoch 5/50

Epoch 5: val_loss did not improve from 0.05063
4/4 - 0s - 51ms/step - loss: 0.0294 - mae: 0.1334 - val_loss: 0.0570 - val_mae: 0.1882
Epoch 6/50

Epoch 6: val_loss did not improve from 0.05063
4/4 - 0s - 61ms/step - loss: 0.0293 - mae: 0.1402 - val_loss: 0.0550 - val_mae: 0.1865
Epoch 7/50

Epoch 7: val_loss did not improve from 0.05063
4/4 - 0s - 52ms/step - loss: 0.0258 - mae: 0.1249 - val_loss: 0.0532 - val_mae: 0.1824
Epoch 8/50

Epoch 8: val_loss did not improve from 0.05063
4/4 - 0s - 65ms/step - loss: 0.0267 - mae: 0.1312 - val_loss: 0.0510 - val_mae: 0.1785
Epoch 9/50

Epoch 9: val_loss improved from 0.05063 to 0.04787, saving model to data/lstm_by_cluster/cluster_3/best_model_1745208679.keras
4/4 - 0s - 61ms/step - loss: 0.0248 - mae: 0.1218 - val_loss: 0.0479 - val_mae: 0.1726
Epoch 10/50

Epoch 10: val_loss improved from 0.04787 to 0.04657, saving model to data/lstm_by_cluster/cluster_3/best_model_1745208679.keras
4/4 - 0s - 55ms/step - loss: 0.0267 - mae: 0.1313 - val_loss: 0.0466 - val_mae: 0.1698
Epoch 11/50

Epoch 11: val_loss improved from 0.04657 to 0.04582, saving model to data/lstm_by_cluster/cluster_3/best_model_1745208679.keras
4/4 - 0s - 58ms/step - loss: 0.0254 - mae: 0.1239 - val_loss: 0.0458 - val_mae: 0.1689
Epoch 12/50

Epoch 12: val_loss improved from 0.04582 to 0.04193, saving model to data/lstm_by_cluster/cluster_3/best_model_1745208679.keras
4/4 - 0s - 57ms/step - loss: 0.0246 - mae: 0.1182 - val_loss: 0.0419 - val_mae: 0.1617
Epoch 13/50

Epoch 13: val_loss improved from 0.04193 to 0.04081, saving model to data/lstm_by_cluster/cluster_3/best_model_1745208679.keras
4/4 - 0s - 57ms/step - loss: 0.0245 - mae: 0.1198 - val_loss: 0.0408 - val_mae: 0.1651
Epoch 14/50

Epoch 14: val_loss did not improve from 0.04081
4/4 - 0s - 45ms/step - loss: 0.0251 - mae: 0.1245 - val_loss: 0.0444 - val_mae: 0.1747
Epoch 15/50

Epoch 15: val_loss improved from 0.04081 to 0.03471, saving model to data/lstm_by_cluster/cluster_3/best_model_1745208679.keras
4/4 - 0s - 55ms/step - loss: 0.0232 - mae: 0.1149 - val_loss: 0.0347 - val_mae: 0.1451
Epoch 16/50

Epoch 16: val_loss improved from 0.03471 to 0.03132, saving model to data/lstm_by_cluster/cluster_3/best_model_1745208679.keras
4/4 - 0s - 56ms/step - loss: 0.0198 - mae: 0.1050 - val_loss: 0.0313 - val_mae: 0.1432
Epoch 17/50

Epoch 17: val_loss did not improve from 0.03132
4/4 - 0s - 48ms/step - loss: 0.0215 - mae: 0.1126 - val_loss: 0.0412 - val_mae: 0.1709
Epoch 18/50

Epoch 18: val_loss did not improve from 0.03132
4/4 - 0s - 50ms/step - loss: 0.0265 - mae: 0.1263 - val_loss: 0.0421 - val_mae: 0.1728
Epoch 19/50

Epoch 19: val_loss improved from 0.03132 to 0.02670, saving model to data/lstm_by_cluster/cluster_3/best_model_1745208679.keras
4/4 - 0s - 68ms/step - loss: 0.0204 - mae: 0.1036 - val_loss: 0.0267 - val_mae: 0.1303
Epoch 20/50

Epoch 20: val_loss did not improve from 0.02670
4/4 - 0s - 52ms/step - loss: 0.0169 - mae: 0.0960 - val_loss: 0.0303 - val_mae: 0.1453
Epoch 21/50

Epoch 21: val_loss did not improve from 0.02670
4/4 - 0s - 74ms/step - loss: 0.0187 - mae: 0.1009 - val_loss: 0.0360 - val_mae: 0.1586
Epoch 22/50

Epoch 22: val_loss did not improve from 0.02670
4/4 - 0s - 48ms/step - loss: 0.0192 - mae: 0.1006 - val_loss: 0.0389 - val_mae: 0.1629
Epoch 23/50

Epoch 23: val_loss did not improve from 0.02670
4/4 - 0s - 45ms/step - loss: 0.0177 - mae: 0.0942 - val_loss: 0.0366 - val_mae: 0.1574
Epoch 24/50

Epoch 24: val_loss did not improve from 0.02670
4/4 - 0s - 50ms/step - loss: 0.0161 - mae: 0.0894 - val_loss: 0.0300 - val_mae: 0.1401
Epoch 25/50

Epoch 25: val_loss did not improve from 0.02670
4/4 - 0s - 65ms/step - loss: 0.0124 - mae: 0.0759 - val_loss: 0.0295 - val_mae: 0.1369
Epoch 26/50

Epoch 26: val_loss did not improve from 0.02670
4/4 - 0s - 52ms/step - loss: 0.0154 - mae: 0.0842 - val_loss: 0.0347 - val_mae: 0.1515
Epoch 27/50

Epoch 27: val_loss did not improve from 0.02670
4/4 - 0s - 55ms/step - loss: 0.0169 - mae: 0.0931 - val_loss: 0.0394 - val_mae: 0.1635
Epoch 28/50

Epoch 28: val_loss improved from 0.02670 to 0.02632, saving model to data/lstm_by_cluster/cluster_3/best_model_1745208679.keras
4/4 - 0s - 72ms/step - loss: 0.0171 - mae: 0.0948 - val_loss: 0.0263 - val_mae: 0.1320
Epoch 29/50

Epoch 29: val_loss did not improve from 0.02632
4/4 - 0s - 63ms/step - loss: 0.0127 - mae: 0.0778 - val_loss: 0.0281 - val_mae: 0.1346
Epoch 30/50

Epoch 30: val_loss did not improve from 0.02632
4/4 - 0s - 50ms/step - loss: 0.0135 - mae: 0.0806 - val_loss: 0.0317 - val_mae: 0.1446
Epoch 31/50

Epoch 31: val_loss did not improve from 0.02632
4/4 - 0s - 51ms/step - loss: 0.0140 - mae: 0.0852 - val_loss: 0.0271 - val_mae: 0.1339
Epoch 32/50

Epoch 32: val_loss improved from 0.02632 to 0.02523, saving model to data/lstm_by_cluster/cluster_3/best_model_1745208679.keras
4/4 - 0s - 59ms/step - loss: 0.0129 - mae: 0.0779 - val_loss: 0.0252 - val_mae: 0.1308
Epoch 33/50

Epoch 33: val_loss improved from 0.02523 to 0.02348, saving model to data/lstm_by_cluster/cluster_3/best_model_1745208679.keras
4/4 - 0s - 60ms/step - loss: 0.0113 - mae: 0.0724 - val_loss: 0.0235 - val_mae: 0.1252
Epoch 34/50

Epoch 34: val_loss did not improve from 0.02348
4/4 - 0s - 55ms/step - loss: 0.0122 - mae: 0.0753 - val_loss: 0.0266 - val_mae: 0.1333
Epoch 35/50

Epoch 35: val_loss did not improve from 0.02348
4/4 - 0s - 61ms/step - loss: 0.0132 - mae: 0.0788 - val_loss: 0.0264 - val_mae: 0.1321
Epoch 36/50

Epoch 36: val_loss improved from 0.02348 to 0.02229, saving model to data/lstm_by_cluster/cluster_3/best_model_1745208679.keras
4/4 - 0s - 81ms/step - loss: 0.0126 - mae: 0.0762 - val_loss: 0.0223 - val_mae: 0.1219
Epoch 37/50

Epoch 37: val_loss did not improve from 0.02229
4/4 - 0s - 52ms/step - loss: 0.0110 - mae: 0.0746 - val_loss: 0.0223 - val_mae: 0.1217
Epoch 38/50

Epoch 38: val_loss did not improve from 0.02229
4/4 - 0s - 53ms/step - loss: 0.0117 - mae: 0.0761 - val_loss: 0.0234 - val_mae: 0.1246
Epoch 39/50

Epoch 39: val_loss did not improve from 0.02229
4/4 - 0s - 51ms/step - loss: 0.0130 - mae: 0.0814 - val_loss: 0.0230 - val_mae: 0.1237
Epoch 40/50

Epoch 40: val_loss improved from 0.02229 to 0.02067, saving model to data/lstm_by_cluster/cluster_3/best_model_1745208679.keras
4/4 - 0s - 51ms/step - loss: 0.0103 - mae: 0.0701 - val_loss: 0.0207 - val_mae: 0.1187
Epoch 41/50

ter/cluster_3/best_model_1745208679.keras
4/4 - 0s - 61ms/step - loss: 0.0113 - mae: 0.0749 - val_loss: 0.0207 - val_mae: 0.1170
Epoch 41/50

Epoch 41: val_loss did not improve from 0.02067
4/4 - 0s - 53ms/step - loss: 0.0102 - mae: 0.0693 - val_loss: 0.0209 - val_mae: 0.1174
Epoch 42/50

Epoch 42: val_loss did not improve from 0.02067
4/4 - 0s - 52ms/step - loss: 0.0117 - mae: 0.0711 - val_loss: 0.0217 - val_mae: 0.1200
Epoch 43/50

Epoch 43: val_loss did not improve from 0.02067
4/4 - 0s - 50ms/step - loss: 0.0119 - mae: 0.0778 - val_loss: 0.0211 - val_mae: 0.1178
Epoch 44/50

Epoch 44: val_loss did not improve from 0.02067
4/4 - 0s - 51ms/step - loss: 0.0113 - mae: 0.0720 - val_loss: 0.0223 - val_mae: 0.1218
Epoch 45/50

Epoch 45: val_loss improved from 0.02067 to 0.01957, saving model to data/lstm_by_cluster/cluster_3/best_model_1745208679.keras
4/4 - 0s - 67ms/step - loss: 0.0105 - mae: 0.0724 - val_loss: 0.0196 - val_mae: 0.1135
Epoch 46/50

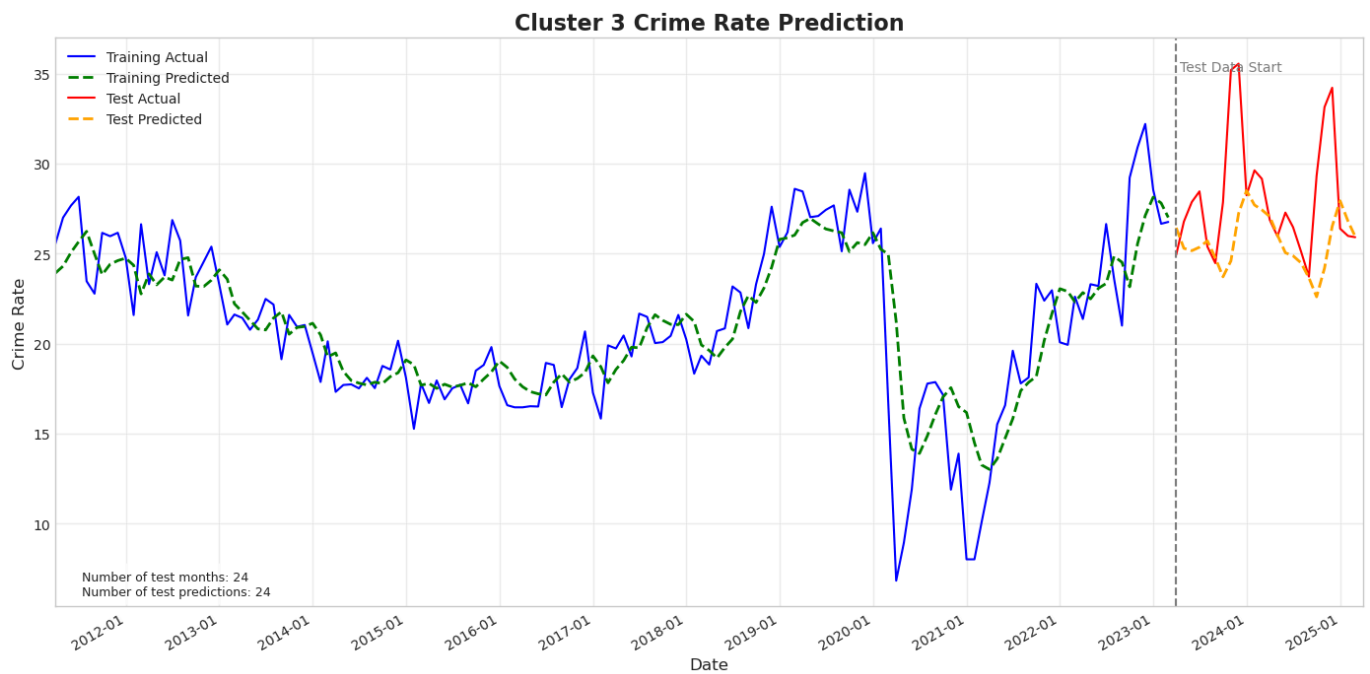
Epoch 46: val_loss did not improve from 0.01957
4/4 - 0s - 56ms/step - loss: 0.0094 - mae: 0.0651 - val_loss: 0.0211 - val_mae: 0.1193
Epoch 47/50

Epoch 47: val_loss did not improve from 0.01957
4/4 - 0s - 65ms/step - loss: 0.0127 - mae: 0.0769 - val_loss: 0.0265 - val_mae: 0.1340
Epoch 48/50

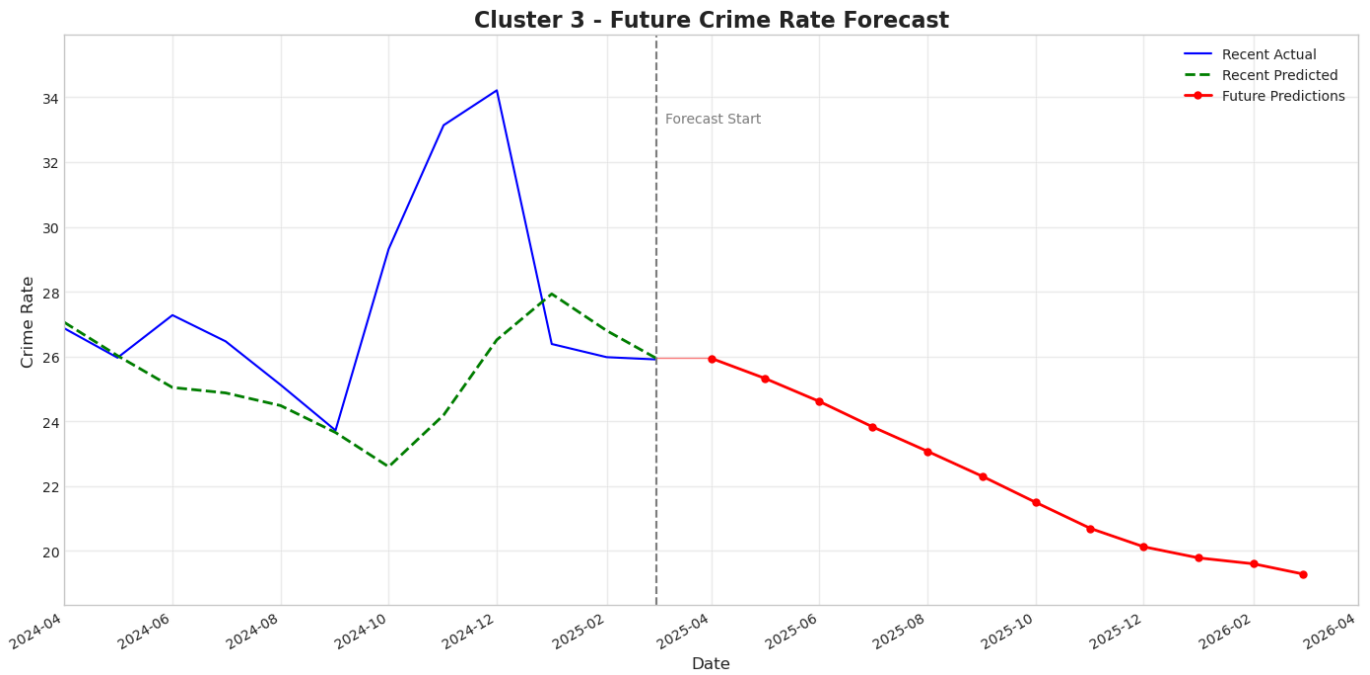
Epoch 48: val_loss did not improve from 0.01957
4/4 - 0s - 57ms/step - loss: 0.0136 - mae: 0.0825 - val_loss: 0.0199 - val_mae: 0.1133
Epoch 49/50

Epoch 49: val_loss improved from 0.01957 to 0.01879, saving model to data/lstm_by_cluster/cluster_3/best_model_1745208679.keras
4/4 - 0s - 62ms/step - loss: 0.0101 - mae: 0.0733 - val_loss: 0.0188 - val_mae: 0.1070
Epoch 50/50

Epoch 50: val_loss did not improve from 0.01879
4/4 - 0s - 67ms/step - loss: 0.0094 - mae: 0.0670 - val_loss: 0.0203 - val_mae: 0.1101
Evaluating cluster 3 LSTM model...
Test Loss: 0.0274
Test MAE: 0.1099
Training RMSE: 2.6370
Testing RMSE: 4.1978
Actual data start date: 2011-04-01 00:00:00
<Figure size 1400x700 with 0 Axes>

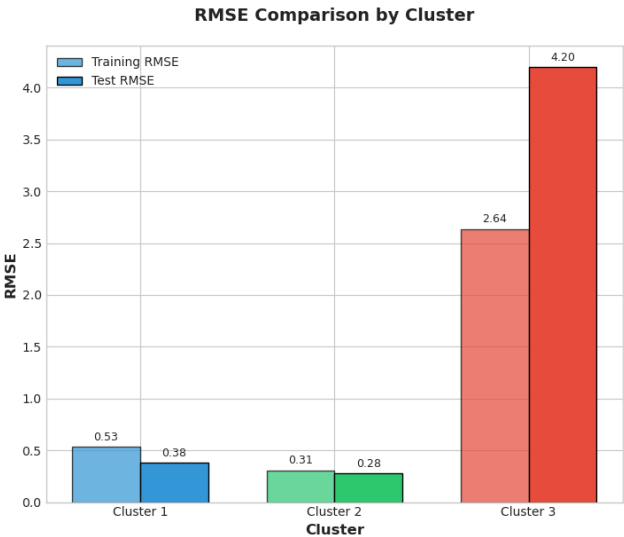


Last test date: 2025-03-01 00:00:00
 Future date range: 2025-04-01 00:00:00 to 2026-03-01 00:00:00
 <Figure size 1400x700 with 0 Axes>



Final model saved to data/lstm_by_cluster/cluster_3/final_model_1745208679.keras
 Cleaning up resources for cluster 3...
 <Figure size 1400x700 with 0 Axes>

LSTM Model Performance Comparison Across Clusters



Model Performance Metrics

Cluster	Model Type	Test RMSE	Test MAE	Train RMSE
Cluster 1	Simple (Single LSTM)	0.3802	0.0513	0.5350
Cluster 2	Medium (Stacked LSTM)	0.2759	0.0753	0.3075
Cluster 3	Complex (Bidirectional LSTM with Attention)	4.1978	0.1099	2.6370

Model Evaluation Summary:

	ClusterID	TestRMSE	TestMAE	TestLoss	TrainRMSE \	ModelType
0	1	0.380205	0.051289	0.004018	0.534978	Simple (Single LSTM)
1	2	0.275949	0.075346	0.009065	0.307517	Medium (Stacked LSTM)
2	3	4.197816	0.109909	0.027356	2.637016	Complex (Bidirectional LSTM with Attention)

LSTM model training and evaluation complete!

```
In [15]: end_time = time.time()
runtime_hours = (end_time - start_time) / 3600
print(f"Runtime: {runtime_hours:.2f} hours")
```

Runtime: 0.12 hours

Results and discussion

[\[go back to the top \]](#)

Cime Prediction Results

The LSTM model demonstrates robust prediction capability for Cluster 1 (eight inner London boroughs), with crime rates ranging primarily between 9.0 and 11.0. The model successfully captures the general cyclic pattern in the historical data (2012-2024) and medium-term trends with reasonable accuracy. The future forecast (April 2025-2026) predicts moderate fluctuations with crime rates mainly between 9.3 and 10.1, showing a slight overall decrease by April 2026.

For Cluster 2 (23 outer London boroughs), which exhibits the lowest crime rates among the three clusters (approximately 5.7-7.0), the model shows effective capture of the historical data's volatility pattern and good approximation of seasonal variations. The future forecast suggests a gradual increase in crime rates from April to August 2025 (peaking at approximately 6.6), followed by a steady decline until January 2026 (approximately 6.3), with a slight uptick by April 2026. This cluster demonstrates the most stable forecast pattern.

Cluster 3 (Westminster) exhibits the highest crime rates (ranging from approximately 17 to 35) and the most dramatic forecast trajectory. The model has captured the high volatility in the historical data and tracked significant trend shifts in the testing period. The future forecast for Cluster 3 shows a consistent, steep downward trend from April 2025 (approximately 25.5) to April 2026 (approximately 17.5), representing a roughly 31% decrease.

LSTM Model Performance Evaluation

The evaluation metrics reveal distinct performance patterns across the three clusters, each modeled with different LSTM architectures. The medium-complexity stacked LSTM applied to Cluster 2 achieved the lowest test error, suggesting optimal complexity for capturing this cluster's temporal crime patterns without overfitting. The minimal gap between training and test errors further indicates robust generalization capabilities.

Cluster 1, analyzed using a simpler single LSTM structure, shows moderate predictive accuracy. While adequate for modeling primary patterns, this architecture may miss certain temporal nuances present in the data.

Despite employing sophisticated architecture—a bidirectional LSTM with attention mechanism, Cluster 3 (Westminster) exhibits notably higher error metrics. This suggests that even advanced neural network architectures face limitations when confronted with highly dynamic crime environments.

Limitations

This study faces several limitations. While LSTM models effectively capture temporal patterns, they do not account for external influences such as policy shifts, economic change, or demographic trends. This is especially evident in Cluster 3, where high volatility limits the model's accuracy despite its complexity, hinting at factors beyond time-series structure.

The analysis's temporal resolution may also miss short-term fluctuations that are important for early intervention. Moreover, the clustering approach, while computationally efficient, may group areas with differing crime dynamics, reducing model sensitivity to local variations.

Last but not the least, as the prediction window extends, uncertainty increases. Projections beyond six months are best viewed as general trends rather than precise estimates.

Conclusion

[\[go back to the top \]](#)

To conclude, this research demonstrates that cluster-based LSTM models offer valuable insights for crime rate forecasting across London boroughs. Performance metrics revealed significant variations across the three identified clusters, with medium-complexity architectures achieving optimal results for moderately volatile areas, while even sophisticated models faced challenges with high-volatility crime environments.

Additionally, it indicates that effective prediction requires tailoring model complexity to each cluster's specific temporal patterns rather than applying uniform approaches. While models show promise for short-term forecasting, their predictive capacity diminishes over longer horizons, particularly in high-volatility areas. This limitation highlights the need for regular recalibration and the integration of domain knowledge when interpreting extended projections.

References

[\[go back to the top \]](#)

- [1] Vijayarani, S., Suganya, E. and Navya, C., 2021. Crime analysis and prediction using enhanced Arima model. Journal homepage: www.ijrpr.com ISSN, 2582, p.7421.
- [2] Faujdar, N., Verma, Y., Rathore, Y.S. and Rohatgi, P.K., 2021. Crime Forecasting Using Time Series Analysis. *Advances in Interdisciplinary Research in Engineering and Business Management*, pp.253-262.
- [3] Safat, W., Asghar, S. and Gillani, S.A., 2021. Empirical analysis for crime prediction and forecasting using machine learning and deep learning techniques. *IEEE access*, 9, pp.70080-70094.
- [4] Agarwal, J., Nagpal, R. and Sehgal, R., 2013. Crime analysis using k-means clustering. *International Journal of Computer Applications*, 83(4).
- [5] Nath, S.V., 2006, December. Crime pattern detection using data mining. In *2006 IEEE/WIC/ACM International Conference on Web Intelligence and Intelligent Agent Technology Workshops* (pp. 41-44). IEEE.
- [6] Mishra, R.K., Ansari, A.R., Jothi, J.A.A. and Mishra, V., 2024. Analysis of criminal landscape by utilizing statistical analysis and deep learning techniques. *Journal of Applied Security Research*, 19(4), pp.560-585.